

MCS-220

WEB TECHNOLOGIES

By Ritesh Tiwari

December, 2021

Qus 1.

(a) What is the design pattern ? What are the advantages of using patterns ? Explain UML diagram of Factory Design Pattern.

Ans :

Design Pattern :

A **Design Pattern** is a **reusable solution** to a **common problem** that occurs in software design.

It is **not code**, but rather a **blueprint or template** for how to solve a particular design issue in different situations.

➡ In short:

A design pattern is a general, reusable solution for solving common object-oriented design problems.

Advantages of Using Design Patterns

Advantage	Explanation
1. Reusability	Patterns can be reused across multiple projects, saving development time.
2. Maintainability	They make code structure clear, which simplifies debugging and updates.

- 3. Scalability** Patterns help design systems that can grow easily without major rework.
- 4. Standardization** They provide a common vocabulary for developers, improving communication.
- 5. Flexibility** Patterns promote loose coupling between classes and make it easy to modify behavior.
- 6. Best Practices** They embody proven solutions tested by experts, ensuring better software design.

Factory Design Pattern (Creational Pattern)

Definition

The **Factory Pattern** is a **Creational Design Pattern** that provides an interface for creating objects **without specifying their exact class**.

In other words, the **Factory Method** lets a class **delegate the instantiation** of objects to subclasses or a separate factory class.

Real-life Example

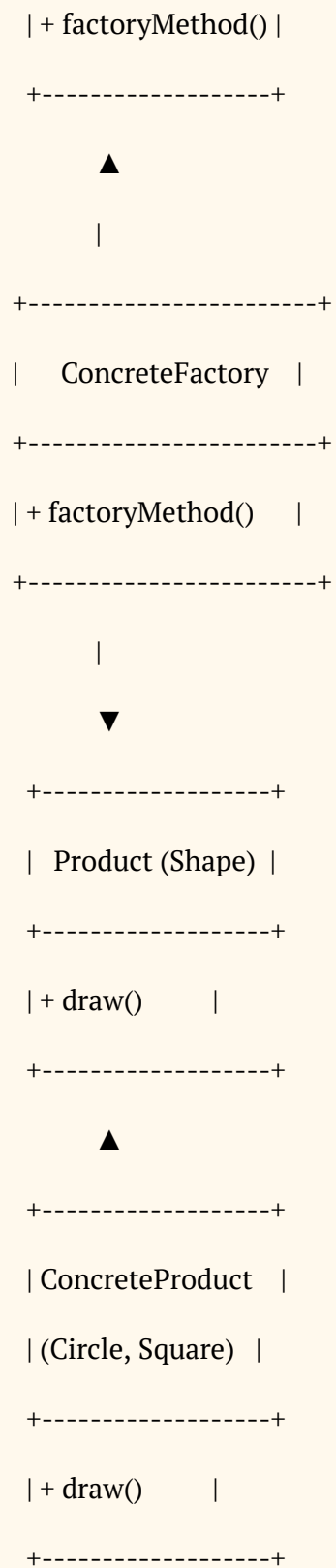
Suppose you are creating a **Shape** interface with multiple implementations like **Circle**, **Rectangle**, and **Square**.

Instead of creating these objects directly using `new`, you can use a **ShapeFactory** class that decides which object to create.

UML Diagram of Factory Design Pattern

Here's how the **UML diagram** looks:

```
+-----+
| Creator (Factory)|
+-----+
```



Example in Java

```
// Step 1: Create an interface
interface Shape {
    void draw();
}

// Step 2: Create concrete classes implementing the same interface
class Circle implements Shape {
    public void draw() {
        System.out.println("Drawing a Circle");
    }
}

class Square implements Shape {
    public void draw() {
        System.out.println("Drawing a Square");
    }
}

// Step 3: Create a Factory class
class ShapeFactory {
    public Shape getShape(String shapeType) {
        if (shapeType == null) {
            return null;
        }
        if (shapeType.equalsIgnoreCase("CIRCLE")) {
            return new Circle();
        } else if (shapeType.equalsIgnoreCase("SQUARE")) {
            return new Square();
        }
        return null;
    }
}

// Step 4: Use the Factory to get object
public class FactoryPatternDemo {
    public static void main(String[] args) {
        ShapeFactory shapeFactory = new ShapeFactory();

        Shape shape1 = shapeFactory.getShape("CIRCLE");
        shape1.draw();
    }
}
```

```

        Shape shape2 = shapeFactory.getShape("SQUARE");
        shape2.draw();
    }
}

```

Summary

Aspect	Factory Pattern
Type	Creational Pattern
Purpose	To create objects without specifying the exact class
Key Benefit	Promotes loose coupling between client code and concrete classes
Common Use	In frameworks, where you want to delegate object creation to a factory class

(b) What is Servlet Programming ? Briefly explain servlet life cycle. Also write a servlet which displays “welcome to web programming.”

Ans :

What is Servlet Programming?

A **Servlet** is a **Java program** that runs on a **web server** and is used to create **dynamic web content**.

It acts as a **middle layer** between a **client request (browser)** and a **server-side resource** such as a database or another application.

➤ Definition:

A servlet is a server-side Java component that handles client requests and generates responses, usually in the form of HTML.

Features of Servlet Programming

Feature	Description
Platform independent	Written in Java, so it runs on any platform with a JVM.
Efficient	Servlets are loaded once and reused for multiple requests.
Secure	Supports HTTPS, authentication, and session management.
Extensible	Easy to integrate with other Java technologies like JSP, JDBC, etc.

Servlet Life Cycle

The **Servlet Life Cycle** is managed by the **Servlet Container** (like Apache Tomcat). It defines how a servlet is **loaded, initialized, executed, and destroyed**.

Phases of Servlet Life Cycle

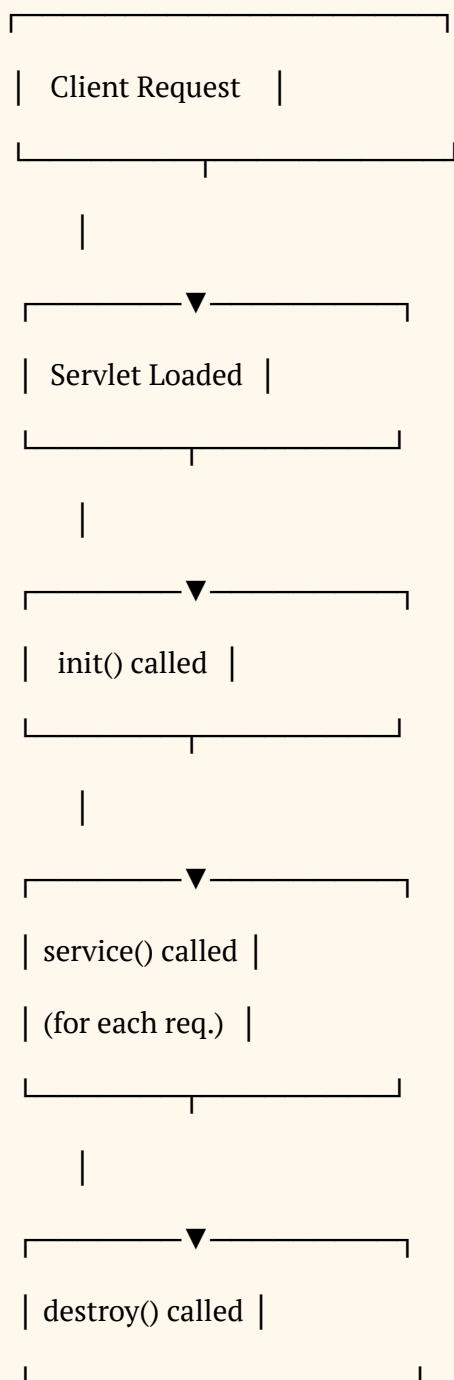
Phase	Method	Description
1. Loading and Instantiation	—	Servlet class is loaded into memory by the container.
2. Initialization	<code>init()</code>	Called once when the servlet is first created; used for initialization.
3. Request Handling	<code>service()</code>	Called for each client request ; handles logic and generates a response.

4. Destruction

`destroy`
`()`

Called once before servlet is unloaded; used to release resources.

Servlet Life Cycle Diagram



Example: Servlet Program

Goal:

Display the message → **Welcome to Web Programming.**

Code:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class WelcomeServlet extends HttpServlet {

    // Step 1: Initialization (optional)
    public void init() throws ServletException {
        System.out.println("Servlet is initializing...");
    }

    // Step 2: Service method (handles requests)
    public void doGet(HttpServletRequest request, HttpServletResponse
response)
        throws ServletException, IOException {

        // Set response content type
        response.setContentType("text/html");

        // Write HTML response
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("<h2>Welcome to Web Programming.</h2>");
        out.println("</body></html>");
    }

    // Step 3: Destroy method (cleanup)
    public void destroy() {
        System.out.println("Servlet is being destroyed...");
    }
}
```


Configuration in `web.xml` (Deployment Descriptor)

```
<web-app>
  <servlet>
    <servlet-name>welcome</servlet-name>
    <servlet-class>WelcomeServlet</servlet-class>
  </servlet>

  <servlet-mapping>
    <servlet-name>welcome</servlet-name>
    <url-pattern>/welcome</url-pattern>
  </servlet-mapping>
</web-app>
```

How it Works

1. The servlet container (e.g., **Apache Tomcat**) loads `WelcomeServlet.class`.
2. It calls the `init()` method once.
3. When a client visits `http://localhost:8080/yourapp/welcome`, the container calls the `service()` method.
4. The response `Welcome to Web Programming.` is displayed in the browser.
5. When the server stops or servlet is removed, `destroy()` is called.

Summary

Aspect	Explanation
Servlet Type	Server-side Java component
Managed by	Servlet Container
Lifecycle Methods	<code>init()</code> , <code>service()</code> , <code>destroy()</code>
Output	Dynamic HTML or data response

Example Output “Welcome to Web Programming.”

(c) What is JSP ? What are JSP directives ? Explain the use of any two JSP directives with the help of a program.

Ans :

What is JSP (Java Server Pages)?

JSP (Java Server Pages) is a **server-side technology** used to create **dynamic web pages** using **Java**.

It allows mixing of **HTML** and **Java code** inside a single page.

➤ Definition:

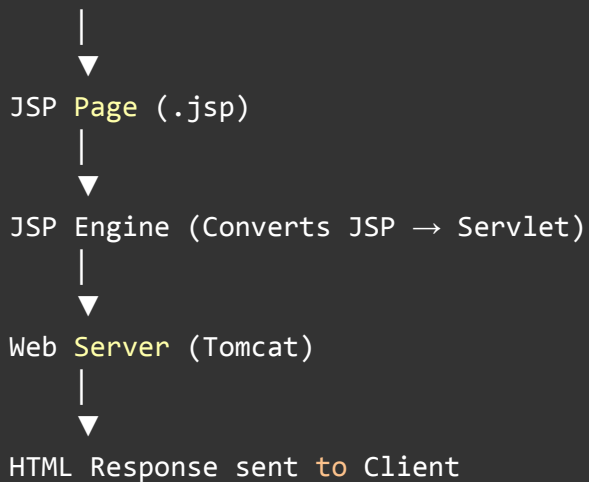
JSP is a technology that helps developers create dynamically generated web pages based on HTML, XML, or other document types using Java.

Features of JSP

Feature	Description
Dynamic content generation	Can include Java code directly inside HTML.
Reusable components	Can include JavaBeans and tag libraries.
Automatic compilation	JSPs are automatically converted into Servlets by the server.
Easy to maintain	Logic (Java) and presentation (HTML) can be separated easily.

JSP Architecture

Browser (Client)



What are JSP Directives?

JSP Directives provide **instructions** to the JSP engine about how to **process the JSP page**. They are used to **set page-level**, **include external files**, and **define tag libraries**.

Syntax:

```
<%@ directive attribute="value" %>
```

Types of JSP Directives

Directive	Purpose
1. Page Directive	Defines page-level information (language, import, error page, etc.)
2. Include Directive	Includes external files (HTML, JSP, etc.) at page translation time
3. Taglib Directive	Declares a tag library used in the JSP page

Explanation of Two JSP Directives

A. Page Directive

Purpose:

Used to define attributes that apply to the whole JSP page such as importing packages, error pages, or content type.

Syntax:

```
<%@ page attribute="value" %>
```

Common Attributes:

Attribute	Meaning
<code>language</code>	Defines the scripting language (usually Java).
<code>import</code>	Imports Java packages (like in Java code).
<code>contentType</code>	Sets MIME type of the response.
<code>errorPage</code>	Specifies another JSP to display in case of an error.

✓ Example 1: Using `page` Directive

File: `welcome.jsp`

```
<%@ page language="java" contentType="text/html" import="java.util.*" %>
<html>
<body>
    <h2>Welcome to JSP Page</h2>
    <%
        Date today = new Date();
        out.println("Current Date and Time: " + today);
    %>
</body>
</html>
```

● Explanation:

- The `page` directive imports the `java.util.*` package.
- The JSP scriptlet prints the current date and time dynamically.

B. Include Directive

Purpose:

Used to **include static or dynamic content** (like a header, footer, or menu) at **translation time** (before the page runs).

Syntax:

```
<%@ include file="filename.jsp" %>
```

Example 2: Using `include` Directive

File 1: `header.jsp`

```
<h2>My Website Header</h2>
<hr>
```

File 2: `main.jsp`

```
<%@ include file="header.jsp" %>
<html>
<body>
    <h3>Welcome to Web Programming</h3>
</body>
</html>
```

● Explanation:

- The `include` directive includes the contents of `header.jsp` inside `main.jsp`.

- The output will show both header and main content together.

Summary Table

Directive	Syntax	Purpose	Example
page	<code><%@ page attribute="value" %></code>	Defines page-level settings	<code><%@ page import="java.util.*" %></code>
include	<code><%@ include file="header.jsp" %></code>	Includes other JSP files	<code><%@ include file="menu.jsp" %></code>
taglib	<code><%@ taglib uri="..." prefix="..." %></code>	Declares custom tag libraries	<code><%@ taglib uri="mytags" prefix="t" %></code>

Final Output (for Example 2)

My Website Header

Welcome to Web Programming

(d) Explain process of setting up database using Hibernate.

Ans :

What is Hibernate?

Hibernate is an **Object Relational Mapping (ORM)** framework for Java.

It allows developers to **map Java objects to database tables** and handle database operations without writing SQL queries directly.

➡ In short:

Hibernate helps connect Java applications with databases by converting Java classes into database tables automatically.

Steps to Set Up a Database Using Hibernate

Let's understand the complete **process of setting up and connecting Hibernate with a database**.

Step 1: Add Hibernate Libraries

You need to include the **Hibernate JAR files** in your project.

If you're using Maven, add this dependency in your `pom.xml`:

```
<dependency>
  <groupId>org.hibernate.orm</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>6.3.1.Final</version>
</dependency>
```

If not using Maven, download Hibernate from hibernate.org and add the JARs to your project's `lib` folder.

Also include:

- JDBC driver for your database (e.g., `mysql-connector-j.jar`)

Step 2: Create Database and Table (Optional)

Example: Create a **MySQL** database named `studentdb`

```
CREATE DATABASE studentdb;
```

Hibernate can also **auto-create** tables based on your Java classes if configured properly (see step 5).

Step 3: Create Hibernate Configuration File

Create a file named **hibernate.cfg.xml** inside the **src** folder.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 5.3//EN"
    "http://hibernate.sourceforge.net/hibernate-configuration-5.3.dtd">

<hibernate-configuration>
    <session-factory>

        <!-- Database Connection Settings -->
        <property
name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
        <property
name="hibernate.connection.url">jdbc:mysql://localhost:3306/studentdb</prop
erty>
        <property name="hibernate.connection.username">root</property>
        <property
name="hibernate.connection.password">yourpassword</property>

        <!-- SQL Dialect -->
        <property
name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>

        <!-- Show SQL in console -->
        <property name="hibernate.show_sql">true</property>

        <!-- Automatically create/update tables -->
        <property name="hibernate.hbm2ddl.auto">update</property>

        <!-- Mapping class -->
        <mapping class="com.example.Student"/>

    </session-factory>
</hibernate-configuration>
```


 Note:

`hibernate.hbm2ddl.auto` can be:

- `create` → Creates tables every time
- `update` → Updates tables if needed
- `validate` → Validates schema
- `none` → Does nothing

Step 4: Create a Persistent Class

Create a simple Java class that will be mapped to a table.

File: `Student.java`

```
package com.example;

import jakarta.persistence.*;

@Entity
@Table(name = "student")
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    @Column(name = "name")
    private String name;

    @Column(name = "course")
    private String course;

    public Student() {}
    public Student(String name, String course) {
        this.name = name;
        this.course = course;
    }

    // Getters and Setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
```

```

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getCourse() { return course; }
    public void setCourse(String course) { this.course = course; }
}

```

● Here:

- `@Entity` marks the class as a database entity.
- `@Table` defines the table name.
- `@Id` and `@GeneratedValue` define the primary key.

Step 5: Create a Hibernate Utility Class

To manage sessions, create a helper class:

File: `HibernateUtil.java`

```

package com.example;

import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public class HibernateUtil {
    private static SessionFactory sessionFactory;

    static {
        try {
            sessionFactory = new
Configuration().configure().buildSessionFactory();
        } catch (Throwable ex) {
            System.err.println("SessionFactory creation failed." + ex);
            throw new ExceptionInInitializerError(ex);
        }
    }

    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }
}

```

```
}
```

Step 6: Create a Main Class to Insert Data

File: `MainApp.java`

```
package com.example;

import org.hibernate.Session;
import org.hibernate.Transaction;

public class MainApp {
    public static void main(String[] args) {
        Session session = HibernateUtil.getSessionFactory().openSession();
        Transaction tx = session.beginTransaction();

        // Create new student object
        Student student = new Student("Ritesh Tiwari", "Web Programming");
        session.save(student); // Insert record into database

        tx.commit();
        session.close();

        System.out.println("Record inserted successfully!");
    }
}
```

Execution Flow

1. Hibernate reads `hibernate.cfg.xml`.
2. Builds a `SessionFactory` (represents database connection).
3. Opens a `Session` (like JDBC connection).
4. Creates or updates table automatically based on `@Entity`.
5. Saves Java object (`Student`) into database table.
6. Closes session and transaction.

Output

If you check your database:

```
SELECT * FROM student;
```

You'll see a record like:

id	name	course
1	Ritesh Tiwari	Web Programming

✓ Summary

Step	Task	File/Component
1	Add Hibernate JARs	Maven or libs
2	Create Database	MySQL/PostgreSQL
3	Configure Hibernate	<code>hibernate.cfg.xml</code>
4	Create Entity Class	<code>Student.java</code>
5	Create Utility Class	<code>HibernateUtil.java</code>
6	Write Main Program	<code>MainApp.java</code>
7	Run and Verify	Check DB table

Qus 2.

(a) Explain process of creating records using Spring Boot and Hibernate.

Ans :

What is Spring Boot with Hibernate?

Spring Boot is a framework that simplifies the setup and development of **Spring-based applications**.

Hibernate is an **ORM (Object Relational Mapping)** framework that maps **Java classes to database tables**.

➡ When combined:

Spring Boot + Hibernate allows you to easily perform **CRUD operations** (Create, Read, Update, Delete) on a database with minimal configuration.

Process of Creating Records Using Spring Boot and Hibernate

Let's understand each step involved in setting up and inserting records.

Step 1: Create a Spring Boot Project

You can create a new Spring Boot project using:

- [Spring Initializr](#)
- or any IDE like IntelliJ / Eclipse.

✓ Project Dependencies

Add the following dependencies:

- **Spring Web**
- **Spring Data JPA**
- **MySQL Driver** (or your preferred DB)

If you're using Maven, your `pom.xml` should include:

```
<dependencies>
```

```

<!-- Spring Boot Starter Web -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<!-- Spring Boot Starter JPA -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<!-- MySQL Database Driver -->
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
</dependency>
</dependencies>

```

Step 2: Configure Database Connection

In `src/main/resources/application.properties` (or `.yaml`), add:

```

# Database Configuration
spring.datasource.url=jdbc:mysql://localhost:3306/studentdb
spring.datasource.username=root
spring.datasource.password=yourpassword

# JPA / Hibernate Configuration
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect

```

Note:

`spring.jpa.hibernate.ddl-auto=update` automatically creates or updates tables.

Step 3: Create Entity Class

This class represents your database table.

File: `Student.java`

```
package com.example.demo.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "students")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String course;

    public Student() {}

    public Student(String name, String course) {
        this.name = name;
        this.course = course;
    }

    // Getters and Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getCourse() { return course; }
    public void setCourse(String course) { this.course = course; }
}
```

 Hibernate automatically maps:

- Class → Table (`Student` → `students`)
- Fields → Columns (`name`, `course`, etc.)

Step 4: Create Repository Interface

This interface helps you perform CRUD operations without writing SQL.

File: `StudentRepository.java`

```
package com.example.demo.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.example.demo.entity.Student;

public interface StudentRepository extends JpaRepository<Student, Long> {
}
```

`JpaRepository` provides built-in methods like `save()`, `findAll()`, `deleteById()`, etc.

Step 5: Create Service Class (Optional)

Used for business logic between controller and repository.

File: `StudentService.java`

```
package com.example.demo.service;

import com.example.demo.entity.Student;
import com.example.demo.repository.StudentRepository;
import org.springframework.stereotype.Service;

@Service
public class StudentService {

    private final StudentRepository studentRepository;

    public StudentService(StudentRepository studentRepository) {
        this.studentRepository = studentRepository;
    }
}
```



```
    public Student saveStudent(Student student) {  
        return studentRepository.save(student); // Insert record into DB  
    }  
}
```

Step 6: Create Controller

Handles HTTP requests (e.g., POST request to add a student).

File: `StudentController.java`

```
package com.example.demo.controller;  
  
import com.example.demo.entity.Student;  
import com.example.demo.service.StudentService;  
import org.springframework.web.bind.annotation.*;  
  
@RestController  
@RequestMapping("/students")  
public class StudentController {  
  
    private final StudentService studentService;  
  
    public StudentController(StudentService studentService) {  
        this.studentService = studentService;  
    }  
  
    // API Endpoint: POST /students/add  
    @PostMapping("/add")  
    public Student addStudent(@RequestBody Student student) {  
        return studentService.saveStudent(student);  
    }  
}
```

Step 7: Run the Application

Run the main application file:

File: `DemoApplication.java`

```
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

Step 8: Test the Application

Use **Postman** or **cURL** to send a POST request:

POST URL:

`http://localhost:8080/students/add`

Request Body (JSON):

```
{
  "name": "Ritesh Tiwari",
  "course": "Web Programming"
}
```

Response:

```
{  
  "id": 1,  
  "name": "Ritesh Tiwari",  
  "course": "Web Programming"  
}
```

✅ Record successfully inserted into the **students** table in your MySQL database.

Flow of Execution

Client (POST Request)



Controller → Service → Repository



Hibernate (JPA)



Database (MySQL)

Summary

Step	Component	Purpose
1	Add dependencies	Enable Spring Boot + Hibernate
2	Configure DB	Set up connection & dialect
3	Entity	Maps class to table

4	Repository	Handles CRUD operations
5	Service	Business logic layer
6	Controller	Handles API requests
7	Run & Test	Create record in DB

(b) Explain how testing of custom login form can be performed with the help of an example.

Ans :

What is Login Form Testing?

Login form testing means verifying that the login page of an application **works correctly, securely, and as expected** under various input conditions.

➤ Purpose:

To ensure that only valid users can log in and that invalid attempts are handled properly without security leaks.

What to Test in a Login Form

Test Type	Description
Functional Testing	Verify correct login for valid credentials.
Validation Testing	Check input fields for blank, invalid, or special characters.
Security Testing	Prevent SQL injection, brute force, and XSS attacks.

Usability Testing	Ensure the form is user-friendly (labels, buttons, messages).
Performance Testing	Test how fast login works under load.

Basic Structure of a Custom Login Form

Let's assume a simple **Spring Boot + JSP** login example for demonstration.

HTML Form (login.jsp):

```
<form action="/login" method="post">
  <label>Username:</label>
  <input type="text" name="username" required><br><br>

  <label>Password:</label>
  <input type="password" name="password" required><br><br>

  <input type="submit" value="Login">
</form>
```

Controller (LoginController.java):

```
@RestController
public class LoginController {

    @PostMapping("/login")
    public String login(@RequestParam String username, @RequestParam String password) {
        if ("admin".equals(username) && "12345".equals(password)) {
            return "Login Successful";
        } else {
            return "Invalid Username or Password";
        }
    }
}
```

Process of Testing the Custom Login Form

We can perform **manual testing** or **automated testing**.

✓ A. Manual Testing Steps

Test Case ID	Test Description	Input	Expected Result
TC01	Valid credentials	Username: admin , Password: 12345	Login successful message
TC02	Invalid username	Username: xyz , Password: 12345	Error message displayed
TC03	Invalid password	Username: admin , Password: wrong	Error message displayed
TC04	Blank fields	Username: (empty), Password: (empty)	Validation message shown
TC05	SQL Injection attempt	Username: ' OR 1=1 --, Password: 12345	Login should fail
TC06	Case sensitivity check	Username: Admin , Password: 12345	Login should fail
TC07	Page redirect check	Valid login	Redirects to dashboard/home
TC08	Session management	Login → Back button	Should not show login again if session is active

B. Automated Testing (Using JUnit & Spring Boot)

Here's how you can automate testing the login logic.

Test Class Example:

```
package com.example.demo;

import static org.assertj.core.api.Assertions.assertThat;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.web.client.TestRestTemplate;
import org.springframework.http.*;

@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
public class LoginControllerTest {

    @Autowired
    private TestRestTemplate restTemplate;

    @Test
    void testValidLogin() {
        HttpHeaders headers = new HttpHeaders();
        headers.setContentType(MediaType.APPLICATION_FORM_URLENCODED);

        String body = "username=admin&password=12345";

        HttpEntity<String> request = new HttpEntity<>(body, headers);
        ResponseEntity<String> response =
            restTemplate.postForEntity("/login", request, String.class);

        assertThat(response.getBody()).contains("Login Successful");
    }

    @Test
    void testInvalidLogin() {
        String body = "username=wrong&password=test";
        HttpHeaders headers = new HttpHeaders();
        headers.setContentType(MediaType.APPLICATION_FORM_URLENCODED);
```

```
        HttpEntity<String> request = new HttpEntity<>(body, headers);
        ResponseEntity<String> response =
restTemplate.postForEntity("/login", request, String.class);

        assertThat(response.getBody()).contains("Invalid Username or
Password");
    }
}
```

Flow of Testing Process

Browser / Test Script



Send HTTP POST → /login



Login Controller (checks credentials)



Returns Response (Success / Error)



Tester verifies output manually or via JUnit

Expected Output

✅ For Valid Credentials:

Login Successful

❌ For Invalid Credentials:

Invalid Username or Password

Summary

Step	Activity	Example
1	Create Login Form	HTML + Controller
2	Define Valid Credentials	“admin” / “12345”
3	Test Manually or with JUnit	Send POST request
4	Verify Response	Check success or error
5	Secure Form	Validate inputs, avoid SQL Injection

✓ Final Takeaway:

Testing a custom login form involves **validating functionality, input handling, and security**, either **manually with test cases** or **automatically using JUnit or Selenium**.

Qus 3.

(a) Explain issues and challenges of web security.

Ans :

Introduction to Web Security

Web Security refers to protecting websites, web applications, and online services from **cyber threats** that exploit weaknesses in the web’s architecture, software, or configuration.

➡ In simple terms:

Web security ensures that data, users, and systems on the internet remain **safe from unauthorized access, misuse, or attacks**.

Why Web Security is Important

- To protect **user data** (like passwords, credit card numbers).
- To prevent **unauthorized access** to web servers and databases.
- To maintain **trust** and **business reputation**.
- To ensure **availability** and **reliability** of web services.

Major Issues and Challenges in Web Security

Let's discuss the **key issues** one by one 📌

A. Unauthorized Access

Attackers may gain access to restricted areas of a website by bypassing authentication systems.

- Weak passwords or poor access control policies are common causes.
- **Example:** Logging in as an admin using default credentials.

B. SQL Injection (SQLi)

When an attacker inserts malicious SQL code into an input field to access or modify database data.

- **Example:**
Input: ' OR 1=1 --
This can force the database to return all records.
- **Impact:** Data theft or deletion.

C. Cross-Site Scripting (XSS)

Occurs when attackers inject malicious scripts (usually JavaScript) into trusted websites.

- **Example:** Displaying a `<script>` in a comment box.
- **Impact:** Steals cookies, session tokens, or redirects users.

D. Cross-Site Request Forgery (CSRF)

Forces a user to perform unwanted actions while logged into a web app.

- **Example:** Clicking a malicious link that triggers money transfer.
- **Impact:** Unauthorized transactions or changes in user settings.

E. Data Breaches

Sensitive user data (like passwords, personal info) may leak due to insecure storage or transmission.

- **Example:** Unencrypted passwords stored in plain text.
- **Impact:** Privacy violations, identity theft.

F. Insecure Session Management

Weak session IDs or lack of logout handling can allow attackers to hijack user sessions.

- **Example:** Reusing old session IDs to impersonate users.
- **Solution:** Use secure, random session tokens and HTTPS.

G. Denial of Service (DoS) and DDoS Attacks

Attackers flood a web server with massive traffic to make it unavailable.

- **Impact:** Downtime, loss of revenue, poor user experience.
- **Solution:** Use firewalls and traffic filtering.

H. Weak Authentication and Password Policies

- Use of weak passwords like “12345” or “admin”.
- No multi-factor authentication (MFA)
- **Impact:** Easier for hackers to gain control.

I. Unpatched Software and Vulnerabilities

- Many websites use outdated CMS (like WordPress or Joomla) or plugins.
- **Impact:** Known security holes can be exploited.

- **Solution:** Regular updates and security patches.

J. Man-in-the-Middle (MITM) Attacks

An attacker intercepts communication between client and server.

- **Example:** Intercepting login credentials on public Wi-Fi.
- **Solution:** Use HTTPS (SSL/TLS encryption).

K. Phishing and Social Engineering

- Attackers trick users into giving login credentials via fake websites or emails.
- **Impact:** Unauthorized access to real accounts.

Common Challenges in Maintaining Web Security

Challenge	Explanation
1. Evolving Threats	New attack techniques appear regularly.
2. Human Errors	Developers or users may misconfigure systems or reuse weak passwords.
3. Integration Complexity	Securing APIs, third-party services, and plugins is difficult.
4. Balancing Security & Usability	Overly strict security can reduce user convenience.
5. Limited Resources	Small organizations may lack budget for dedicated security tools.

Best Practices to Handle These Issues

Practice	Description
Use HTTPS	Encrypts communication between client and server.

Input Validation	Prevents SQL injection and XSS.
Regular Patching	Updates frameworks, plugins, and OS.
Strong Authentication	Enforce strong passwords and 2FA.
Web Application Firewalls (WAF)	Filters out malicious traffic.
Security Testing	Use tools like OWASP ZAP or Burp Suite to find vulnerabilities.

Summary Table

Category	Common Issue	Impact
Authentication	Weak passwords	Unauthorized access
Data Handling	SQL Injection, XSS	Data theft
Network	MITM, DDoS	Downtime, data interception
Application	CSRF, session hijacking	User impersonation
Human Factors	Misconfigurations	Security loopholes

Final Summary:

Web security faces continuous challenges such as **data breaches, injections, XSS, weak authentication, and DoS attacks.**

To overcome these, developers must **regularly test, update, encrypt, and validate** their web applications.

(b) What is Spring Boot DevTools ? Briefly explain its features.

Ans :

What is Spring Boot DevTools?

Spring Boot DevTools is a **development-time tool** provided by Spring Boot to **enhance developer productivity** by automatically detecting code changes and applying them without restarting the entire application manually.

→ In simple words:

Spring Boot DevTools helps developers **develop faster** by providing **automatic restarts, live reload, and better debugging support** during application development.

How It Works

When you add DevTools to your Spring Boot project, it monitors your classpath for changes.

- Whenever you **save** or **modify** a file (like a controller, HTML page, or configuration),
- DevTools **automatically restarts** the application or **reloads the browser** (if LiveReload is enabled).

Example of Adding DevTools Dependency

If you are using **Maven**, add this dependency in your **pom.xml**:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <scope>runtime</scope>
  <optional>true</optional>
</dependency>
```

Main Features of Spring Boot DevTools

❶ Automatic Restart

- Automatically restarts the application whenever it detects code or classpath changes.
- Much faster than manually stopping and running the app again.

 Example:

If you update a controller or service class and save it, DevTools restarts the app automatically.

❷ Live Reload

- Automatically refreshes the browser when static files (HTML, CSS, JS) are changed.
- Works with **LiveReload browser extension**.

 Example:

When you modify an HTML template (like `index.html`), the browser updates instantly.

❸ Property Defaults for Development

- Enables developer-friendly settings by default:
 - Disables caching for templates.
 - Enables detailed error messages.
 - Enables fast restart mode.

❹ Remote Debugging (Optional)

- You can use DevTools remotely to reload applications running on a different machine, though this requires explicit setup for security reasons.

❺ Automatic Resource Caching Disable

- DevTools automatically disables caching for **Thymeleaf, Freemarker, or JSP templates** during development.
- This ensures that template changes appear immediately

❻ Improved Logging

- Provides more detailed log output for debugging during development.

- Example: displays stack traces and debug-level logs automatically.

7 Global Settings

- You can place a `spring-devtools.properties` file in your **home directory** to define settings that apply to all Spring Boot projects.

Advantages of Using DevTools

Feature	Benefit
Automatic restart	Saves time during frequent code changes
Live reload	Improves front-end testing speed
No cache for templates	Immediate visibility of UI updates
Enhanced debugging	Easier to detect and fix errors
Developer-friendly defaults	Optimized for development phase

✓ Summary

Aspect	Description
Tool Name	Spring Boot DevTools
Purpose	Increase productivity during development
Key Features	Auto-restart, LiveReload, Logging, Template cache disable
Used In	Spring Boot development environment only (not in production)

✖ Final Answer (Short Exam Version):

Spring Boot DevTools is a module in Spring Boot that provides developer-friendly features such as **automatic application restarts**, **live reload**, **enhanced logging**, and **no-template caching**, which help speed up the development and debugging process.

Qus 4.

(a) Briefly explain Crud Repository of Spring Boot.

Ans :

What is CrudRepository?

CrudRepository is an **interface provided by Spring Data JPA** that defines **basic CRUD operations** — **Create, Read, Update, and Delete** — for database entities.

It is a **generic interface**, part of the package:

`org.springframework.data.repository.CrudRepository`

Syntax:

`public interface CrudRepository<T, ID> extends Repository<T, ID>`

Where:

- **T** → Entity class type (e.g., **Student**)
- **ID** → Type of the entity's primary key (e.g., **Long**, **Integer**)

Common Methods of CrudRepository

Method	Description
<code>save(S entity)</code>	Save a new entity or update an existing one

<code>findById(ID id)</code>	Retrieve an entity by its ID
<code>findAll()</code>	Retrieve all entities
<code>deleteById(ID id)</code>	Delete an entity by its ID
<code>existsById(ID id)</code>	Check if an entity exists by ID
<code>count()</code>	Get the total number of entities

Example Usage

Entity Class:

```
@Entity
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String course;
}
```

Repository Interface:

```
public interface StudentRepository extends CrudRepository<Student, Long> {
}
```

Service / Usage:

```
@Autowired
private StudentRepository studentRepo;

// Create
studentRepo.save(new Student("Ritesh", "MCA"));

// Read
Student s = studentRepo.findById(1L).orElse(null);

// Update
s.setCourse("MBA");
studentRepo.save(s);

// Delete
studentRepo.deleteById(1L);
```

Advantages

- Reduces **boilerplate code** (no need to write SQL queries).
- Provides **built-in methods** for common CRUD operations.
- Easily extended to include **custom queries** using method names or `@Query`.

In short:

CrudRepository in Spring Boot simplifies database operations by providing ready-made methods to **create, read, update, and delete entities** with minimal code.

(b) Explain Spring MVC configuration process.

Ans :

What is Spring MVC?

Spring MVC (Model-View-Controller) is a **framework for building web applications** in Java.

It separates an application into three main components:

Component	Role
Model	Represents application data and business logic
View	User interface (HTML, JSP, Thymeleaf, etc.)
Controller	Handles user requests and manages flow between Model and View

Steps to Configure Spring MVC

Step 1: Add Dependencies

Add Spring MVC dependencies in `pom.xml` (Maven):

```
<dependency>

  <groupId>org.springframework.boot</groupId>

  <artifactId>spring-boot-starter-web</artifactId>

</dependency>
```

✓ This includes **Spring MVC**, **Tomcat**, and **other web dependencies**.

Step 2: Configure DispatcherServlet

- **DispatcherServlet** is the **front controller** in Spring MVC.
- It intercepts all incoming HTTP requests and forwards them to the appropriate controller

In Spring Boot, this is **auto-configured**, so you don't need to declare it manually.

Step 3: Create Controller

A **Controller class** handles client requests.

```
@Controller
public class HomeController {

    @GetMapping("/welcome")
    public String showWelcome(Model model) {
        model.addAttribute("message", "Welcome to Spring MVC!");
        return "welcome"; // Name of the view (welcome.jsp or
welcome.html)
    }
}
```

Step 4: Configure View Resolver

The **ViewResolver** maps the logical view name returned by the controller to an actual template.

For JSP:

```
spring.mvc.view.prefix=/WEB-INF/views/
```

```
spring.mvc.view.suffix=.jsp
```

- Returning "welcome" resolves to `/WEB-INF/views/welcome.jsp`.

For Thymeleaf:

```
spring.thymeleaf.prefix=classpath:/templates/
```

```
spring.thymeleaf.suffix=.html
```

Step 5: Create View

Example JSP file (`/WEB-INF/views/welcome.jsp`):

```
<html>
```

```

<body>

<h2>${message}</h2>

</body>

</html>

```

Step 6: Run Application

- Start the Spring Boot application.
- Access URL: <http://localhost:8080/welcome>
- Output: [Welcome to Spring MVC!](#)

Spring MVC Request Flow

Browser → DispatcherServlet → Controller → Model → ViewResolver → View → Browser

- **DispatcherServlet:** Front controller intercepts request.
- **Controller:** Handles business logic.
- **Model:** Passes data to the view.
- **ViewResolver:** Maps view name to template
- **View:** Renders the response.

Summary Table

Step	Component	Purpose
1	Dependencies	Add Spring MVC starter
2	DispatcherServlet	Front controller for all requests
3	Controller	Handle requests and prepare model
4	ViewResolver	Maps logical view to actual template
5	View	Displays output to user

✓ Key Points:

- Spring MVC separates **concerns**: Model, View, Controller.
- Spring Boot simplifies setup via **auto-configuration**.
- DispatcherServlet acts as the **central request handler**.
- ViewResolver connects **controller output** to **UI templates**.

If you want, I can also create a **diagram showing the Spring MVC flow** from **request** → **controller** → **view** for easier understanding.

Qus 5. Write short notes on the following : each

(a) JSP implicit objects

(b) Role based login

(c) Session management

(d) J2EE Architecture

Ans :

(a) JSP Implicit Objects

Definition:

JSP provides **9 built-in objects** that are automatically available in a JSP page without explicit declaration. These are called **implicit objects**.

Object	Description
<code>request</code>	Represents the <code>HttpServletRequest</code> object; contains client request data.
<code>response</code>	Represents the <code>HttpServletResponse</code> object; used to send data back to the client.
<code>out</code>	Used to send output to the client (HTML content).
<code>session</code>	Represents the <code>HttpSession</code> object; used for session tracking.

application Represents **ServletContext**; used to share data across all JSPs in the application.

config Represents **ServletConfig**; contains servlet configuration data.

pageContext Provides access to page attributes and other implicit objects.

page Represents the current JSP page instance (**this**).

exception Available only in error pages; represents the exception that occurred.

✓ **Use:** Simplifies access to request, response, session, and application objects without explicit coding.

(b) Role-Based Login

Definition:

Role-based login restricts access to parts of a web application based on the **user's role** (e.g., Admin, User, Manager).

How it works:

1. User logs in with username and password.
2. Application checks credentials and retrieves the **role** from the database
3. Redirects user to **pages specific to their role**.
4. Access control prevents unauthorized access to other pages.

Example:

- Admin → Can access user management and reports
- User → Can only view their own dashboard

✓ **Benefits:** Security, controlled access, personalized UI.

(c) Session Management

Definition:

Session management keeps track of a **user's interaction with a web application** across multiple requests.

Techniques:

1. **Cookies** – Store session ID in browser cookies.
2. **URL Rewriting** – Append session ID to URL.
3. **HttpSession** – Server-side object to store user-specific data.

Use Cases:

- Login authentication
- Shopping cart in e-commerce
- Tracking user preferences

Example in JSP:

```
<%
session.setAttribute("username", "Ritesh");
String user = (String)session.getAttribute("username");
%>
```

✓ **Purpose:** Maintains state in a stateless HTTP environment.

(d) J2EE Architecture**Definition:**

J2EE (Java 2 Platform, Enterprise Edition) provides a **platform for developing distributed, multi-tier enterprise applications**.

Architecture Layers:

Layer

Components / Purpose

Client Layer	Web browsers, desktop apps, or mobile apps that send requests.
Web Layer	JSP, Servlets, HTML, responsible for request handling and presentation.
Business Layer	EJB (Enterprise Java Beans) or POJOs that implement business logic.
Integration Layer	Connects to databases, messaging systems, legacy systems.
Enterprise Information Systems (EIS) Layer	Databases, ERP, legacy systems storing actual data.

Key Features:

- Platform-independent
- Multi-tiered architecture
- Supports scalability, reliability, and security

 **Purpose:** Simplifies building **robust, secure, and distributed enterprise applications.**

June, 2022

Qus 1.

(a) What is J2EE ? Briefly explain J2EE architecture.

Ans :

J2EE (Java 2 Platform, Enterprise Edition) is a **platform for developing and deploying enterprise-level web applications** using Java.

It provides a **set of APIs, services, and protocols** to build distributed, multi-tier, and scalable applications.

◆ **Definition:**

J2EE is a collection of **technologies and specifications** that simplify the development of **multi-tier enterprise applications**, such as web-based, distributed, and transactional systems.

◆ **Key Features of J2EE:**

1. **Platform Independence** – Runs on any system supporting Java.
2. **Component-Based Architecture** – Uses reusable components (Servlets, JSP, EJB).
3. **Multi-Tier Architecture** – Separates presentation, business, and data layers.
4. **Scalability and Security** – Supports large-scale and secure applications.
5. **Integration** – Works with web services, XML, JDBC, etc.

J2EE Architecture

The **J2EE architecture** is based on a **multi-tier (layered) model**, typically divided into **four tiers**:

Client Tier

- Represents the **end-user interface** (web browser, mobile app, desktop client).
- Communicates with the server through HTTP requests.
- Examples: HTML pages, JSP pages, or Java applications.

Web Tier

- Responsible for **handling client requests** and **generating responses**.
- Components:
 - **Servlets** – Handle requests and responses.
 - **JSP (JavaServer Pages)** – Generate dynamic web content.
- Acts as a **controller** and **view** in MVC architecture.

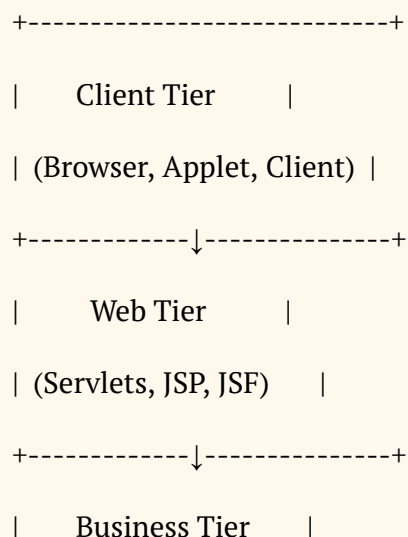
Business Tier

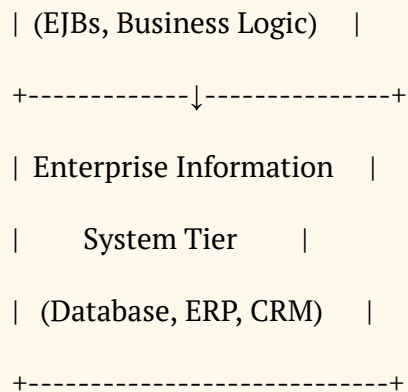
- Contains the **business logic** or rules of the application.
- Components:
 - **EJB (Enterprise JavaBeans)** – Encapsulate business logic.
 - **Java Classes** – Perform computations and business processes.
- Handles **transactions**, **security**, and **database operations**.

Enterprise Information System (EIS) Tier

- Interacts with the **database or legacy systems**.
- Components:
 - **JDBC (Java Database Connectivity)** – For database access.
 - **JCA (Java Connector Architecture)** – For connecting to enterprise systems like ERP, CRM.

J2EE Architecture Diagram





♦ Example Workflow

1. The **client** sends a request via a browser.
2. The **web tier** (Servlet/JSP) receives and processes the request.
3. The **business tier** (EJB or service class) executes business logic.
4. The **EIS tier** retrieves or stores data in the database.
5. The response is sent back to the client.

Advantages of J2EE:

Advantage	Description
Reusability	Components can be reused across applications.
Scalability	Supports distributed and large-scale applications.
Security	Provides built-in authentication and authorization mechanisms.
Portability	Runs on any Java-compliant server.
Integration	Easily integrates with other enterprise systems.

Short Exam Answer Summary

J2EE (Java 2 Enterprise Edition) is a Java platform used for building multi-tier, distributed, and web-based enterprise applications.

J2EE Architecture consists of four tiers:

1. **Client Tier** – User interface.
2. **Web Tier** – Request processing (Servlets, JSP).
3. **Business Tier** – Business logic (EJB).
4. **EIS Tier** – Database and enterprise system interaction.

(b) Explain servlet architecture. Also briefly explain the use of servlet with the help of an example.

Ans :

A **Servlet** is a **Java program** that runs on a **web server** and is used to create **dynamic web content**.

Servlets handle **client requests**, process them, and generate **responses** (usually in HTML).

Servlets are part of **Java EE (J2EE)** and are managed by the **Servlet Container** (e.g., Apache Tomcat).

Servlet Architecture

The **Servlet Architecture** defines how servlets interact with the web server and client to process web requests.

Key Components:

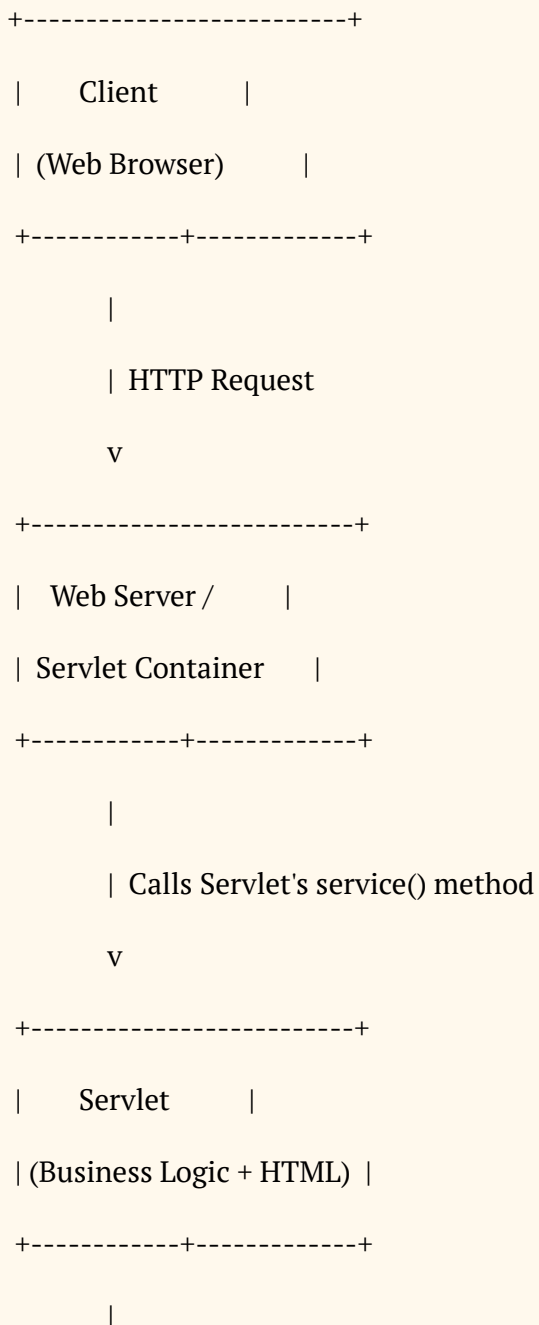
1. **Client (Browser):**
Sends an HTTP request to the web server.
2. **Web Server / Servlet Container:**
 - Hosts and manages the servlets.
 - Creates servlet instances, manages their lifecycle, and handles communication between client and servlet.
 - Examples: Apache Tomcat, Jetty, GlassFish.
3. **Servlet:**

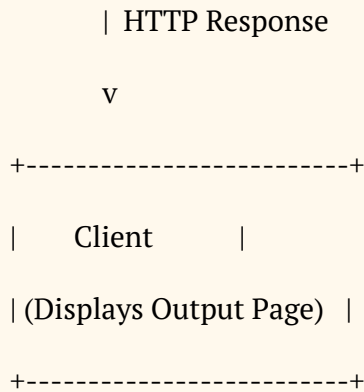
- Java class that processes client requests and generates responses.
- Uses **HttpServletRequest** and **HttpServletResponse** objects.

4. Database / Business Logic (optional):

- The servlet can connect to databases or other systems to fetch or process data.

Servlet Architecture Diagram





Servlet Lifecycle (in Brief)

1. **Initialization** – `init()`
Called once when the servlet is first loaded.
2. **Request Handling** – `service()`
Called every time a request comes from the client.
Delegates to `doGet()` or `doPost()`.
3. **Destruction** – `destroy()`
Called once when the servlet is unloaded from memory.

Example: Servlet to Display “Welcome to Web Programming”

1 HTML Form (welcome.html)

```

<!DOCTYPE html>
<html>
<head>
<title>Welcome Page</title>
</head>
<body>
<form action="WelcomeServlet" method="get">
  
```



```

        <input type="submit" value="Click Here">
    </form>
</body>
</html>

```

2 Servlet Program (WelcomeServlet.java)

```

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class WelcomeServlet extends HttpServlet {

    public void doGet(HttpServletRequest request, HttpServletResponse
response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<html><body>");
        out.println("<h2>Welcome to Web Programming</h2>");
        out.println("</body></html>");

    }
}

```

Deployment Descriptor (web.xml)

```

<web-app>
    <servlet>
        <servlet-name>welcome</servlet-name>
        <servlet-class>WelcomeServlet</servlet-class>
    </servlet>

    <servlet-mapping>
        <servlet-name>welcome</servlet-name>
        <url-pattern>/WelcomeServlet</url-pattern>
    </servlet-mapping>
</web-app>

```

Working of the Example

1. User opens `welcome.html` and clicks the button.
2. The request goes to the **Servlet Container** (e.g., Tomcat).
3. The container loads `WelcomeServlet`, calls `doGet()`.
4. The servlet sends an HTML response:
👉 “Welcome to Web Programming.”
5. Browser displays the response.

Advantages of Servlet Architecture

Feature	Description
Platform Independent	Written in Java, runs anywhere.
Efficient	Multithreaded; one instance handles many requests.
Secure	Uses Java’s security model.
Reusable & Scalable	Modular and easily extendable.
Integration	Can easily connect to databases using JDBC.

Short Exam Summary

Servlet Architecture defines how servlets interact with the web server and client to handle requests and responses.

Components: **Client** → **Web Server/Container** → **Servlet** → **Database (optional)**

Example: Servlet displaying “Welcome to Web Programming.”

Lifecycle: `init()` → `service()` → `destroy()`.

(c) Explain JSP scripting elements with the help of a program.

Ans :

JSP (JavaServer Pages) scripting elements are used to **embed Java code directly into an HTML page**.

They allow the developer to write Java logic inside the JSP for dynamic content generation.

Types of JSP Scripting Elements

There are **three main scripting elements** in JSP:

Type	Syntax	Purpose
1. Declaration Tag	<code><%! ... %></code>	Declares variables or methods that can be used throughout the JSP.
2. Scriptlet Tag	<code><% ... %></code>	Contains Java code that is executed each time the page is requested.
3. Expression Tag	<code><%= ... %></code>	Outputs the value of a Java expression directly into the HTML response.

Declaration Tag (`<%! ... %>`)

Used to **declare variables or methods** at the class level (outside of the `service()` method).

Example:

```
<%! int counter = 0; %>
<%!
    String greetUser(String name) {
        return "Hello, " + name + "!";
    }
%>
```

Scriptlet Tag (<% ... %>)

Used to **write Java code** that executes every time the JSP page is requested.

Example:

```
<%
    counter++;
    String user = "Ritesh";
%>
```

Expression Tag (<%= ... %>)

Used to **display output directly** to the client (browser).

It automatically calls `out.print()`.

Example:

```
<p>Welcome message: <%= greetUser(user) %></p>
```

```
<p>Number of visits: <%= counter %></p>
```

Complete Example Program: JSP Using Scripting Elements

File: `scriptingExample.jsp`

```
<%@ page language="java" contentType="text/html; charset=UTF-8" %>
<html>
<head>
<title>JSP Scripting Elements Example</title>
</head>
<body>

<%!
    int counter = 0;
    String greetUser(String name) {
        return "Hello, " + name + "!";
    }
%>
```

```

<%
    counter++;
    String name = "Ritesh";
%>

<h2>Demonstrating JSP Scripting Elements</h2>
<p><b>Greeting Message:</b> <%= greetUser(name) %></p>
<p><b>Number of Visits:</b> <%= counter %></p>

</body>
</html>

```

Explanation

Part	Tag Used	Purpose
Variable & method declaration	<%! ... %>	Declares a variable (counter) and a method (greetUser()).
Request-time logic	<% ... %>	Increments counter and sets a name.
Output display	<%= ... %>	Displays the greeting and counter value.

✓ Output (in Browser)

Demonstrating JSP Scripting Elements

Greeting Message: Hello, Ritesh!

Number of Visits: 1

f refreshed, the counter will reinitialize because JSP reloads per request unless stored in session.)

Short Exam Summary

JSP Scripting Elements are used to insert Java code into JSP pages.

- **Declaration** (<%! ... %>): Declare variables and methods.
- **Scriptlet** (<% ... %>): Execute Java statements per request.
- **Expression** (<%= ... %>): Display results in the web page.

Example:

```
<%! int x = 5; %>
<% x++; %>
<%= x %> → Output: 6
```

(d) What is Spring Boot ? Briefly explain Spring Boot application properties.

Ans :

Spring Boot is a framework built on top of the **Spring Framework** that is used to create **stand-alone, production-ready Spring applications** quickly and easily with **minimum configuration**.

It simplifies the setup of new Spring projects by using **auto-configuration**, **embedded servers**, and **starter dependencies**.

Definition

Spring Boot is a framework that helps developers build Spring-based applications easily, without needing to manually configure XML or external servers.

Key Features of Spring Boot

1. **Auto-Configuration:**
Automatically configures beans and settings based on dependencies in your classpath.
2. **Standalone Applications:**
Creates applications that can run directly using `java -jar`.
3. **Embedded Servers:**
Comes with **Tomcat**, **Jetty**, or **Undertow** embedded — no need to deploy WAR files.
4. **Starter Dependencies:**
Provides pre-configured dependencies for various functionalities (e.g.,

`spring-boot-starter-web, spring-boot-starter-data-jpa`).

5. Actuator:

Monitors and manages applications in production (e.g., `/actuator/health`).

Spring Boot Application Properties

Spring Boot uses a configuration file named **`application.properties`** (or **`application.yml`**) to set up application-specific and environment-specific parameters.

This file is typically located in the **`src/main/resources`** folder of your project.

Common Uses of **`application.properties`**

Category	Property	Example	Purpose
Server	<code>server.port</code>	<code>server.port=8081</code>	Changes the default server port (default: 8080)
Database	<code>spring.datasource.url</code>	<code>spring.datasource.url=jdbc:mysql://localhost:3306/testdb</code>	Database connection URL
	<code>spring.datasource.username</code>	<code>spring.datasource.username=root</code>	Database

			username
	<code>spring.datasource.password</code>	<code>spring.datasource.password=root123</code>	Database password
JPA/Hibernate	<code>spring.jpa.hibernate.ddl-auto</code>	<code>spring.jpa.hibernate.ddl-auto=update</code>	Schema update mode
	<code>spring.jpa.show-sql</code>	<code>spring.jpa.show-sql=true</code>	Show SQL queries in console
Application Info	<code>spring.application.name</code>	<code>spring.application.name=MyApp</code>	Sets application name
Logging	<code>logging.level.root</code>	<code>logging.level.root=INFO</code>	Controls logging level
Context Path	<code>server.servlet.context-path</code>	<code>server.servlet.context-path=/myapp</code>	Sets application context path

Example: application.properties

```
# Server Configuration
server.port=8081
server.servlet.context-path=/webapp

# Application Name
spring.application.name=StudentManagementSystem

# Database Configuration
spring.datasource.url=jdbc:mysql://localhost:3306/studentdb
spring.datasource.username=root
spring.datasource.password=admin123
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# Logging
logging.level.org.springframework=INFO
```

How It Works

- When the application starts, **Spring Boot** reads the **application.properties** file.
- The values are automatically **injected into Spring Beans** and **used to configure the system**.
- Developers can **override default configurations** without modifying code.

Advantages of Using **application.properties**

- Centralized configuration management.
- Easy to change environment-specific values (e.g., dev, test, prod).
- Reduces hardcoding of values in the code.
- Improves maintainability and flexibility.

Short Exam Summary

Spring Boot is a framework for building stand-alone Spring applications easily using **auto-configuration**, **starter dependencies**, and **embedded servers**.

application.properties is a configuration file that defines key settings like **server port**, **database URL**, **JPA behavior**, **logging**, and **application name**.

Example:

```
server.port=8081
spring.datasource.url=jdbc:mysql://localhost:3306/testdb
spring.jpa.show-sql=true
```

Qus 2.

(a) What is a design pattern ? Explain the advantages of using a design pattern in enterprise applications development, with the help of an example.

Ans :

A Design Pattern is a proven, reusable solution to a commonly occurring problem in software design.

It provides a standard way to structure code that makes applications easier to maintain, understand, and scale.

◆ Definition

A Design Pattern is a general reusable template or blueprint that provides a standard solution for solving common software design problems.

◆ Key Features

- **Reusable and time-tested solutions.**
- Improves communication among developers.
- Increases maintainability and flexibility.
- Reduces development time.

Advantages of Using Design Patterns in Enterprise Applications

Advantage	Explanation
1. Reusability	Design patterns can be applied in multiple projects, reducing redundant coding.

- | | |
|---------------------------|--|
| 2. Maintainability | Makes enterprise applications easier to modify and extend. |
| 3. Scalability | Ensures systems can handle growth without redesign. |
| 4. Consistency | Provides a standard structure that all developers can follow. |
| 5. Faster Development | Solves recurring problems quickly using known templates. |
| 6. Improved Communication | Developers can discuss designs easily using pattern names (like Singleton, Factory, etc.). |

Example: Singleton Design Pattern

Purpose

The Singleton Pattern ensures that only one instance of a class is created throughout the application and provides a global point of access to that instance.

This is useful in enterprise applications for managing shared resources like database connections, logging, or configuration management.

Example Program: Singleton Design Pattern

```
// Singleton class for Database Connection
public class DatabaseConnection {

    // Step 1: Create a private static instance of the class
    private static DatabaseConnection instance = null;

    // Step 2: Private constructor to prevent direct instantiation
    private DatabaseConnection() {
        System.out.println("Database Connected...");
    }

    // Step 3: Public method to provide access to the instance
    public static DatabaseConnection getInstance() {
        if (instance == null) {
```

```

        instance = new DatabaseConnection();
    }
    return instance;
}
}

// Testing the Singleton Pattern
public class TestSingleton {
    public static void main(String[] args) {
        DatabaseConnection c1 = DatabaseConnection.getInstance();
        DatabaseConnection c2 = DatabaseConnection.getInstance();

        // Both will refer to the same instance
        System.out.println(c1 == c2); // Output: true
    }
}

```

Explanation

1. The constructor is private, so no other class can create an object directly.
2. `getInstance()` checks whether an instance exists. If not, it creates one.
3. Any subsequent call returns the same instance.

Use in Enterprise Applications

- Database Connection Pooling – Ensure one global connection manager.
- Logging Frameworks – Centralized logger shared across modules.
- Configuration Manager – One instance to read and manage application settings.

Short Exam Summary

A Design Pattern is a reusable solution to a common software design problem.

Advantages: Reusability, maintainability, scalability, consistency, faster development.

Example (Singleton Pattern):

Ensures only one instance of a class exists, useful for shared resources like database connections or logging systems.

(b) Explain Hibernate configuration with annotations.

Ans :

Hibernate is an **Object Relational Mapping (ORM)** framework in Java that maps Java classes to database tables.

Traditionally, Hibernate configuration was done using an **XML file** (**hibernate.cfg.xml**) and **mapping files**.

However, in modern applications, Hibernate allows you to perform the same configuration using **Java annotations**, making the code more concise and easier to maintain.

◆ **Definition**

Hibernate Configuration with Annotations means configuring the mapping between Java classes and database tables using **annotations** placed directly in the source code, instead of separate XML files.

Steps to Configure Hibernate with Annotations**① Add Required Hibernate Dependencies**

If using **Maven**, include these dependencies in **pom.xml**:

```
<dependencies>
  <dependency>
    <groupId>org.hibernate.orm</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>6.0.2.Final</version>
  </dependency>

  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>8.0.33</version>
  </dependency>
</dependencies>
```

```
</dependencies>
```

② Create Hibernate Configuration File (**hibernate.cfg.xml**)

This file contains database connection details and class mappings.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
"-//Hibernate/Hibernate Configuration DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">

<hibernate-configuration>
  <session-factory>

    <!-- Database connection settings -->
    <property
name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
    <property
name="hibernate.connection.url">jdbc:mysql://localhost:3306/studentdb</prop
erty>
    <property name="hibernate.connection.username">root</property>
    <property name="hibernate.connection.password">admin123</property>

    <!-- Hibernate properties -->
    <property
name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>
    <property name="hibernate.hbm2ddl.auto">update</property>
    <property name="hibernate.show_sql">true</property>

    <!-- Annotated class -->
    <mapping class="com.example.Student"/>

  </session-factory>
</hibernate-configuration>
```

③ Create the Entity Class Using Annotations

```
package com.example;
```

```
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import jakarta.persistence.Column;

@Entity
@Table(name = "student") // Maps to 'student' table in the database
public class Student {

    @Id
    private int id;

    @Column(name = "name")
    private String name;

    @Column(name = "course")
    private String course;

    // Constructors
    public Student() {}
    public Student(int id, String name, String course) {
        this.id = id;
        this.name = name;
        this.course = course;
    }

    // Getters and setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getCourse() { return course; }
    public void setCourse(String course) { this.course = course; }
}
```

Explanation of Annotations

Annotation	Description
<code>@Entity</code>	Marks the class as a Hibernate entity (mapped to a table).
<code>@Table(name="table_name")</code>	Specifies the table name in the database.
<code>@Id</code>	Marks the primary key field.
<code>@Column(name="column_name")</code>	Maps the field to a specific column in the table.
<code>@GeneratedValue</code>	Specifies automatic generation of primary key values.

4 Create Main Class to Save Data

```
package com.example;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public class HibernateAnnotationExample {
    public static void main(String[] args) {
        // Create configuration object
        Configuration cfg = new Configuration();
        cfg.configure("hibernate.cfg.xml");

        // Build SessionFactory
        SessionFactory factory = cfg.buildSessionFactory();

        // Open a new Session
        Session session = factory.openSession();
        session.beginTransaction();

        // Create a Student object
        Student student = new Student(101, "Ritesh Tiwari", "MCA");
```



```

        // Save object to database
        session.save(student);

        // Commit transaction
        session.getTransaction().commit();
        session.close();

        System.out.println("Record inserted successfully!");
    }
}

```

How It Works

1. `hibernate.cfg.xml` provides database configuration.
2. Hibernate scans the **annotated class** (`@Entity`) for mapping information.
3. `SessionFactory` creates sessions for interacting with the database.
4. The **object is converted to SQL statements** and saved to the table.

Advantages of Annotation-Based Configuration

Advantage	Description
No XML needed	All mapping done directly in Java class.
Readable and maintainable	Easier to understand and modify.
Less boilerplate code	Reduces duplication and configuration errors.
Type-safe and refactor-friendly	Changes in class names or fields automatically update mapping.

Short Exam Summary

Hibernate Configuration with Annotations allows developers to define mappings between Java classes and database tables using annotations instead of XML.

Example:

```
@Entity
@Table(name="student")
public class Student {
    @Id
    @Column(name="id")
    private int id;
}
```

Advantages: Easier configuration, no XML, type safety, and better maintainability.

Qus 3.

(a) Explain Spring MVC framework with the help of a suitable diagram.

Ans :

Spring MVC (Model–View–Controller) is a **web framework** provided by the **Spring Framework** for building **dynamic, web-based applications** in Java.

It follows the **MVC architecture**, which separates application logic into **three layers**:

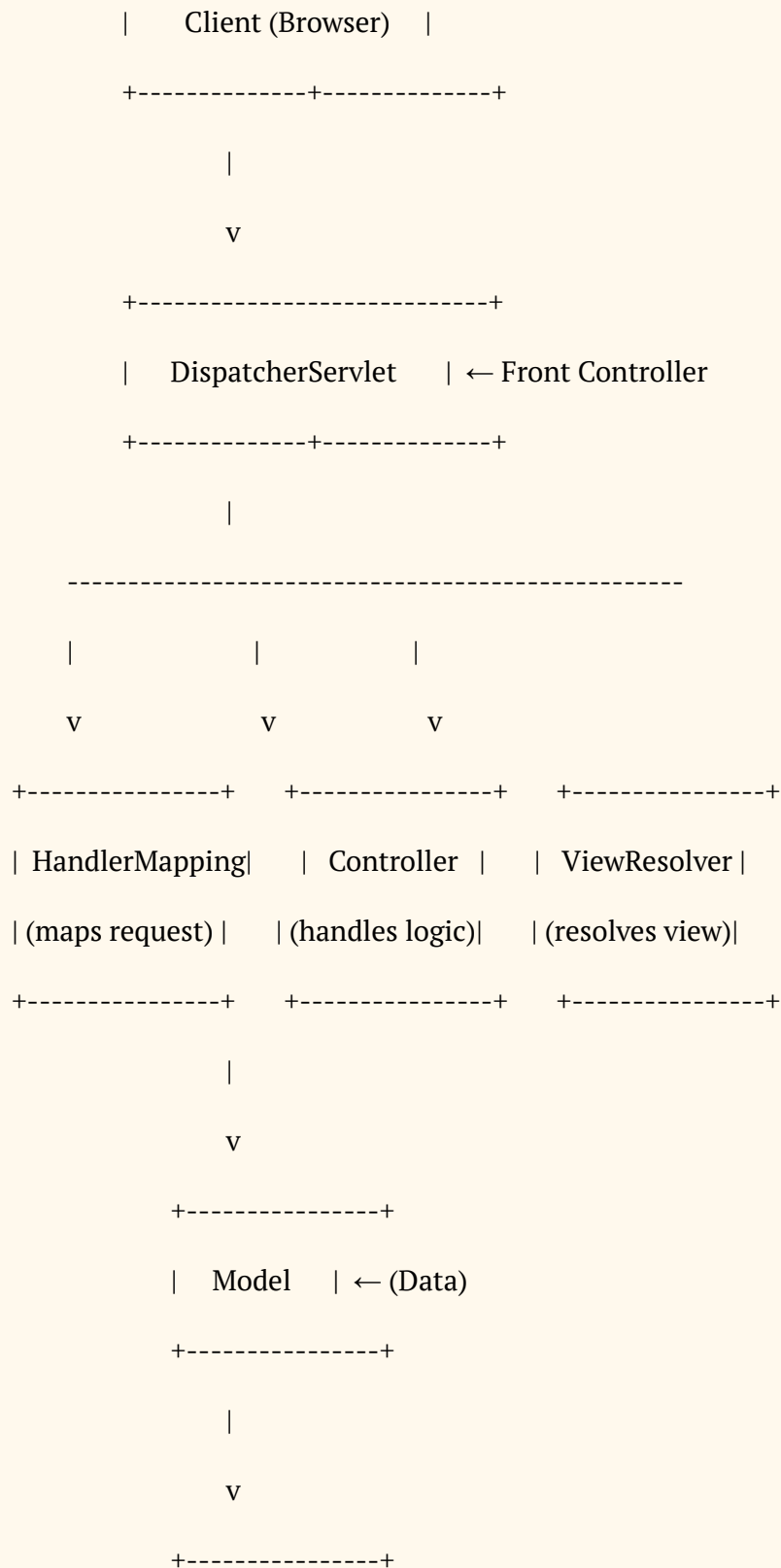
- **Model** → Handles data and business logic
- **View** → Handles presentation (UI)
- **Controller** → Handles user requests and controls the flow

Key Concept

Spring MVC uses the **DispatcherServlet** as the **front controller**, which receives all incoming web requests and delegates them to appropriate controllers.

Spring MVC Architecture Diagram

+-----+



| View | ← (JSP/HTML)

+-----+

Flow of Control in Spring MVC

Step	Description
1. Client Request	The user sends a request to the server (e.g., via a browser).
2. DispatcherServlet	Acts as the Front Controller . It receives the request.
3. Handler Mapping	Finds which Controller should handle the request.
4. Controller	Processes the request, interacts with the Model , and returns data + view name.
5. Model and View	The controller returns a ModelAndView object to DispatcherServlet.
6. ViewResolver	Maps the view name to an actual JSP/HTML page.
7. Response	The View (e.g., JSP) is rendered and sent back to the client browser.

Core Components of Spring MVC

Component	Description
DispatcherServlet	Front controller that manages the entire request flow.

HandlerMapping	Maps incoming requests to the appropriate controller.
Controller	Handles user input and updates the model.
Model	Contains data or business logic.
ViewResolver	Maps logical view names to actual views (like JSP).
View	The presentation layer (UI) that displays the data.

Example: Simple Spring MVC Application

1 Controller Class

```
@Controller
public class HelloController {

    @RequestMapping("/welcome")
    public ModelAndView displayMessage() {
        String message = "Welcome to Spring MVC!";
        return new ModelAndView("hello", "msg", message);
    }
}
```

2 View (hello.jsp)

```
<html>
<body>
    <h2>${msg}</h2>
</body>
</html>
```

③ Configuration (web.xml)

```
<web-app>
  <servlet>
    <servlet-name>spring</servlet-name>

    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
  </servlet>

  <servlet-mapping>
    <servlet-name>spring</servlet-name>
    <url-pattern>/</url-pattern>
  </servlet-mapping>
</web-app>
```

④ Spring Configuration (spring-servlet.xml)

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:mvc="http://www.springframework.org/schema/mvc"
       xmlns:context="http://www.springframework.org/schema/context">

  <context:component-scan base-package="com.example.controller" />
  <mvc:annotation-driven />
  <bean
class="org.springframework.web.servlet.view.InternalResourceViewResolver">
    <property name="prefix" value="/WEB-INF/views/" />
    <property name="suffix" value=".jsp" />
  </bean>
</beans>
```

Advantages of Spring MVC

- **Clear separation** of concerns (Model, View, Controller).
- **Highly flexible** and **easy to test**.
- **Integration ready** with other frameworks like Hibernate or Spring Boot.

- **Annotation-based** configuration reduces XML usage.

Summary

Aspect	Description
Full Form	Model–View–Controller
Main Component	<code>DispatcherServlet</code>
Purpose	To simplify web app development using a modular, layered approach
Example	User request → Controller → Model → View → Response

(b) Explain the working and flow of Struts 2.

Ans :

Apache Struts 2 is a **Java-based MVC (Model-View-Controller)** web application framework used to build **enterprise-level web applications**.

It is the **improved version of Struts 1**, combining the best features of **Struts** and **WebWork** frameworks.

Architecture Overview

Struts 2 separates the application into three main layers:

- **Model** → Business logic and data (Java classes, DB interaction)
- **View** → User Interface (JSP, HTML)
- **Controller** → Handles user requests and manages navigation (Actions, Interceptors)

Struts 2 Flow / Working

Below is the **step-by-step request flow** in the Struts 2 framework:

Step 1 – Client Sends Request

The user sends a request (e.g., `login.action`) from a web browser.

Example:

`http://localhost:8080/Struts2App/login.action`

Step 2 – `FilterDispatcher` / `StrutsPrepareAndExecuteFilter`

The request first passes through the `StrutsPrepareAndExecuteFilter` (defined in `web.xml`).

This filter is the **Front Controller** in Struts 2.

It:

- Initializes the framework
- Determines which action should handle the request
- Delegates control to the appropriate Action class

Step 3 – Interceptors

Before reaching the action, the request passes through a **series of interceptors**.

Interceptors perform common tasks like:

- Input validation
- File upload
- Logging
- Authentication

All interceptors are configured in `struts.xml` or via annotations.

Step 4 – Action Execution

The **Action class** executes the business logic.

It interacts with the **Model (Service/DAO layer)** to process the request and return a **result string** (like `"success"`, `"error"`).

Example:


```

public class LoginAction extends ActionSupport {
    private String username;
    private String password;

    public String execute() {
        if(username.equals("admin") && password.equals("1234"))
            return SUCCESS;
        else
            return ERROR;
    }

    // Getters and Setters
}

```

Step 5 – Result / View Selection

Based on the **result string** returned by the Action,

Struts 2 looks up the corresponding **view (JSP page)** in the `struts.xml` configuration.

Example (`struts.xml`):

```

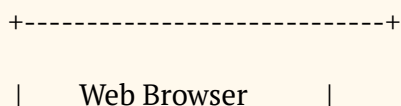
<package name="default" extends="struts-default">
    <action name="login" class="com.example.LoginAction">
        <result name="success">/welcome.jsp</result>
        <result name="error">/error.jsp</result>
    </action>
</package>

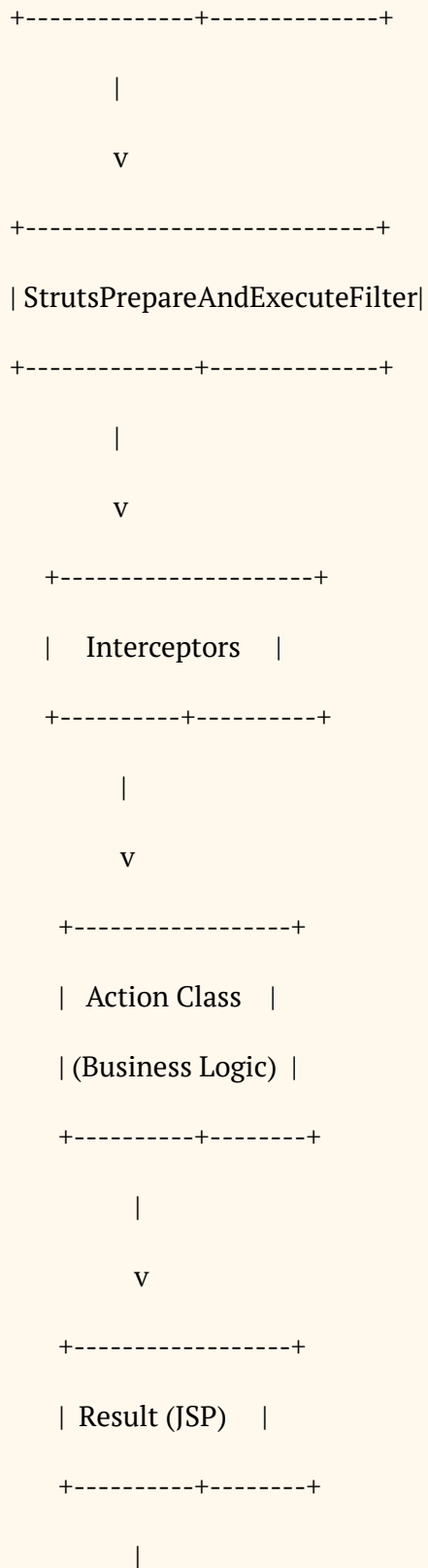
```

Step 6 – Response Rendered

The selected **JSP view** (like `welcome.jsp`) is rendered and the **response is sent back to the browser**.

Struts 2 Request Flow Diagram





```

      v
+-----+
| Client Response |
+-----+

```

Configuration Files

1 web.xml

Defines the front controller filter:

```

<filter>
  <filter-name>struts2</filter-name>

  <filter-class>org.apache.struts2.dispatcher.filter.StrutsPrepareAndExecuteFilter</filter-class>
</filter>

<filter-mapping>
  <filter-name>struts2</filter-name>
  <url-pattern>/*</url-pattern>
</filter-mapping>

```

2 struts.xml

Maps actions to results:

```

<package name="default" extends="struts-default">
  <action name="login" class="com.example.LoginAction">
    <result name="success">/welcome.jsp</result>
    <result name="error">/error.jsp</result>
  </action>
</package>

```

Key Components in Struts 2

Component	Description
Action	Executes business logic and returns a result string.
Interceptor	Performs pre/post-processing tasks.
ValueStack	Stores all objects (data) for access in JSP using OGNL.
Result	Defines the view (JSP/HTML) for the outcome.
Configuration Files	<code>web.xml</code> and <code>struts.xml</code> define application flow.

Summary Table

Step	Component	Description
1	Client	Sends request to web server
2	<code>StrutsPrepareAndExecuteFilter</code>	Front controller
3	Interceptors	Perform cross-cutting tasks
4	Action	Executes business logic
5	Result	Returns appropriate JSP
6	Response	JSP rendered to client

In Short

Struts 2 follows an MVC pattern where:

- **Controller:** `StrutsPrepareAndExecuteFilter` + Action classes
- **Model:** Business logic / Database layer
- **View:** JSP or HTML pages

Flow: Request → Filter → Interceptors → Action → Result → Response

Qus 4.

(a) Explain the steps of database connection using JSP and JDBC, with the help of a program.

Ans :

JSP (Java Server Pages) can interact with databases using **JDBC (Java Database Connectivity)** API.

JDBC provides classes and interfaces in `java.sql` package to **connect, execute queries, and retrieve data** from databases like MySQL, Oracle, etc.

Steps to Connect Database Using JSP and JDBC

Step	Description
❶ Load JDBC Driver	Register the database driver class using <code>Class.forName()</code> .
❷ Create Connection	Use <code>DriverManager.getConnection()</code> to establish a connection with the database.
❸ Create Statement	Use <code>Connection.createStatement()</code> or <code>PreparedStatement</code> to write SQL queries.

- 4 **Execute Query** Run the query using `executeQuery()` (for SELECT) or `executeUpdate()` (for INSERT, UPDATE, DELETE).
- 5 **Process Results** Use `ResultSet` to read data from the database.
- 6 **Close Connection** Always close the connection and statement objects to free resources.

Example: Connecting JSP with MySQL Database

Database Setup

- Database Name: `studentdb`
- Table Name: `students`
- Columns: `id`, `name`, `course`

```
CREATE DATABASE studentdb;
USE studentdb;

CREATE TABLE students (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50),
    course VARCHAR(50)
);
```

JSP Program: `dbconnect.jsp`

```
<%@ page import="java.sql.*" %>
<html>
<head>
<title>JSP Database Connection Example</title>
</head>
<body>

<%
```

```

// Database credentials
String url = "jdbc:mysql://localhost:3306/studentdb";
String user = "root";
String password = "admin123";

Connection con = null;
Statement stmt = null;
ResultSet rs = null;

try {
    // ① Load JDBC Driver
    Class.forName("com.mysql.cj.jdbc.Driver");

    // ② Establish Connection
    con = DriverManager.getConnection(url, user, password);

    // ③ Create Statement
    stmt = con.createStatement();

    // ④ Execute SQL Query
    String sql = "SELECT * FROM students";
    rs = stmt.executeQuery(sql);

    // ⑤ Process Results
    out.println("<h2>Student Records:</h2>");
    out.println("<table
border='1'><tr><th>ID</th><th>Name</th><th>Course</th></tr>");

    while(rs.next()) {
        out.println("<tr>");
        out.println("<td>" + rs.getInt("id") + "</td>");
        out.println("<td>" + rs.getString("name") + "</td>");
        out.println("<td>" + rs.getString("course") + "</td>");
        out.println("</tr>");
    }
    out.println("</table>");
}
catch(Exception e) {
    out.println("<h3>Error: " + e.getMessage() + "</h3>");
}
finally {

```

```

        // 6 Close resources
        if(rs != null) rs.close();
        if(stmt != null) stmt.close();
        if(con != null) con.close();
    }
%>

</body>
</html>

```

Explanation of Important Code Parts

Code	Purpose
<code>Class.forName("com.mysql.cj.jdbc.Driver")</code>	Loads the MySQL JDBC driver.
<code>DriverManager.getConnection()</code>	Establishes the connection to MySQL.
<code>Statement stmt = con.createStatement()</code>	Creates a statement to execute SQL.
<code>ResultSet rs = stmt.executeQuery("SELECT * FROM students")</code>	Executes SQL query and stores result.
<code>rs.next()</code>	Iterates through result rows.
<code>rs.getString("name")</code>	Retrieves value from a column.
<code>con.close()</code>	Closes the connection to avoid memory leaks.

Output (Example)

ID	Name	Course
1	Ritesh	MCA
2	Aman	BCA
3	Neha	MBA

Best Practices

- Use **PreparedStatement** instead of **Statement** to prevent SQL Injection.
- Always **close resources** in **finally** block.
- Use **connection pooling** in real applications for better performance.

Short Exam Summary

To connect a database using JSP:

1. Load JDBC driver
2. Create a database connection
3. Create and execute SQL statements
4. Process the result using **ResultSet**
5. Close connection

Example:

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

```
Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/studentdb",  
"root", "admin123");
```

(b) Explain the use of load() and get() methods of Hibernate session, with the help of an example.

Ans :

In Hibernate, the **Session** interface represents a connection between the Java application and the database.

It provides methods to perform **CRUD operations** (Create, Read, Update, Delete) on database entities.

Two commonly used methods to **retrieve data** from the database are:

- `get()`
- `load()`

`get()` Method

Definition:

The `get()` method **immediately fetches** the object from the database using its **primary key**.

Syntax:

```
Object obj = session.get(EntityClass.class, primaryKey);
```

Key Points:

- Hits the **database immediately**.
- Returns the **actual object** if found.
- Returns **null** if no record exists.
- Often used when you are **not sure** whether the record exists or not.

Example:

```
Student student = session.get(Student.class, 101);
if(student != null) {
    System.out.println("Name: " + student.getName());
} else {
    System.out.println("Student not found!");
}
```

👉 Explanation:

If a student with `id = 101` exists, Hibernate fetches it immediately.

If not, it returns `null` without throwing any exception.

load() Method

◆ Definition:

The `load()` method returns a **proxy object** and **fetches data lazily** (only when needed).

◆ Syntax:

```
Object obj = session.load(EntityClass.class, primaryKey);
```

◆ Key Points:

- Returns a **proxy object** (not fetched immediately).
- Hits the database **only when a property is accessed**.
- Throws an exception (`ObjectNotFoundException`) if the object doesn't exist.
- Useful when you are **sure** the record exists.

◆ Example:

```
Student student = session.load(Student.class, 102);
```

```
System.out.println("Name: " + student.getName());
```

👉 Explanation:

- When `load()` is called, Hibernate **does not hit the database immediately**.
- When you try to access `student.getName()`, Hibernate fetches the data.
- If no record is found for `id = 102`, it throws **`ObjectNotFoundException`**.

Complete Example: Comparing `get()` and `load()`

```
import org.hibernate.*;
import org.hibernate.cfg.Configuration;

public class HibernateGetLoadExample {
```

```

public static void main(String[] args) {
    Configuration cfg = new Configuration();
    cfg.configure("hibernate.cfg.xml");
    SessionFactory factory = cfg.buildSessionFactory();
    Session session = factory.openSession();

    // Using get()
    System.out.println("Using get() method:");
    Student s1 = session.get(Student.class, 101);
    if (s1 != null)
        System.out.println("Student Name: " + s1.getName());
    else
        System.out.println("Record not found using get()!");

    // Using load()
    System.out.println("\nUsing load() method:");
    try {
        Student s2 = session.load(Student.class, 102);
        System.out.println("Student Name: " + s2.getName());
    } catch (ObjectNotFoundException e) {
        System.out.println("Record not found using load()!");
    }

    session.close();
    factory.close();
}
}

```

Difference Between `get()` and `load()`

Feature	<code>get()</code>	<code>load()</code>
Data Fetching	Eager (immediate)	Lazy (on-demand)
When database hit occurs	When called	When data is accessed

Return type	Actual object	Proxy object
If record not found	Returns <code>null</code>	Throws <code>ObjectNotFoundException</code>
Usage	When unsure if record exists	When record definitely exists
Performance	Slightly slower (immediate query)	Better if object may not be used

Summary (For Exams)

- `get()` : Immediately fetches data from DB and returns `null` if not found.
- `load()` : Returns a **proxy object** and fetches data only when needed; throws exception if not found.

Example:

```
Student s1 = session.get(Student.class, 101); // Fetch now
```

```
Student s2 = session.load(Student.class, 102); // Fetch later
```

Qus 5. Write short notes on the following :

- (a) Java Cryptography Architecture (JCA)
- (b) JSP Standard Tag Library (JSTL)
- (c) JSP Life Cycle
- (d) Factory Design Pattern

Ans :

- (a) Java Cryptography Architecture (JCA)

Java Cryptography Architecture (JCA) is a framework that provides **cryptographic services** in Java.

It allows developers to implement **encryption, decryption, digital signatures, and secure key management**.

Key Components:

1. **Message Digests** – Generates hash values (e.g., SHA-256).
2. **Digital Signatures** – Ensures authenticity and integrity.
3. **Key Management** – Handles cryptographic keys securely.
4. **Cipher** – For encryption and decryption using algorithms like AES, DES.
5. **Provider** – Supplies the implementation of cryptographic algorithms.

Advantages:

- Secure data transmission.
- Standardized APIs.
- Extensible through providers.

(b) JSP Standard Tag Library (JSTL)

JSTL is a collection of **custom JSP tags** that simplifies **common JSP tasks** without writing Java code in scriptlets.

Core JSTL Tags:

1. **Core Tags (<c: . . . >)**: Conditional, looping, URL rewriting.

Example:

```
<c:if test="${user != null}">
```

```
Welcome, ${user}!
```

```
</c:if>
```

2. **Formatting Tags (<fmt: . . . >)**: Format dates, numbers, and currency.
3. **SQL Tags (<sql: . . . >)**: Execute queries and data manipulation.

4. **XML Tags (<x: . . . >):** Process XML documents.

Advantages:

- Reduces Java code in JSP.
- Improves readability and maintainability.
- Provides built-in functions for common tasks.

(c) JSP Life Cycle

A **JSP** page goes through the following **life cycle phases**:

Phase	Description
1. Translation	JSP is converted into a Servlet by the container.
2. Compilation	Generated Servlet is compiled into a .class file.
3. Initialization	jspInit() method is called once. Used for setup.
4. Execution	jspService() method is called for every request. Outputs dynamic content.
5. Cleanup / Destruction	jspDestroy() is called once before removing JSP from memory.

Diagrammatic Flow

JSP Page → Translation → Compilation → Servlet Initialization

→ Request Handling (jspService)

→ Response → Cleanup (jspDestroy)

(d) Factory Design Pattern

Factory Pattern is a **creational design pattern** that provides an **interface to create objects** but allows subclasses to decide **which class to instantiate**.

Advantages:

- Encapsulates object creation.
- Promotes loose coupling.
- Easier to extend with new types.

Example

```
// Product Interface
interface Shape {
    void draw();
}

// Concrete Products
class Circle implements Shape {
    public void draw() { System.out.println("Drawing Circle"); }
}

class Rectangle implements Shape {
    public void draw() { System.out.println("Drawing Rectangle"); }
}

// Factory Class
class ShapeFactory {
    public Shape getShape(String shapeType) {
        if(shapeType.equalsIgnoreCase("CIRCLE")) return new Circle();
        else if(shapeType.equalsIgnoreCase("RECTANGLE")) return new
Rectangle();
        return null;
    }
}

// Client
public class FactoryDemo {
    public static void main(String[] args) {
        ShapeFactory factory = new ShapeFactory();
        Shape s1 = factory.getShape("CIRCLE");
        s1.draw(); // Output: Drawing Circle
    }
}
```



```
}  
}
```

Summary Table

Topic	Key Points
JCA	Provides cryptographic services in Java: Cipher, MessageDigest, Digital Signature.
JSTL	Predefined JSP tags for loops, conditions, formatting, SQL, XML.
JSP Life Cycle	Translation → Compilation → Initialization → Execution (jspService) → Cleanup.
Factory Pattern	Encapsulates object creation, promotes loose coupling, example: ShapeFactory.

December, 2022

Qus 1.

(a) What is web application ? Explain the use of web server for development of web applications. Also, describe the differences between web server and web container.

Ans :

What is a Web Application?

A **web application** is a software program that runs on a **web server** and can be accessed through a **web browser** using the internet or an intranet.

→ In simple terms:

A web application is an application that users interact with through a browser (like Chrome or Firefox) instead of installing it locally.

Examples:

- Online Banking System
- Gmail, Facebook, YouTube
- E-commerce sites like Amazon or Flipkart
- Student Portal or Online Exam System

Components of a Web Application

1. **Client (Browser)** – Sends request (HTTP/HTTPS).
2. **Web Server** – Handles the request and forwards it to the application
3. **Application Logic** – Business logic written in Java, PHP, etc.
4. **Database** – Stores data used by the application

Use of Web Server for Development of Web Applications

A **web server** is software that:

- Receives **HTTP requests** from clients (browsers).
- Processes those requests (directly or via a web container).

- Sends **HTTP responses** (like HTML, JSON, etc.) back to the client.

Common Web Servers:

- **Apache HTTP Server**
- **Nginx**
- **Microsoft IIS**
- **Tomcat (also acts as a web container)**

Role of Web Server in Web Application Development

Function	Description
Request Handling	Accepts client requests and sends responses.
Hosting	Hosts web pages and dynamic content.
Routing	Forwards requests to the correct web container or application.
Security	Manages HTTPS, SSL certificates, and user authentication.
Logging	Records client requests and server responses for debugging.
Load Management	Balances load among multiple servers for scalability.

Example:

When you type `http://localhost:8080/myapp`,

- The **web server** (Tomcat) receives your request,
- Passes it to the **Servlet (in the web container)**,
- Gets the response and sends it back to the browser.

Difference Between Web Server and Web Container

Feature	Web Server	Web Container (Servlet Container)
---------	------------	-----------------------------------

Definition	A server that handles HTTP requests and responses.	A part of the web server that manages the execution of Servlets and JSPs.
Main Function	Serves static content (HTML, CSS, JS, images).	Executes dynamic content (Servlets, JSP).
Example	Apache HTTP Server, Nginx, IIS	Apache Tomcat, Jetty, GlassFish
Processes	Handles only HTTP requests.	Manages servlet lifecycle (init, service, destroy).
Communication	Sends static files directly to the browser.	Interacts with Java code to generate dynamic responses.
Dependency	Works independently.	Works inside or along with a web server.
Type of Content	Static content	Dynamic content

Relationship Between the Two:

Browser → Web Server → Web Container → Servlet/JSP → Response → Browser

- **Web Server** handles HTTP communication.
- **Web Container** (inside the server) runs the servlet or JSP that generates the dynamic output.

Summary

Aspect	Explanation
Web Application	Software accessible via a browser using HTTP/HTTPS.

Web Server	Handles client requests, hosts web apps, and serves responses.
Web Container	Executes servlets/JSPs and manages their lifecycle.
Key Difference	Web server → static content; Web container → dynamic content.

Short Exam Answer Summary:

A **web application** is an internet-based application accessed via a browser.

A **web server** is used to host and deliver web applications by handling client requests and responses.

A **web container** (like Tomcat) executes dynamic content like servlets and JSPs.

Web Server serves **static content**, while **Web Container** handles **dynamic Java-based content**.

Would you like me to include a **simple diagram** showing the flow from **browser** → **web server** → **web container** → **servlet** → **database** → **response** for better visualization?

(b) What is design pattern ? Explain the use of Singleton design pattern, with the help of an example.

Ans :

What is a Design Pattern?

A **Design Pattern** is a **standard and reusable solution** to a **common software design problem** that occurs frequently in software development.

 In simple terms:

A **design pattern** provides a **proven blueprint or template** for solving particular design problems in object-oriented programming.

Key Points:

- It represents **best practices** used by experienced developers.
- Improves **code reusability, readability, and maintainability**
- Promotes **loose coupling** and **flexibility** in software design.

Categories of Design Patterns:

1. **Creational Patterns** – Deal with object creation (e.g., Singleton, Factory, Builder).
2. **Structural Patterns** – Deal with object composition (e.g., Adapter, Decorator).
3. **Behavioral Patterns** – Deal with communication between objects (e.g., Observer, Strategy).

Singleton Design Pattern

Definition:

The **Singleton pattern** ensures that **only one instance** of a class is created throughout the application and provides a **global point of access** to that instance.

When to Use Singleton:

- To manage **shared resources**, like:
 - Database connections
 - Logging
 - Configuration files
 - Thread pools

Key Features:

1. Only **one instance** of the class exists.
2. Provides a **global access point** to that instance.
3. Restricts **direct instantiation** from outside the class.

3 Example: Singleton Design Pattern in Java

```
// Singleton Class
public class DatabaseConnection {
    // Step 1: Create a private static instance of the class
    private static DatabaseConnection instance;
```

```

// Step 2: Make constructor private to prevent object creation
private DatabaseConnection() {
    System.out.println("Database Connected!");
}

// Step 3: Provide a public static method to get the single instance
public static DatabaseConnection getInstance() {
    if (instance == null) {
        instance = new DatabaseConnection(); // Create new instance
only once
    }
    return instance;
}
}

// Testing the Singleton Pattern
public class Main {
    public static void main(String[] args) {
        DatabaseConnection db1 = DatabaseConnection.getInstance();
        DatabaseConnection db2 = DatabaseConnection.getInstance();

        // Both objects will point to the same instance
        System.out.println(db1 == db2); // Output: true
    }
}

```

Output:

Database Connected!

true

✓ Explanation:

- The constructor is **private**, so no new object can be created using **new**.
- **getInstance()** creates an object only once and returns the same object every time it's called.

- Both **db1** and **db2** refer to the **same instance**, proving the Singleton pattern.

Advantages of Singleton Pattern

Advantage	Description
Controlled access	Ensures only one instance exists.
Memory efficiency	Avoids multiple object creation.
Global access	Provides a single, globally accessible point.
Consistency	Maintains consistent behavior across the system.

Summary

Aspect	Description
Design Pattern	A reusable solution to common software design problems.
Singleton Pattern	Ensures only one instance of a class exists.
Use Case	Database connections, logging, configuration managers.
Main Principle	Controlled instance creation + global access point.

Short Exam Answer:

A **Design Pattern** is a reusable solution to common software design problems.
 The **Singleton Design Pattern** ensures that only one instance of a class is created and provides a global point of access to it.

Example: A database connection manager where multiple requests share the same instance.

Would you like me to include the **UML diagram of the Singleton Pattern** (class + client relationship) for your notes?

(c) What is Servlet ? Explain the use of HTTP protocol in servlet programming. Explain the use of GET and POST methods, with the help of examples.

Ans :

What is a Servlet?

A **Servlet** is a **Java program** that runs on a **web server** and is used to **create dynamic web content**.

It acts as a **middle layer** between a client request (from a browser) and the server-side resources (like databases, files, or business logic).

Definition (Simple Form):

A **Servlet** is a server-side Java component that handles client requests, processes them, and generates dynamic responses using the **HTTP protocol**.

Location & Execution:

- Servlets run inside a **web container** (like Apache Tomcat).
- The web container manages the servlet's **lifecycle** and **communication** through the **HTTP protocol**.

Use of HTTP Protocol in Servlet Programming

The **HTTP (HyperText Transfer Protocol)** is the **communication protocol** used between:

- The **client (browser)** → sends an HTTP request
- The **server (servlet)** → sends an HTTP response

Servlet Communication Process (HTTP Based):

1. Client sends an **HTTP Request** (GET/POST) to the server.
2. The **Servlet container** receives the request and passes it to the corresponding **Servlet**.
3. The **Servlet processes** the request (reads parameters, accesses DB, etc.).
4. The Servlet sends an **HTTP Response** (usually HTML or JSON) back to the client.

◆ **In Servlet:**

Servlets use two main classes from `javax.servlet.http` package:

- `HttpServletRequest` → To read client request data.
- `HttpServletResponse` → To send response back to the client.

GET and POST Methods in Servlets

Method	Description
GET	Sends data through the URL ; suitable for simple requests.
POST	Sends data through the body of the request; suitable for sensitive or large data.

A. GET Method Example

HTML Form (getExample.html):

```
<form action="GetExample" method="GET">
  Name: <input type="text" name="username">
  <input type="submit" value="Submit">
</form>
```

Servlet (GetExample.java):

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
```

```

public class GetExample extends HttpServlet {
    public void doGet(HttpServletRequest request, HttpServletResponse
response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        String name = request.getParameter("username");
        out.println("<h2>Hello, " + name + "! (Using GET method)</h2>");
    }
}

```

✓ Explanation:

- The form sends data using the **GET** method.
- The data appears in the URL (e.g., `?username=Ritesh`).
- Servlet uses `request.getParameter("username")` to fetch data.

B. POST Method Example

HTML Form (postExample.html):

```

<form action="PostExample" method="POST">
    Email: <input type="text" name="email">
    <input type="submit" value="Submit">
</form>

```

Servlet (PostExample.java):

```

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class PostExample extends HttpServlet {
    public void doPost(HttpServletRequest request, HttpServletResponse

```

```

response)
    throws ServletException, IOException {

    response.setContentType("text/html");
    PrintWriter out = response.getWriter();

    String email = request.getParameter("email");
    out.println("<h2>Your email is: " + email + " (Using POST
method)</h2>");
    }
}

```

✓ Explanation:

- The form sends data through the **body** of the request (not visible in the URL).
- Used for **secure** and **large data submissions**, like passwords or form details.

Key Differences: GET vs POST

Aspect	GET	POST
Data Transmission	Through URL	Through request body
Security	Less secure (data visible)	More secure (data hidden)
Data Size	Limited (max ~2048 chars)	Unlimited
Caching	Can be cached	Not cached
Use Case	Simple requests, queries	Form submissions, file uploads

Summary

Aspect	Description
--------	-------------

Servlet A Java program that handles web requests and generates dynamic responses.

HTTP Protocol The communication protocol between browser and servlet.

GET Method Sends data via URL — used for simple requests.

POST Method Sends data in the request body — used for secure and large data.

Short Exam Answer:

A **Servlet** is a Java-based server component used to create dynamic web pages. It uses the **HTTP protocol** for communication between client and server.

The **GET** method is used to request data from the server, while the **POST** method is used to send data securely to the server.

Example:

`doGet()` handles GET requests, and `doPost()` handles POST requests in the servlet.

Would you like me to include a **simple diagram showing how the HTTP request flows between client → servlet → server → response**? It helps a lot in 5-mark or 10-mark answers.

(d) What is Struts 2 ? Explain the core components of Struts 2

Ans :

What is Struts 2?

Struts 2 is an **open-source MVC (Model-View-Controller) framework** for developing **Java-based web applications**.

It simplifies the development of web applications by separating **business logic**, **presentation**, and **controller logic**.

Definition:

Struts 2 is a **Java web application framework** that follows the **MVC design pattern** and helps developers build flexible, maintainable, and scalable web applications.

Origin:

- Struts 2 is the **successor of Struts 1**, developed by merging **Apache Struts framework** with the **WebWork framework**.
- Package: `org.apache.struts2`

Struts 2 Architecture (MVC Pattern)

Model → Business logic and database.

View → JSP or HTML pages (user interface).

Controller → Struts 2 framework (Action classes + XML configuration).

 Struts 2 Request Flow (Simplified):

Client (Browser)



FilterDispatcher (Controller)



ActionProxy → ActionInvocation → Action Class (Model)



Result (JSP/View)



Response back to Client

Core Components of Struts 2

Here are the **main building blocks** of a Struts 2 application 📌

(a) Action

- The **Action class** contains the **business logic** of the application.
- It is similar to a **Servlet** but lighter and more flexible.
- Every user request is mapped to an Action class.

Example:

```
public class LoginAction {
    private String username;
    private String password;

    public String execute() {
        if ("admin".equals(username) && "1234".equals(password)) {
            return "success";
        } else {
            return "error";
        }
    }

    // Getters and setters
}
```

(b) Interceptors

- Interceptors are **objects that intercept requests and responses** before or after an Action executes.
- Used for tasks like:
 - Input validation
 - Logging
 - File upload
 - Authentication

Example:

The **params** interceptor populates request parameters into the Action class automatically.

(c) Result

- Specifies **what happens after the Action class executes**.
- Usually, it is a **JSP page** or another resource to be displayed.

Example (in `struts.xml`):

```
<result name="success">welcome.jsp</result>
```

```
<result name="error">error.jsp</result>
```

(d) Value Stack

- A **core data structure** that stores application data during request processing.
- JSP pages access Action class properties through the **Value Stack** using OGNL (Object Graph Navigation Language).

Example (in JSP):

Hello, \${username}

(e) OGNL (Object Graph Navigation Language)

- OGNL is used to **access and manipulate** data objects stored in the Value Stack.
- It allows easy data binding between JSP and Action classes.

(f) Configuration Files

struts.xml → Central configuration file that defines actions, results, and mappings.

```
<action name="login" class="com.example.LoginAction">
```

```
  <result name="success">welcome.jsp</result>
```

```
  <result name="error">error.jsp</result>
```

```
</action>
```


1. **struts.properties** → For framework-level settings.
2. **web.xml** → Defines the Struts 2 filter (**FilterDispatcher** or **StrutsPrepareAndExecuteFilter**).

(g) View (JSP Pages)

- JSP files act as the **view layer** to display data and results to users.
- Data is fetched using OGNL expressions from the Value Stack.

Example (welcome.jsp):

```
<h2>Welcome, ${username}</h2>
```

Summary Table: Core Components of Struts 2

Component	Description
Action	Contains business logic; processes user requests.
Interceptor	Pre/post-processing logic (e.g., validation, logging).
Result	Defines the next view (e.g., JSP page).
Value Stack	Stores application data during request processing.
OGNL	Expression language to access data from Value Stack.
struts.xml	Main configuration file for actions and results.
JSP/View	Presentation layer to display output.

Example: Complete Flow

1. User submits login form → **/login.action**
2. Struts 2 filter intercepts request

3. Calls **LoginAction.execute()**
4. Based on return value (“success” or “error”), loads respective JSP
5. JSP displays output using OGNL and Value Stack.

Short Exam Answer Summary:

Struts 2 is an MVC-based Java framework used to create dynamic web applications.

Its **core components** are:

- **Action** (business logic)
- **Interceptors** (pre/post processing)
- **Result** (defines view)
- **Value Stack & OGNL** (data storage and access)
- **Configuration files (struts.xml, web.xml)**

Together, they make Struts 2 flexible, reusable, and easy to maintain.

Would you like me to include a **diagram of the Struts 2 architecture (showing flow from browser → Action → JSP)** for your notes? It’s often asked in exams for 10 marks.

Qus 2.

(a) What are JSP implicit objects ? Explain the use of any four implicit objects.

Ans :

What are JSP Implicit Objects?

In **JSP (Java Server Pages)**, **implicit objects** are **predefined objects** that are **automatically created** by the JSP container.

➡ They allow developers to **access request, response, session, and application data** easily **without explicit declaration**.

Definition:

JSP **implicit objects** are built-in Java objects provided by the JSP container that can be directly used in JSP pages to interact with clients, servers, and application data.

Key Points:

- No need to create them manually (they're automatically available).
- They simplify web development by avoiding repetitive Java code.
- There are **9 implicit objects** in JSP.

List of JSP Implicit Objects

No.	Object Name	Type	Description
1	<code>request</code>	<code>HttpServletRequest</code>	Represents client request data.
2	<code>response</code>	<code>HttpServletResponse</code>	Used to send response to the client.
3	<code>out</code>	<code>JspWriter</code>	Used to send output to the client.
4	<code>session</code>	<code>HttpSession</code>	Stores user session data.
5	<code>application</code>	<code>ServletContext</code>	Represents application-wide data.
6	<code>config</code>	<code>ServletConfig</code>	Stores servlet configuration information.
7	<code>pageContext</code>	<code>PageContext</code>	Provides access to all JSP objects.
8	<code>page</code>	<code>Object</code>	Refers to the current JSP page instance (like <code>this</code>).

9 **exception** **Throwable** Handles exceptions in error pages.

Explanation of Any Four Implicit Objects

(a) **request** Object

- Represents the **HTTP request** sent by the client (browser).
- It allows access to form data, headers, and request parameters.

Example:

```
<%  
  
String name = request.getParameter("username");  
  
out.println("Welcome, " + name);  
  
%>
```

✓ Explanation:

`request.getParameter("username")` fetches data from an HTML form field named *username*.

(b) **response** Object

- Represents the **HTTP response** that will be sent back to the client.
- Used to control the response, like setting content type or redirecting pages.

Example:

```
<%  
  
response.setContentType("text/html");  
  
response.sendRedirect("welcome.jsp");  
  
%>
```

✓ Explanation:

The `response` object redirects the user to *welcome.jsp* after processing.

(c) **session** Object

- Used to **store user-specific data** across multiple requests.
- Useful for maintaining **user login information** or shopping cart data.

Example:

```
<%  
  
session.setAttribute("user", "Ritesh");  
  
String user = (String) session.getAttribute("user");  
  
out.println("Logged in user: " + user);  
  
%>
```

✓ Explanation:

The **session** object stores user data (**user = Ritesh**) that can be accessed across pages.

(d) **application** Object

- Represents the **entire web application** and is shared by **all users**.
- Used to store **global data** like counters or configuration values.

Example:

```
<%  
  
Integer count = (Integer)application.getAttribute("visitors");  
  
if(count == null) count = 1;  
  
else count++;  
  
application.setAttribute("visitors", count);  
  
out.println("Total visitors: " + count);  
  
%>
```

✓ **Explanation:**

`application` object stores a global visitor count accessible by all JSP pages.

Summary Table

Object	Scope	Purpose / Use
<code>request</code>	Request	Handles client request data (form fields, headers).
<code>response</code>	Request	Sends data back to client (redirects, headers).
<code>session</code>	Session	Stores data for a particular user session.
<code>application</code>	Application	Stores global data shared across all users.

Short Exam Answer Summary:

JSP Implicit Objects are built-in objects automatically available in JSP pages for easy access to web-related data.

Common implicit objects are:

- **request** → To read client request data
- **response** → To send output or redirect pages
- **session** → To store user session data
- **application** → To store global data shared by all users

Would you like me to include a **diagram showing scopes of implicit objects (page, request, session, application)**? It's often asked in JSP theory questions for 10 marks.

(b) Explain the use of Authorize Tag for providing view layer security, with the help of program code.

Ans :

What is the `<authorize>` Tag?

The `<authorize>` tag (from **Spring Security**) is used in the **view layer (JSP, Thymeleaf, etc.)** to **control access to specific parts of a webpage** based on the user's **roles or authorities**.

➡ It helps implement **view layer security** — meaning certain HTML or JSP content is visible **only to authorized users**.

Definition:

The `<authorize>` tag allows developers to **show or hide content in JSP pages** depending on the **user's authentication status or roles** in Spring Security.

Why Use the `<authorize>` Tag?

- To **restrict access** to specific UI elements (buttons, menus, links, etc.).
- To ensure that users **only see what they are allowed** to see.
- To prevent unauthorized users from performing admin or sensitive actions.

Adding Spring Security Tag Library

To use the `<authorize>` tag in a JSP file, first include the Spring Security tag library at the top of the JSP page:

```
<%@ taglib prefix="sec" uri="http://www.springframework.org/security/tags" %>
```

Example: Using `<authorize>` Tag in JSP

Here's a **JSP example** that restricts content visibility based on user roles 📌

File: home.jsp

```
<%@ taglib prefix="sec" uri="http://www.springframework.org/security/tags"
%>
<html>
<head><title>Spring Security Example</title></head>
<body>
```

```

<h2>Welcome to Web Application</h2>

<!-- Visible to everyone -->
<p>This content is visible to all logged-in users.</p>

<!-- Visible only to ADMIN -->
<sec:authorize access="hasRole('ROLE_ADMIN')">
    <h3>Admin Panel</h3>
    <a href="admin/dashboard">Go to Admin Dashboard</a>
</sec:authorize>

<!-- Visible only to USER -->
<sec:authorize access="hasRole('ROLE_USER')">
    <h3>User Section</h3>
    <p>Welcome, regular user! You can view your profile here.</p>
</sec:authorize>

<!-- Visible only if user is not authenticated -->
<sec:authorize access="!isAuthenticated()">
    <a href="login">Login Here</a>
</sec:authorize>

</body>
</html>

```

Explanation:

Code Part	Description
<pre><sec:authorize access="hasRole('ROLE_ADMIN')"></pre>	Displays content only for ADMIN users .
<pre><sec:authorize access="hasRole('ROLE_USER')"></pre>	Displays content only for USER role .

<code><sec:authorize access="!isAuthenticated()"></code>	Displays content for unauthenticated users (like login links).
<code>access</code> attribute	Contains Spring Expression Language (SpEL) conditions for security checks.

Spring Security Configuration (Java-Based)

Here's a minimal example of Spring Security configuration for roles 📌

```
@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {
    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http.authorizeRequests()
            .antMatchers("/admin/**").hasRole("ADMIN")
            .antMatchers("/user/**").hasRole("USER")
            .antMatchers("/", "/login").permitAll()
            .and()
            .formLogin();
    }

    @Autowired
    public void configureGlobal(AuthenticationManagerBuilder auth) throws
    Exception {
        auth.inMemoryAuthentication()
            .withUser("admin").password("{noop}admin123").roles("ADMIN")
            .and()
            .withUser("user").password("{noop}user123").roles("USER");
    }
}
```

✅ Explanation:

- Two users are created: `admin` (ROLE_ADMIN) and `user` (ROLE_USER).

- The `<authorize>` tag in JSP will automatically check these roles during rendering.

Summary Table

Feature	Description
Tag	<code><sec:authorize></code>
Library	http://www.springframework.org/security/tags
Purpose	Provides role-based visibility control in JSP pages
Common Access Conditions	<code>hasRole()</code> , <code>isAuthenticated()</code> , <code>permitAll</code> , <code>denyAll</code>
Example Usage	<pre><sec:authorize access="hasRole('ROLE_ADMIN')">...</sec:authorize></pre>

Short Exam Answer Summary:

The `<authorize>` tag in Spring Security is used to **restrict or display JSP content** based on user roles or authentication.

It provides **view-layer security**, ensuring that only authorized users can access certain parts of a web page.

Example:

```
<sec:authorize access="hasRole('ROLE_ADMIN')">
<h3>Admin Panel</h3>
</sec:authorize>
```

This code ensures only users with the `ADMIN` role can see the Admin Panel link.

Would you like me to also show the **Thymeleaf version** of this tag (using `sec:authorize` in HTML templates)? It's often used in modern Spring Boot projects.

Qus 3.

(a) What is Session Management ? Write a servlet program to demonstrate the use of HttpSession and its methods setAttribute and getAttribute.

Ans :

What is Session Management?

Session Management is the process of **maintaining user data across multiple HTTP requests** in a web application.

➡ Since the **HTTP protocol is stateless**, each request is treated as **independent** — the server doesn't remember users automatically.

So, **session management** helps the server **identify users** and **preserve their data** (like login info, shopping cart, etc.) between requests.

Definition:

Session management is a mechanism to maintain state (user data) between multiple client requests in a web application.

Ways to Manage Sessions in Java Servlets

Technique	Description
Cookies	Stores data on client-side (browser).
URL Rewriting	Adds session ID to the URL.
Hidden Form Fields	Sends hidden input data through forms.

HttpSession Stores session data on the server side.
(Recommended)

✓ **HttpSession** is the **most secure and commonly used** method.

HttpSession Interface

The `HttpSession` interface (in `javax.servlet.http`) is used to create and manage session objects.

Common Methods:

Method	Description
<code>setAttribute(String name, Object value)</code>	Stores data in the session.
<code>getAttribute(String name)</code>	Retrieves stored session data.
<code>removeAttribute(String name)</code>	Removes data from session.
<code>invalidate()</code>	Ends the session.
<code>getId()</code>	Returns the unique session ID.

Example: Servlet Program Demonstrating `HttpSession`

This example shows how to store and retrieve session data using `setAttribute()` and `getAttribute()` methods.

Step 1: HTML Form (sessionForm.html)

```
<form action="SessionServlet" method="post">
  Name: <input type="text" name="username">
  <input type="submit" value="Start Session">
</form>
```

Step 2: Servlet Program (SessionServlet.java)

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class SessionServlet extends HttpServlet {

    public void doPost(HttpServletRequest request, HttpServletResponse
response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Get user input
        String name = request.getParameter("username");

        // Create or retrieve existing session
        HttpSession session = request.getSession();

        // Store data in session
        session.setAttribute("user", name);

        out.println("<h2>Session Started Successfully!</h2>");
        out.println("<p>Stored username in session: " +
session.getAttribute("user") + "</p>");
        out.println("<p>Session ID: " + session.getId() + "</p>");
    }
}
```

Step 3: Another Servlet (DisplaySession.java)

To demonstrate retrieving data from the same session 📌

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
```

```

public class DisplaySession extends HttpServlet {

    public void doGet(HttpServletRequest request, HttpServletResponse
response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Retrieve existing session
        HttpSession session = request.getSession(false); // false = don't
create new session

        if (session != null) {
            String user = (String) session.getAttribute("user");
            out.println("<h2>Welcome back, " + user + "!</h2>");
        } else {
            out.println("<h3>No active session found. Please log in
again.</h3>");
        }
    }
}

```

Step 4: web.xml Configuration

```

<web-app>
  <servlet>
    <servlet-name>SessionServlet</servlet-name>
    <servlet-class>SessionServlet</servlet-class>
  </servlet>

  <servlet-mapping>
    <servlet-name>SessionServlet</servlet-name>
    <url-pattern>/SessionServlet</url-pattern>
  </servlet-mapping>

  <servlet>
    <servlet-name>DisplaySession</servlet-name>
    <servlet-class>DisplaySession</servlet-class>

```

```

</servlet>

<servlet-mapping>
  <servlet-name>DisplaySession</servlet-name>
  <url-pattern>/DisplaySession</url-pattern>
</servlet-mapping>
</web-app>

```

Explanation:

1. The user submits their name in the form.
2. `SessionServlet` creates an `HttpSession` and stores "user" using `setAttribute()`.
3. When the same user accesses `DisplaySession`, data is fetched using `getAttribute()`.
4. The session persists until it's invalidated or expires.

Output Example:

After submitting the form:

Session Started Successfully!

Stored username in session: Ritesh

Session ID: 9F12C34A1B

On accessing `/DisplaySession`:

Welcome back, Ritesh!

Summary Table

Method

Purpose

<code>setAttribute("name", value)</code>	Store data in session
<code>getAttribute("name")</code>	Retrieve stored data
<code>getId()</code>	Returns session ID
<code>invalidate()</code>	Ends user session

Short Exam Answer Summary:

Session Management is the process of maintaining user data between multiple HTTP requests.

In Servlets, the **HttpSession** interface is used for this purpose.

Methods like `setAttribute()` and `getAttribute()` help store and retrieve session data.

Example:

```
session.setAttribute("user", "Ritesh");
```

```
String name = (String) session.getAttribute("user");
```

This stores "Ritesh" in the session and retrieves it later.

Would you like me to include a **diagram showing how the client, server, and HttpSession interact** (for 10-mark visual answers)?

(b) Explain how Java Secure Socket Extension (JSSE) is used in web security.

Ans :

Java Secure Socket Extension (JSSE)

What is JSSE?

JSSE (Java Secure Socket Extension) is a **set of packages in Java** that provides a framework and implementation for **secure communication over networks** using protocols like **SSL (Secure Sockets Layer)** and **TLS (Transport Layer Security)**.

→ It enables **encryption, authentication, and data integrity** between client and server applications written in Java.

Definition:

JSSE is a Java API that provides support for **secure network communication** using **SSL and TLS protocols**, ensuring **confidentiality, integrity, and authentication** of data transmitted over the internet.

Purpose of JSSE in Web Security

Purpose	Description
Encryption	Protects sensitive data (like passwords, bank info) from being read by unauthorized parties.
Authentication	Verifies the identity of clients and servers through digital certificates.
Integrity	Ensures that the data is not modified or corrupted during transmission.
Confidentiality	Uses SSL/TLS to ensure that only the intended recipient can read the data.

Components of JSSE

JSSE mainly consists of the following components:

Component	Description
SSLSocket and SSLServerSocket	Used to create secure client and server sockets that communicate over SSL/TLS.

SSLContext	Acts as a factory for secure socket factories; manages SSL configuration.
KeyManager and TrustManager	Handle authentication and trust management using key stores and certificates.
KeyStore	Stores cryptographic keys and certificates used for authentication.

How JSSE Ensures Web Security

When a client communicates with a web server using JSSE:

1. **Handshake Phase:**
 - Client and server exchange certificates.
 - Authentication of both parties occurs (via **KeyManager / TrustManager**)
 - Session keys are generated for encryption.
2. **Data Transfer Phase:**
 - Data is encrypted using the session key.
 - Integrity checks ensure no tampering.
 - The connection remains secure until closed.

Example: JSSE Secure Communication

Below is a simple example of using JSSE for a **secure client-server connection**:

Server Program:

```
import javax.net.ssl.*;
import java.io.*;
import java.security.KeyStore;

public class SecureServer {
    public static void main(String[] args) throws Exception {
        System.setProperty("javax.net.ssl.keyStore", "serverkeystore.jks");
        System.setProperty("javax.net.ssl.keyStorePassword", "password");
```

```

        SSLServerSocketFactory ssf = (SSLServerSocketFactory)
SSLServerSocketFactory.getDefault();
        SSLServerSocket serverSocket = (SSLServerSocket)
ssf.createServerSocket(9999);

        System.out.println("Secure server started... waiting for client.");

        SSLSocket socket = (SSLSocket) serverSocket.accept();
        BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);

        String message = in.readLine();
        System.out.println("Received from client: " + message);
        out.println("Secure Hello from Server!");
    }
}

```

Client Program:

```

import javax.net.ssl.*;
import java.io.*;

public class SecureClient {
    public static void main(String[] args) throws Exception {
        System.setProperty("javax.net.ssl.trustStore",
"clienttruststore.jks");
        System.setProperty("javax.net.ssl.trustStorePassword", "password");

        SSLSocketFactory sf = (SSLSocketFactory)
SSLSocketFactory.getDefault();
        SSLSocket socket = (SSLSocket) sf.createSocket("localhost", 9999);

        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
        BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));

        out.println("Hello Secure Server!");
        System.out.println("Server Response: " + in.readLine());
    }
}

```

```
}
}
```

How This Works

1. The server and client both use **SSL/TLS sockets** (`SSLSocket`, `SSLServerSocket`).
2. The **keystore** and **truststore** hold digital certificates.
3. During the handshake:
 - The server authenticates itself using its certificate.
 - A secure channel is established
4. Data sent through this connection is **encrypted and secure**.

Advantages of Using JSSE

Advantage	Description
Security	Provides built-in SSL/TLS encryption.
Authentication	Supports both one-way and mutual authentication.
Data Integrity	Prevents message tampering.
Flexibility	Works with any Java-based network application.
Compatibility	Fully integrated with Java SE since JDK 1.4+.

Summary for Exam Answer

JSSE (Java Secure Socket Extension) provides APIs for enabling **SSL/TLS security** in Java applications.

It ensures **confidentiality, integrity, and authentication** during data transmission.

JSSE uses **SSLContext**, **SSLSocket**, **SSLServerSocket**, **KeyManager**, and **TrustManager** to create secure network communication.

Example:

```
SSLServerSocketFactory ssf = (SSLServerSocketFactory) SSLServerSocketFactory.getDefault();
SSLServerSocket server = (SSLServerSocket) ssf.createServerSocket(9999);
```

This creates a **secure SSL/TLS-enabled server socket** for encrypted communication.

Would you like me to include a **diagram of JSSE architecture** (showing client, SSL layer, and server) for better 10-mark exam presentation?

Qus 4.

(a) Explain core container and its components in Spring Framework.

Ans :

The **Core Container** is the **fundamental part** of the **Spring Framework**.

It provides the **basic functionality for managing beans (objects)** — including their creation, configuration, and lifecycle.

➡ In simple terms, it is the **heart of Spring**, responsible for **Dependency Injection (DI)** and **Inversion of Control (IoC)**.

Definition:

The **Core Container** in Spring Framework is a group of modules that provide the **IoC and Dependency Injection features**, enabling developers to build loosely coupled applications.

Core Container Modules

The Core Container consists of **four main modules**:

Module

Description

- | | |
|------------------------------------|--|
| 1. spring-core | Provides the fundamental parts of the Spring Framework, including the IoC and Dependency Injection mechanisms. |
| 2. spring-beans | Contains the BeanFactory and related classes to manage bean definitions, lifecycle, and configuration. |
| 3. spring-context | Provides context information to the Spring application and builds on the beans module. |
| 4. spring-expression (SpEL) | Supports powerful expression language for querying and manipulating objects at runtime. |

Components Explained

(a) spring-core Module

- Foundation of the Spring framework.
- Provides essential utilities and classes for dependency injection.
- Contains interfaces such as **BeanFactory**.

Example:

```
ApplicationContext context = new ClassPathXmlApplicationContext("applicationContext.xml");  
MyService obj = (MyService) context.getBean("myService");
```

(b) spring-beans Module

- Handles **bean definitions, property injection, and bean lifecycle**.
- Responsible for **instantiating, configuring, and managing beans**.
- Works closely with the **spring-core** module.

Example XML snippet:

```
<bean id="myService" class="com.example.MyService" />
```

(c) spring-context Module

- Builds on the **core** and **beans** modules.
- Provides the **ApplicationContext** interface, which is a more advanced container than **BeanFactory**.
- Supports features like:
 - Internationalization (i18n)
 - Resource loading
 - Event propagation

Example:

```
ApplicationContext context = new ClassPathXmlApplicationContext("applicationContext.xml");
```

(d) spring-expression (SpEL) Module

- Short for **Spring Expression Language**.
- Enables querying and manipulating object graphs at runtime.
- Often used in annotations like **@Value**.

Example:

```
@Value("#{2 + 3}") // Injects 5
```

```
private int sum;
```

How It Works

1. **Spring Core** provides the base framework.
2. **Spring Beans** defines and manages objects (beans).
3. **Spring Context** offers a complete framework environment to access beans.
4. **SpEL** adds dynamic expression evaluation.

➡ Together, they form the **Core Container**, enabling **dependency injection** and **loose coupling**.

Example Structure

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="helloBean" class="com.example.HelloWorld">
        <property name="message" value="Welcome to Spring Core
Container!"/>
    </bean>

</beans>
```

Java Class:

```
public class HelloWorld {
    private String message;
    public void setMessage(String message) { this.message = message; }
    public void getMessage() { System.out.println(message); }
}
```

Main Class:

```
public class MainApp {
    public static void main(String[] args) {
        ApplicationContext context = new
ClassPathXmlApplicationContext("applicationContext.xml");
        HelloWorld obj = (HelloWorld) context.getBean("helloBean");
        obj.getMessage();
    }
}
```


Advantages of Core Container

Advantage	Description
Loose Coupling	Promotes low dependency between components.
Reusability	Beans can be reused across applications.
Easy Configuration	Beans can be managed via XML, annotations, or Java code.
Testability	Easy to test components independently.

Summary for Exam Answer

The **Core Container** is the foundation of the Spring Framework. It provides **IoC and Dependency Injection** through four main modules: **spring-core, spring-beans, spring-context, and spring-expression**.

These modules manage object creation, configuration, and lifecycle, enabling developers to build **loosely coupled and easily maintainable applications**.

Core Container:

The heart of the Spring Framework that provides the IoC and DI functionality.

Modules:

1. **spring-core** – Provides core features like BeanFactory.
2. **spring-beans** – Manages bean definitions and lifecycle.
3. **spring-context** – Provides ApplicationContext and resource access.
4. **spring-expression (SpEL)** – Enables dynamic expression evaluation.

Together, these form the **foundation of Spring applications**.

Would you like me to include a **diagram showing the Core Container modules and their relationships** (for visual 10-mark answers)?

(b) Explain how CRUD is mapped to SQL statements, with examples.

Ans :

CRUD stands for **Create, Read, Update, Delete** — the four basic operations performed on a database table.

Operation	Description
Create	Insert new records into a table.
Read	Retrieve existing records from a table.
Update	Modify existing records in a table.
Delete	Remove records from a table.

Mapping CRUD to SQL Statements

CRUD Operation	SQL Statement	Explanation
Create	<code>INSERT INTO table_name (column1, column2) VALUES (value1, value2);</code>	Adds a new row in the table.
Read	<code>SELECT column1, column2 FROM table_name WHERE condition;</code>	Fetches data from the table.
Update	<code>UPDATE table_name SET column1=value1 WHERE condition;</code>	Modifies existing row(s).
Delete	<code>DELETE FROM table_name WHERE condition;</code>	Removes row(s) from the table.

Example: Employee Table

Assume we have a table **Employee**:

emp_id	name	salary
--------	------	--------

101	Ritesh	50000
-----	--------	-------

102	Priya	45000
-----	-------	-------

(a) CREATE Operation

INSERT INTO Employee (emp_id, name, salary) VALUES (103, 'Ankit', 40000);

✓ Explanation:

- Adds a new employee record with **emp_id = 103**.

(b) READ Operation

```
SELECT * FROM Employee;  
SELECT name, salary FROM Employee WHERE emp_id = 101;
```

✓ Explanation:

- First query fetches **all employees**.
- Second query fetches only **Ritesh's name and salary**.

UPDATE Operation

```
UPDATE Employee SET salary = 55000 WHERE emp_id = 101;
```

✓ Explanation:

- Updates **Ritesh's salary** to 55000.

(d) DELETE Operation

```
DELETE FROM Employee WHERE emp_id = 102;
```

✓ Explanation:

- Deletes the employee record with `emp_id = 102` (Priya).

Mapping CRUD in Java (JDBC Example)

Java Example for Employee Table:

```
Connection con = DriverManager.getConnection(url, user, password);
Statement stmt = con.createStatement();

// CREATE
stmt.executeUpdate("INSERT INTO Employee VALUES(104, 'Simran', 48000)");

// READ
ResultSet rs = stmt.executeQuery("SELECT * FROM Employee");
while(rs.next()) {
    System.out.println(rs.getInt("emp_id") + " " + rs.getString("name"));
}

// UPDATE
stmt.executeUpdate("UPDATE Employee SET salary=50000 WHERE emp_id=104");

// DELETE
stmt.executeUpdate("DELETE FROM Employee WHERE emp_id=104");
```

✓ Explanation:

- `executeUpdate()` is used for **INSERT, UPDATE, DELETE**.
- `executeQuery()` is used for **SELECT** statements.

Summary Table

CRUD	SQL Command	Purpose
Create	INSERT	Add new record
Read	SELECT	Fetch records
Update	UPDATE	Modify existing records
Delete	DELETE	Remove records

Short Exam Answer Summary

CRUD operations are the basic database operations:

Create → INSERT, **Read** → SELECT, **Update** → UPDATE, **Delete** → DELETE.

Example:

```
INSERT INTO Employee VALUES (103, 'Ankit', 40000);
SELECT * FROM Employee WHERE emp_id=101;
UPDATE Employee SET salary=55000 WHERE emp_id=101;
DELETE FROM Employee WHERE emp_id=102;
```

Qus 5.

(a) What is Role-based Login ? Explain how user's access can be restricted using Role-based Login.

Ans :

Role-Based Login is a **security mechanism** where users are **granted access to an application based on their roles**.

- Each user is assigned one or more **roles** (e.g., ADMIN, USER, GUEST).
- The **access rights** (pages, menus, actions) are determined by the user's role.
- Helps enforce **authorization** after authentication.

♦ Definition:

Role-Based Login is a system where **access to application resources is controlled based on user roles**, ensuring users see only what they are permitted to access.

♦ **Advantages:**

Advantage	Explanation
Security	Prevents unauthorized users from accessing sensitive data.
Manageability	Roles simplify access control for multiple users.
Scalability	Adding new users doesn't require changing security logic; just assign a role.
Flexibility	Different roles can have different privileges.

How Access is Restricted Using Roles

1. **User Authentication:** Verify user credentials (username & password).
2. **Role Assignment:** Assign a role (ADMIN, USER, etc.) to the authenticated user.
3. **Authorization Check:** Grant or deny access to resources based on the role.

Flow Diagram (Conceptual)

User Login → Authentication → Check User Role → Grant/Deny Access

Example:

- ADMIN → Access to Admin Dashboard
- USER → Access to User Dashboard
- GUEST → Access to public pages only

Example in Java Servlet (Role-Based Login)

Step 1: HTML Login Form (login.html)

```
<form action="LoginServlet" method="post">
    Username: <input type="text" name="username">
    Password: <input type="password" name="password">
    <input type="submit" value="Login">
</form>
```

Step 2: Servlet Implementation (LoginServlet.java)

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.util.*;

public class LoginServlet extends HttpServlet {
    // Sample users and roles
    private Map<String, String> userRoleMap = Map.of(
        "admin", "ADMIN",
        "user", "USER"
    );

    private Map<String, String> userPassMap = Map.of(
        "admin", "admin123",
        "user", "user123"
    );

    protected void doPost(HttpServletRequest request, HttpServletResponse
response)
        throws ServletException, IOException {

        String username = request.getParameter("username");
        String password = request.getParameter("password");

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        if(userPassMap.containsKey(username) &&
userPassMap.get(username).equals(password)) {
            // Authentication successful
            String role = userRoleMap.get(username);
```

```

        HttpSession session = request.getSession();
        session.setAttribute("username", username);
        session.setAttribute("role", role);

        // Role-based redirection
        if("ADMIN".equals(role)) {
            response.sendRedirect("adminDashboard.jsp");
        } else if("USER".equals(role)) {
            response.sendRedirect("userDashboard.jsp");
        }
        } else {
            out.println("<h3>Invalid username or password</h3>");
        }
    }
}

```

Step 3: Role-Based Access in JSP Pages

adminDashboard.jsp

```

<%
String role = (String) session.getAttribute("role");
if(!"ADMIN".equals(role)) {
    response.sendRedirect("accessDenied.jsp");
}
%>
<h2>Welcome Admin!</h2>

```

userDashboard.jsp

```

<%
String role = (String) session.getAttribute("role");
if(!"USER".equals(role)) {
    response.sendRedirect("accessDenied.jsp");
}
%>
<h2>Welcome User!</h2>

```


`accessDenied.jsp`

`<h3>Access Denied! You are not authorized to view this page.</h3>`

How It Works

1. User logs in via `login.html`.
2. `LoginServlet` authenticates credentials.
3. Assigns role (ADMIN/USER) to session.
4. Each JSP page checks the session role before displaying content.
5. Unauthorized access redirects to `accessDenied.jsp`.

Key Points

Feature	Description
Authentication	Verify username & password.
Role Assignment	Assign a specific role to the user.
Authorization	Check role before granting access to pages.
Session Tracking	Use <code>HttpSession</code> to store user role.

Short Exam Answer Summary

Role-Based Login is a system where users are assigned roles, and their access to application resources is restricted based on those roles.

Implementation steps:

1. Authenticate user credentials.
2. Assign role (e.g., ADMIN, USER) after successful login.
3. Use session or security checks to restrict page access based on the role.

Example: `if("ADMIN".equals(session.getAttribute("role"))) { show
admin content } else { redirect to accessDenied.jsp }.`

(b) Write short notes on the following :

(i) Servlet Collaboration

(ii) JSP Life Cycle

Ans :

(i) Servlet Collaboration

Definition:

Servlet collaboration is the **technique by which multiple servlets work together** to process a single client request or share data. It allows **reuse of existing servlets** and modular development.

Ways Servlets Collaborate:

1. **RequestDispatcher (Forwarding & Including)**

Forward: Passes a request from one servlet to another.

```
RequestDispatcher rd = request.getRequestDispatcher("NextServlet");  
rd.forward(request, response);
```

Include: Includes output of another servlet or JSP into the response.

```
rd.include(request, response);
```

2. **ServletContext (Sharing Data Across Servlets)**

Use `setAttribute()` and `getAttribute()` to share data.

```
getServletContext().setAttribute("key", value);
```

```
Object data = getServletContext().getAttribute("key");
```

3. Session Tracking

Use `HttpSession` to share user-specific data among servlets for the same session.

Advantages of Servlet Collaboration:

- **Code Reusability:** Reuse common functionality.
- **Modularity:** Break large programs into smaller, manageable servlets.
- **Efficiency:** Forwarding avoids multiple client requests.

(ii) JSP Life Cycle

Definition:

JSP life cycle is the **process by which a JSP page is converted into a servlet** and executed by the JSP container.

Phases of JSP Life Cycle:

1. **Translation Phase**
 - JSP page is converted into a **Java servlet** by the container.
2. **Compilation Phase**
 - The generated servlet is **compiled into a class file**.
3. **Loading & Instantiation**
 - Servlet class is **loaded into memory** and an instance is created.
4. **Initialization**
 - `jspInit()` method is called to perform initialization tasks.
5. **Request Handling**
 - `jspService()` method is called **for every client request** to generate dynamic content.
6. **Destroy**
 - `jspDestroy()` method is called when the container removes the JSP, releasing resources.

Life Cycle Methods:

Method	Purpose
--------	---------

`jspInit()` Initialization (called once)

`jspService()` Handles client requests (called multiple times)

`jspDestroy()` Cleanup before JSP is destroyed

Advantages of JSP Life Cycle Understanding:

- Helps optimize **resource management**.
- Explains **when and how JSP is converted into servlet**.
- Useful for **custom initialization and cleanup** tasks.

Short Exam Summary:

1. Servlet Collaboration

- Multiple servlets work together using `RequestDispatcher`, `ServletContext`, or `HttpSession`.
- Benefits: modularity, code reuse, and efficiency.

2. JSP Life Cycle

- Phases: Translation → Compilation → Loading → Initialization → Request Handling → Destroy.
- Key methods: `jspInit()`, `jspService()`, `jspDestroy()`.
- Ensures **dynamic content generation and resource management**.

June, 2023

Qus 1.

(a) Differentiate between web server and web container. Give example for each.

Ans :

Web Server vs Web Container

Feature	Web Server	Web Container (Servlet Container)
Definition	A web server is software that handles HTTP requests from clients (browsers) and serves static content like HTML, CSS, images, JavaScript.	A web container is a component of a web server that manages Java-based web applications (servlets, JSPs) and executes dynamic content .
Content Served	Static content (HTML, images, CSS, JS).	Dynamic content (Servlets, JSP, EJB integration).
Execution	Does not execute Java code ; just serves files.	Executes Java classes (Servlets/JSP) and provides lifecycle management.
HTTP Support	Basic support for HTTP requests and responses.	Full support for HTTP along with servlet/JSP APIs.
Lifecycle Management	No lifecycle management for Java objects.	Manages lifecycle of servlets (<code>init()</code> , <code>service()</code> , <code>destroy()</code>).
Examples	Apache HTTP Server, Nginx, Microsoft IIS	Apache Tomcat, Jetty, GlassFish (servlet/JSP container)

Key Points:

- Web server handles **static content**, whereas web container handles **dynamic Java content**.
- Web container is sometimes integrated into a web server (e.g., Tomcat can act as both).
- Web containers follow **Servlet/JSP specifications**.

Example Scenario:

- If you request `index.html` → Web server (Apache HTTP Server) serves the file directly.
- If you request `login.jsp` → Web container (Tomcat) executes JSP, generates HTML, and sends it back to the browser.

Short Exam Tip:

Remember: **Web Server** → **Static**, **Web Container** → **Dynamic (Servlet/JSP)**.

Examples: Apache HTTP Server (Web Server), Tomcat (Web Container).

(b) Compare `GenericServlet` with `HttpServlet`.

Ans :

Comparison: `GenericServlet` vs `HttpServlet`

Feature	<code>GenericServlet</code>	<code>HttpServlet</code>
Definition	An abstract class that implements <code>Servlet</code> interface and provides a generic framework for any protocol.	An abstract class that extends <code>GenericServlet</code> specifically for HTTP protocol .
Protocol Support	Protocol-independent (can be used for FTP, SMTP, etc.).	Protocol-dependent (only for HTTP).
Service Method	Must override <code>service(ServletRequest request, ServletResponse response)</code> method.	Overrides <code>service(HttpServletRequest request, HttpServletResponse response)</code> method. Also provides

`doGet()`, `doPost()`, `doPut()`, etc. for HTTP methods.

Convenience	Less convenient for HTTP requests; developer has to manually cast <code>ServletRequest</code> to <code>HttpServletRequest</code> .	Convenient for HTTP; directly provides methods like <code>doGet()</code> , <code>doPost()</code> , etc.
Use Case	Used for writing servlets for any protocol.	Used for writing HTTP web applications .
Inheritance	Implements <code>Servlet</code> and <code>ServletConfig</code> interfaces.	Extends <code>GenericServlet</code> and provides HTTP-specific functionality.
Example	A servlet that handles generic protocol messages (not just HTTP).	A servlet that handles GET/POST requests from a browser .

Key Points:

1. **GenericServlet** is protocol-independent; **HttpServlet** is HTTP-specific.
2. **HttpServlet** is widely used for web applications.
3. **HttpServlet** provides convenient methods like `doGet()` and `doPost()`, reducing boilerplate code.

Example: `doGet()` in `HttpServlet`

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class HelloServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse
response)
        throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
```

```

        out.println("<h1>Welcome to HttpServlet Example!</h1>");
    }
}

```

In **GenericServlet**, you would need to cast **ServletRequest** and **ServletResponse** manually to handle HTTP requests, making it less convenient.

Do you want me to also create a **diagram showing GenericServlet → HttpServlet hierarchy** for better understanding?

(c) What is session management ? Explain how cookies can be used to session management.

Ans :

Session Management is a technique used by web applications to **maintain user-specific data across multiple HTTP requests**.

HTTP is a **stateless protocol**, meaning each request is independent. Session management allows the server to **remember users** and their data during a session.

Why is it needed?

- To track logged-in users.
- To store user preferences, shopping cart, or temporary data.
- To maintain continuity in multi-page web applications.

Techniques for Session Management

1. **Cookies** – Small data stored on the client browser.
2. **URL Rewriting** – Append session ID in the URL.
3. **Hidden Form Fields** – Include session ID in hidden HTML fields.
4. **HttpSession Object** – Managed on the server side.

Using Cookies for Session Management

Cookies are **small text files** stored in the client's browser.

They can store **user-specific data**, like username or session ID, and are sent automatically with each request to the server.

Steps to Manage Session with Cookies

1. Create a Cookie on Login

```
Cookie userCookie = new Cookie("username", "Ritesh");
```

```
userCookie.setMaxAge(30*60); // 30 minutes
```

```
response.addCookie(userCookie);
```

2. Retrieve the Cookie on Subsequent Requests

```
Cookie[] cookies = request.getCookies();
if(cookies != null){
    for(Cookie c : cookies){
        if(c.getName().equals("username")){
            out.println("Welcome back, " + c.getValue());
        }
    }
}
```

3. Delete a Cookie (Logout)

```
Cookie userCookie = new Cookie("username", "");
```

```
userCookie.setMaxAge(0); // Delete immediately
```

```
response.addCookie(userCookie);
```

Advantages of Using Cookies

- Simple and easy to implement.
- Maintains user-specific information across multiple requests.
- Can persist even after closing the browser (if max age is set).

Disadvantages

- Limited size (usually 4KB).
- Users can **disable cookies** in the browser.
- Security risk if sensitive data is stored without encryption.

Example: JSP Program Using Cookies

```
<%@ page import="javax.servlet.http.*" %>
<%
    String username = request.getParameter("username");
    if(username != null){
        // Create cookie
        Cookie userCookie = new Cookie("username", username);
        userCookie.setMaxAge(30*60); // 30 minutes
        response.addCookie(userCookie);
        out.println("Cookie created. Welcome, " + username);
    } else {
        // Read cookie
        Cookie[] cookies = request.getCookies();
        if(cookies != null){
            for(Cookie c : cookies){
                if(c.getName().equals("username")){
                    out.println("Welcome back, " + c.getValue());
                }
            }
        } else {
            out.println("No cookies found.");
        }
    }
}%>
```

Summary Table

Aspect	Explanation
Session Management	Maintains user-specific data across multiple HTTP requests.

Cookies	Small text stored in the browser to identify the user/session.
Creation	<code>new Cookie(name, value)</code> and <code>response.addCookie()</code> .
Reading	<code>request.getCookies()</code> returns array of cookies.
Deleting	<code>setMaxAge(0)</code> removes the cookie.

(d) Write JSP program using scriptlets that adds numbers from 1 to 10 and prints the results. Support your code with suitable comments.

Ans :

Here's a **complete JSP program** using **scriptlets** to add numbers from 1 to 10, with **comments** explaining each step:

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
    <title>Sum of Numbers from 1 to 10</title>
</head>
<body>
    <h2>Sum of Numbers from 1 to 10</h2>

    <%
        // Declare a variable to store the sum
        int sum = 0;

        // Loop from 1 to 10
        for(int i = 1; i <= 10; i++) {
            sum += i; // Add current number to sum
        }

        // Print the result
```

```

        out.println("<p>The sum of numbers from 1 to 10 is: " + sum +
"</p>");
    %>
</body>
</html>

```

Explanation of Code

1. `<%@ page ... %>` – JSP page directive specifying Java as the language and content type.
2. `<% ... %>` – Scriptlet tags used to embed Java code inside JSP.
3. `int sum = 0;` – Initializes the sum variable.
4. `for(int i = 1; i <= 10; i++)` – Loop to iterate numbers from 1 to 10.
5. `sum += i;` – Adds each number to the sum.
6. `out.println()` – Prints the result to the browser.

✓ Output in Browser:

Sum of Numbers from 1 to 10

The sum of numbers from 1 to 10 is: 55

If you want, I can also write a **version using JSP expression tags** `<%= %>` to make the code even cleaner and avoid `out.println()`.

(e) What is a ValueStack in struts ? How ValueStack is related to OGNL (Object Graph Navigation Language) ?

Ans :

ValueStack is a **core component** of Struts 2 that stores all **data objects** (Action objects, application, session, request, and other objects) related to the current request.

It acts as a **stack of objects** that the **view layer (JSP or template)** can access to display data.

Key Points:

- Every HTTP request in Struts 2 has its **own ValueStack**.
- It stores objects in **Last-In-First-Out (LIFO)** order.
- Helps in **data transfer from Action classes to JSP pages**.
- JSP pages use **OGNL expressions** to access the data in the ValueStack.

How ValueStack Works

1. Action class executes and places its properties on the ValueStack.
2. Interceptors (like `params`, `modelDriven`) can also push objects onto the stack.
3. The JSP page (view) uses **OGNL expressions** to retrieve values from the top of the stack.

Example Flow:

Client Request → Action Execution → Action Properties pushed to ValueStack → JSP reads from ValueStack → Response

Example**Action Class (Java):**

```
package com.example;

import com.opensymphony.xwork2.ActionSupport;

public class UserAction extends ActionSupport {
    private String name = "Ritesh Tiwari";
    private int age = 25;

    public String execute() {
        return SUCCESS;
    }

    // Getters required for OGNL
    public String getName() { return name; }
    public int getAge() { return age; }
}
```

JSP Page (View)

```
<%@ taglib prefix="s" uri="/struts-tags" %>
<html>
<body>
  <h2>User Details</h2>
  <p>Name: <s:property value="name"/></p>
  <p>Age: <s:property value="age"/></p>
</body>
</html>
```

Explanation:

- The **Action object (UserAction)** is automatically pushed onto the **ValueStack**.
- **<s:property>** tag uses **OGNL expressions (name, age)** to access the properties from the **top of the ValueStack**.

Relation Between ValueStack and OGNL

Feature	Description
ValueStack	Stores all the objects and data for the current request.
OGNL (Object Graph Navigation Language)	Expression language used to access data from the ValueStack .
Integration	JSP tags like <s:property> or <s:form> use OGNL expressions to automatically get/set values from objects in the ValueStack.
Example	value="name" → OGNL searches the top object in the ValueStack for property name .

Summary

- **ValueStack** → Holds all objects for a request (Action, Model, etc.).
- **OGNL** → Language used to **read/write properties** from the ValueStack in JSP pages.
- Together, they **simplify data transfer** from backend to frontend in Struts 2.

(f) Differentiate between Spring and Hibernate.

Ans :

Spring vs Hibernate

Feature	Spring Framework	Hibernate Framework
Definition	A comprehensive Java framework for building enterprise applications, providing features like IoC, AOP, transaction management, and MVC.	An Object-Relational Mapping (ORM) framework for Java that maps Java objects to database tables and handles database operations.
Primary Purpose	Simplifies application development and integration of various Java EE technologies.	Simplifies database operations by mapping objects to relational tables, reducing JDBC boilerplate code.
Focus	Application infrastructure (IoC, AOP, MVC, Security).	Data persistence and database management.
Layer	Can be used at all layers of an application (service, DAO, controller, etc.).	Works mainly at the data access layer (DAO).

Transaction Management	Spring provides declarative and programmatic transaction management across various resources (JDBC, Hibernate, JPA).	Hibernate supports transaction management via JDBC or JTA, but often integrated with Spring for easier management.
Configuration	Can be configured using XML, annotations, or Java config .	Can be configured using XML mapping files or annotations for entity classes.
Dependency Injection (DI)	Fully supports DI to reduce coupling.	Does not provide DI ; often used with Spring for DI.
Caching	Spring provides caching support at the service layer.	Hibernate has built-in first-level and second-level caching .
Use Case	Enterprise application development with multiple layers, security, and MVC support.	Persistent data storage, ORM, database CRUD operations.
Examples	Spring Boot, Spring MVC, Spring Security.	Hibernate ORM, JPA with Hibernate provider.

Key Points

- **Spring** = Application framework for **all layers**, not just database.
- **Hibernate** = ORM framework focused on **data persistence**.
- They are **often used together**: Spring for managing application services and transactions, Hibernate for database access.

(g) What is Dependency Injection in spring framework ? Name the ways through which Dependency Injection can be performed.

Ans :

Dependency Injection (DI) is a **design pattern** in which an object (the *dependent object*) receives its **dependencies from an external source** rather than creating them itself.

In Spring, DI is the core principle of **Inversion of Control (IoC)**, which allows Spring to manage object creation and their dependencies.

Advantages of Dependency Injection

- Reduces **tight coupling** between classes.
- Makes code **more testable** (easier to mock dependencies).
- Improves **maintainability** and **readability**.
- Allows **flexible configuration** using XML, annotations, or Java config.

Ways to Perform Dependency Injection in Spring

❶ Constructor-Based Injection

- Dependencies are provided through **class constructors**.
- Ensures that **required dependencies** are injected at the time of object creation.

Example:

```
// Service class
public class EmployeeService {
    private EmployeeDAO employeeDAO;

    // Constructor-based DI
    public EmployeeService(EmployeeDAO employeeDAO) {
        this.employeeDAO = employeeDAO;
    }

    public void showDetails() {
        employeeDAO.printEmployee();
    }
}
```

Spring XML Configuration:

```
<bean id="employeeDAO" class="com.example.EmployeeDAO"/>
<bean id="employeeService" class="com.example.EmployeeService">
    <constructor-arg ref="employeeDAO"/>
</bean>
```

② Setter-Based Injection

- Dependencies are provided through **setter methods**.
- Allows optional dependencies to be injected.

Example:

```
public class EmployeeService {
    private EmployeeDAO employeeDAO;

    // Setter-based DI
    public void setEmployeeDAO(EmployeeDAO employeeDAO) {
        this.employeeDAO = employeeDAO;
    }

    public void showDetails() {
        employeeDAO.printEmployee();
    }
}
```

Spring XML Configuration:

```
<bean id="employeeService" class="com.example.EmployeeService">
    <property name="employeeDAO" ref="employeeDAO"/>
</bean>
```

③ Annotation-Based Injection

- Uses annotations such as **@Autowired** (Spring 3+) to inject dependencies.

- Works with **constructor, setter, or field injection**.

Example:

```
@Component
public class EmployeeService {
    @Autowired
    private EmployeeDAO employeeDAO;

    public void showDetails() {
        employeeDAO.printEmployee();
    }
}
```

- `@Component` marks the class as a Spring bean.
- `@Autowired` tells Spring to **inject the dependency automatically**.

Summary Table

Injection Type	How It Works	Example
Constructor Injection	Dependencies passed via constructor	<code>new EmployeeService(employeeDAO)</code>
Setter Injection	Dependencies set via setter methods	<code>employeeService.setEmployeeDAO(employeeDAO)</code>
Annotation-Based Injection	Uses annotations like <code>@Autowired</code> , <code>@Component</code>	<code>@Autowired private EmployeeDAO dao;</code>

Key Points

- **DI decouples classes** by removing direct instantiation of dependencies.
- Spring supports **XML, annotations, and Java-based configuration** for DI.

- Constructor injection is preferred for **mandatory dependencies**. Setter injection is used for **optional dependencies**.

(h) What is Bootstrap ? Briefly discuss the factors that makes Bootstrap a popular choice for the development of web applications.

Ans :

Bootstrap is a **free, open-source front-end framework** used for designing **responsive and mobile-first web pages**.

It provides **predefined CSS classes, JavaScript components, and layout templates**, which help developers **build modern web applications quickly**.

Key Features:

- Responsive grid system
- Prebuilt components (buttons, navbars, forms, modals, etc.)
- Cross-browser compatibility
- Easy to integrate with HTML, CSS, and JS

Official Website: <https://getbootstrap.com>

Factors That Make Bootstrap Popular

1 Responsive Design

- Uses a **12-column fluid grid system**.
- Ensures web pages **adapt to desktops, tablets, and mobile screens** automatically.

2 Predefined Components

- Provides ready-made UI elements like **buttons, forms, modals, carousels, navbars**.
- Saves development time and effort.

3 Cross-Browser Compatibility

- Works seamlessly with modern browsers like **Chrome, Firefox, Safari, Edge**.
- Handles browser-specific CSS issues internally.

4 Customizable

- Developers can **customize Bootstrap** by including only required components.
- Supports **SASS variables** and **themes** for branding.

5 Integration with JavaScript

- Includes interactive components like **dropdowns, modals, tabs, tooltips**.
- No need to write complex JavaScript; just include Bootstrap JS files.

6 Community Support

- Large developer community and active documentation.
- Plenty of **templates, tutorials, and third-party plugins** available.

7 Time-Saving

- Reduces **design and coding effort**.
- Rapid prototyping of web pages is possible.

Example of Bootstrap Usage

```
<!DOCTYPE html>
<html>
<head>
  <title>Bootstrap Example</title>
  <!-- Include Bootstrap CSS -->
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.c
ss" rel="stylesheet">
</head>
<body>

<div class="container">
  <h1 class="text-center mt-5">Welcome to Bootstrap</h1>
  <button class="btn btn-primary mt-3">Click Me</button>
</div>

<!-- Include Bootstrap JS -->
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.
min.js"></script>
```

```
</body>
</html>
```

Explanation:

- `container` → Bootstrap container class for responsive layout
- `btn btn-primary` → Predefined button styles
- `text-center` → Centers text
- `mt-5` → Adds margin-top using Bootstrap spacing utility

Summary Table

Feature	Description
Responsive Grid	12-column layout for mobile-first design
Prebuilt Components	Buttons, navbars, forms, modals, etc.
Cross-Browser Support	Works on Chrome, Firefox, Safari, Edge
Customizable	Can include only required components or themes
Time-Saving	Rapid UI development with minimal coding

Qus 2.

(a) What is Spring Boot ? Write the steps to deploy Spring Boot web application as JAR and WAR files ?

Ans :

Spring Boot is a framework built on top of the **Spring framework** that simplifies the development of **standalone, production-ready Spring applications**.

Key Features:

- **Auto-configuration** – Automatically configures beans based on classpath and settings.
- **Standalone applications** – Can run without external web servers using embedded servers (Tomcat, Jetty, Undertow).
- **Opinionated defaults** – Reduces boilerplate configuration.
- **Embedded servers** – Supports running as **executable JARs**.
- **Spring Boot Starter** – Predefined dependency sets for rapid development.

Deploying Spring Boot Web Application

Spring Boot applications can be deployed as:

1. **Executable JAR** (default and easiest)
2. **WAR File** (for deployment on external servlet containers like Tomcat)

Deploying as Executable JAR

Steps:

1. **Create Spring Boot Project**
 - Use Spring Initializr or IDE (Maven/Gradle project).
2. **Include Dependencies**
 - Example `pom.xml` snippet for web:

```
<dependency>
```

```
<groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-web</artifactId>
```

```
</dependency>
```

3. **Build the Project**

Using Maven:

```
mvn clean package
```

This creates a **JAR file** in the `target/` folder.

Run the JAR

```
java -jar target/myapp-0.0.1-SNAPSHOT.jar
```

Spring Boot runs with **embedded Tomcat**, accessible at `http://localhost:8080`.

2 Deploying as WAR File

Steps:

1. Modify Project Structure

- Change `pom.xml` packaging to `war`:

```
<packaging>war</packaging>
```

Extend `SpringBootServletInitializer`

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import
org.springframework.boot.web.servlet.support.SpringBootServletInitializer;
import org.springframework.boot.builder.SpringApplicationBuilder;

@SpringBootApplication
public class MyApplication extends SpringBootServletInitializer {

    @Override
    protected SpringApplicationBuilder configure(SpringApplicationBuilder
application) {
        return application.sources(MyApplication.class);
    }
}
```



```

public static void main(String[] args) {
    SpringApplication.run(MyApplication.class, args);
}
}

```

2. Build WAR File

mvn clean package

WAR file will be in **target/** folder.

3. Deploy WAR to External Server

- Copy WAR file to the **webapps** folder of Tomcat or any servlet container.
- Start the server and access at <http://localhost:8080/myapp>.

Comparison Table: JAR vs WAR Deployment

Feature	JAR Deployment	WAR Deployment
Server Required	Embedded Tomcat	External servlet container
Configuration	Minimal	Needs servlet initializer
Startup	<code>java -jar myapp.jar</code>	Deploy to webapps folder
Use Case	Standalone microservices	Traditional enterprise applications

Summary

- **Spring Boot** → Simplifies Spring apps, embedded servers, auto-configuration.
- **JAR Deployment** → Executable, quick, standalone.
- **WAR Deployment** → External container, traditional setup.
- Steps involve **project setup, packaging, and running or deploying**.

(b) What is Hibernate Query Language (HQL) ? How does HQL differ from SQL ? Write HQL queries for Select, Update, Delete and Insert (make suitable assumptions).

Ans :

Hibernate Query Language (HQL) is an **object-oriented query language** provided by Hibernate to interact with the **database using persistent Java objects** rather than tables.

Key Points:

- HQL is **similar to SQL** but works with **entity objects and their properties**.
- Queries are written in terms of **class names and attributes**, not database tables and columns.
- Fully supports **polymorphism, inheritance, and associations**.

Differences Between HQL and SQL

Feature	HQL	SQL
Query Language Type	Object-oriented	Relational
Target	Java entities (classes)	Database tables
Properties	Uses class properties (fields)	Uses columns
Joins	Can use object associations	Uses foreign keys
Database Independent	Yes, works with any DB supported by Hibernate	Database-specific syntax may vary
Example	<code>FROM Employee WHERE salary > 5000</code>	<code>SELECT * FROM employee WHERE salary > 5000</code>

HQL Queries Examples

Assumption:

- Entity Class: `Employee`
- Properties: `id`, `name`, `salary`

1) SELECT Query

```
// HQL to fetch all employees with salary > 5000
String hql = "FROM Employee E WHERE E.salary > 5000";
Query query = session.createQuery(hql);
List<Employee> results = query.list();
```

UPDATE Query

```
// HQL to increase salary by 10% for employees with id = 101

String hql = "UPDATE Employee SET salary = salary * 1.10 WHERE id = 101";

Query query = session.createQuery(hql);

int rowsUpdated = query.executeUpdate();
```

DELETE Query

```
// HQL to delete employee with id = 102

String hql = "DELETE FROM Employee WHERE id = 102";

Query query = session.createQuery(hql);

int rowsDeleted = query.executeUpdate();
```

INSERT Query

Note: HQL does not support standard `INSERT INTO ... VALUES (...)`.

It supports `INSERT INTO ... SELECT` from existing entities.

```
// HQL to insert records into another entity (EmployeeArchive) from Employee
```

```
String hql = "INSERT INTO EmployeeArchive(id, name, salary) SELECT E.id, E.name, E.salary
FROM Employee E WHERE E.salary < 3000";
```

```
Query query = session.createQuery(hql);
```

```
int rowsInserted = query.executeUpdate();
```

Summary

- **HQL** works with **Java objects**, not database tables.
- **SQL** works directly with database tables and columns.
- HQL supports **SELECT, UPDATE, DELETE, and INSERT INTO ... SELECT**, but **simple value INSERT is not supported**.
- Queries are **database-independent** and leverage **Hibernate ORM mapping**.

Qus 3.

(a) Briefly discuss the term “Spring Data Repository”. Also explain the CrudRepository, PagingAndSorting Repository, and JpaRepository interfaces of Spring DataRepository.

Ans :

Spring Data Repository is a **central concept in Spring Data JPA** that provides **abstraction for data access operations**.

It simplifies **CRUD (Create, Read, Update, Delete)** operations and reduces boilerplate code by providing **predefined interfaces** for interacting with the database.

Key Points:

- Based on **Repository Pattern**.
- Supports **automatic implementation** of queries using method naming conventions.
- Reduces the need to write explicit **DAO implementations**.
- Works with various persistence stores (JPA, MongoDB, Elasticsearch, etc.).

Main Repository Interfaces in Spring Data JPA

CrudRepository

Purpose: Provides **basic CRUD operations**.

Key Methods:

Method	Description
<code>save(S entity)</code>	Save or update an entity
<code>findById(ID id)</code>	Find an entity by its primary key
<code>findAll()</code>	Retrieve all entities
<code>delete(T entity)</code>	Delete an entity
<code>count()</code>	Returns the total number of entities

Example:

```
import org.springframework.data.repository.CrudRepository;

public interface EmployeeRepository extends CrudRepository<Employee, Long> {
    // No need to implement; Spring generates implementation automatically
}
```

PagingAndSortingRepository

Purpose: Extends `CrudRepository` and provides **pagination and sorting** support.

Additional Methods:

Method	Description
--------	-------------

<code>findAll(Sort sort)</code>	Retrieves all entities sorted
<code>findAll(Pageable pageable)</code>	Retrieves a page of entities

Example:

```
import org.springframework.data.repository.PagingAndSortingRepository;
import org.springframework.data.domain.Sort;
import org.springframework.data.domain.PageRequest;

public interface EmployeeRepository extends
PagingAndSortingRepository<Employee, Long> {}

// Usage
Page<Employee> page = employeeRepository.findAll(PageRequest.of(0, 5)); //
first 5 records
```

JpaRepository

Purpose: Extends `PagingAndSortingRepository` and provides **JPA-specific operations** along with batch updates and flushing.

Additional Methods:

Method	Description
<code>saveAndFlush(entity)</code>	Save and immediately flush changes to the database
<code>deleteInBatch(Iterable<T> entities)</code>	Delete a batch of entities efficiently
<code>findAll()</code>	Returns a <code>List<T></code> instead of <code>Iterable<T></code>

Example:

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    // Custom query method
    List<Employee> findByDepartment(String department);
}
```

Summary Table

Repository	Extends	Purpose	Key Feature
CrudRepository	Repository	Basic CRUD operations	save, findById, delete
PagingAndSortingRepository	CrudRepository	CRUD + pagination and sorting	findAll(Pageable), findAll(Sort)
JpaRepository	PagingAndSortingRepository	CRUD + pagination + JPA-specific operations	saveAndFlush, deleteInBatch

Key Points

- Spring Data Repository **reduces boilerplate DAO code**.
- **JpaRepository > PagingAndSortingRepository > CrudRepository** in functionality.
- Allows **automatic query generation** from method names.

(b) Explain the following :

(i) Java Cryptography Architecture (JCA)

(ii) Java Secure Socket Extension (JSSE)

Ans :

(i) Java Cryptography Architecture (JCA)

Java Cryptography Architecture (JCA) is a **framework in Java** that provides **cryptographic services** such as encryption, decryption, digital signatures, and secure key management.

Key Components:

1. **Message Digests** – Provides hash functions (e.g., SHA-256) for data integrity.
2. **Digital Signatures** – Ensures authenticity and integrity of messages.
3. **Key Management** – Securely generates and manages cryptographic keys.
4. **Cipher** – Performs encryption and decryption using algorithms like AES, DES, RSA.
5. **Providers** – External libraries that implement cryptographic algorithms.

Advantages:

- Standardized API for cryptography.
- Extensible via **providers**.
- Ensures **data security** and integrity.

Example:

```
import java.security.*;
import javax.crypto.*;

public class JCAExample {
    public static void main(String[] args) throws Exception {
        // Generate SHA-256 message digest
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        byte[] hash = md.digest("Hello World".getBytes());
        System.out.println(java.util.Arrays.toString(hash));
    }
}
```


(ii) Java Secure Socket Extension (JSSE)

Java Secure Socket Extension (JSSE) is a **Java API** that provides **secure communication over network** using **SSL (Secure Sockets Layer)** and **TLS (Transport Layer Security)** protocols.

Key Features:

1. **SSL/TLS Support** – Secure data transfer over TCP/IP.
2. **Socket Factory** – Creates secure client and server sockets.
3. **Key and Trust Managers** – Manages certificates and authentication.
4. **Data Encryption** – Encrypts data to ensure privacy.
5. **Authentication** – Supports client and server authentication using certificates.

Advantages

- Provides **secure communication** between client and server.
- Supports **certificate-based authentication**.
- Integrated with **Java networking APIs**.

Example: HTTPS Connection

```
import javax.net.ssl.HttpsURLConnection;
import java.net.URL;

public class JSSEExample {
    public static void main(String[] args) throws Exception {
        URL url = new URL("https://www.example.com");
        HttpsURLConnection con = (HttpsURLConnection) url.openConnection();
        System.out.println("Response Code: " + con.getResponseCode());
    }
}
```

Summary Table

Feature	JCA	JSSE
Purpose	Provides cryptographic services (encryption, hashing, signing)	Provides secure network communication (SSL/TLS)

Main Components	Cipher, MessageDigest, Key Management, Providers	SSL/TLS Sockets, KeyManagers, TrustManagers
Use Case	Data encryption, digital signatures, secure key storage	Secure HTTPS connections, encrypted socket communication
Example	<code>MessageDigest.getInstance("SHA-256")</code>	<code>HttpsURLConnection</code> for HTTPS

Qus 4.

(a) What is Spring Security ? Discuss the terms “Authentication” and “Authorization” in context of Spring Security. Also mention methods for both.

Ans :

Spring Security is a **powerful and customizable security framework** for Java applications, especially **Spring-based applications**.

It provides **authentication, authorization, and protection against common security threats** like CSRF, session fixation, and more.

Key Features:

- **Authentication** – Verify user identity.
- **Authorization** – Control access to resources based on roles or permissions.
- Protection against **common attacks** (CSRF, XSS, Clickjacking).
- Integration with **Spring MVC, REST APIs, OAuth2, JWT**.

Authentication

Definition:

Authentication is the process of **verifying the identity of a user or system**.

In Spring Security:

- Users provide **credentials** (username/password, token, certificate).

- Spring Security checks credentials against a **user store** (database, LDAP, in-memory, etc.).

Methods of Authentication in Spring Security:

Method	Description
In-Memory Authentication	Users and passwords stored in application memory (good for testing).
Database Authentication (JDBC)	Users and roles stored in a relational database.
LDAP Authentication	Authenticate against LDAP directory servers.
OAuth2 / JWT	Token-based authentication for REST APIs.

Example (In-Memory Authentication):

```
@Override
protected void configure(AuthenticationManagerBuilder auth) throws
Exception {
    auth.inMemoryAuthentication()
        .withUser("admin").password("{noop}admin123").roles("ADMIN")
        .and()
        .withUser("user").password("{noop}user123").roles("USER");
}
```

Authorization

Definition:

Authorization is the process of **granting or denying access to specific resources** based on the user's **roles or permissions**.

In Spring Security:

- Authorization occurs **after authentication**.
- Access rules can be defined using **roles, expressions, or annotations**.

Methods of Authorization in Spring Security:

Method	Description
Role-based Access Control (RBAC)	Access resources based on user roles (e.g., <code>ROLE_ADMIN</code>).
Method-level Security	Use annotations like <code>@PreAuthorize</code> , <code>@Secured</code> on service methods.
URL-based Security	Configure access rules for URL patterns in <code>HttpSecurity</code> .

Example (URL-based Authorization):

```
@Override
protected void configure(HttpSecurity http) throws Exception {
    http.authorizeRequests()
        .antMatchers("/admin/**").hasRole("ADMIN")
        .antMatchers("/user/**").hasAnyRole("USER", "ADMIN")
        .antMatchers("/", "/home").permitAll()
        .and().formLogin();
}
```

Example (Method-level Authorization):

```
@PreAuthorize("hasRole('ADMIN')")

public void deleteUser(Long userId) {

    userRepository.deleteById(userId);

}
```

Summary Table: Authentication vs Authorization

Aspect	Authentication	Authorization
Definition	Verify identity of user	Grant access to resources
When	First step	After authentication
Methods in Spring Security	In-memory, JDBC, LDAP, OAuth2, JWT	Role-based, URL-based, Method-level security
Example	Logging in with username/password	Allowing only <code>ROLE_ADMIN</code> to access <code>/admin</code> URLs

Key Points

- **Spring Security = Authentication + Authorization + Attack Protection**
- **Authentication** answers: “Who are you?”
- **Authorization** answers: “What can you access?”

(b) Why login form customization is required in an application ? Explain the customized login form configuration with a simple code. Also, discuss `loginPage()` and `loginProcessing Url()` in Spring Boot Security Configuration.

Ans :

Why Login Form Customization is Required

The **default Spring Security login form** is functional but **very basic**. Customizing the login form is important because:

1. **Branding & UI consistency** – Aligns with your application’s design and layout.
2. **Additional fields** – Allows extra inputs like captcha, “remember me,” or multi-factor authentication.
3. **Better user experience** – Provides user-friendly messages, error handling, and styling.

4. **Custom URLs and redirects** – Enables redirecting to different pages after login or failure.

Customized Login Form Configuration in Spring Boot

Create a Custom Login Page

HTML File (**login.html**) in **src/main/resources/templates**

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Custom Login</title>
    <link rel="stylesheet" href="/css/style.css">
</head>
<body>
    <h2>Login to Your Account</h2>
    <form th:action="@{/perform_login}" method="post">
        <label>Username:</label>
        <input type="text" name="username"/><br><br>
        <label>Password:</label>
        <input type="password" name="password"/><br><br>
        <button type="submit">Login</button>
    </form>
    <p th:if="${param.error}">Invalid username or password.</p>
    <p th:if="${param.logout}">You have been logged out.</p>
</body>
</html>
```

Spring Security Configuration

Java Configuration Class

```
import org.springframework.context.annotation.Configuration;
import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebS
```

```

security;
import
org.springframework.security.config.annotation.web.configuration.WebSecurity
yConfigurerAdapter;

@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .authorizeRequests()
                .antMatchers("/login", "/css/**").permitAll() // Allow
public access to login page & CSS
                .anyRequest().authenticated()
            .and()
            .formLogin()
                .loginPage("/login") // Custom login page URL
                .loginProcessingUrl("/perform_login") // URL to submit
username & password
                .defaultSuccessUrl("/home", true) // Redirect after
successful login
                .failureUrl("/login?error=true") // Redirect on login
failure
            .permitAll()
            .and()
            .logout()
                .logoutUrl("/perform_logout")
                .logoutSuccessUrl("/login?logout=true")
                .permitAll();
    }
}

```

Explanation of Key Methods

Method

Purpose

```
loginPage("/login")
```

Defines the **custom login page URL**. Users accessing protected resources will be redirected here.

```
loginProcessingUrl("/perform_login")
```

URL where **Spring Security expects form submission** with **username** and **password**. Spring automatically handles authentication here.

Notes:

- The form **action** attribute must match `loginProcessingUrl()`.
- `defaultSuccessUrl()` defines where to go after successful login.
- `failureUrl()` defines where to go after failed login.

Flow of Customized Login Form

1. User accesses a protected resource.
2. Spring Security redirects user to **custom login page** (`loginPage()`).
3. User submits username/password to `loginProcessingUrl()`.
4. Spring Security authenticates credentials.
 - On success → redirect to `defaultSuccessUrl()`.
 - On failure → redirect to `failureUrl()`.

This configuration allows **full control over the login page UI, error messages, and authentication behavior**, while still using Spring Security's backend mechanisms.

Qus 5. Write short notes on the following :

(i) Servlet Architecture

(ii) JSP life cycle

(iii) Annotations and its types

(iv) Aspect-oriented programming

Ans :

(i) Servlet Architecture

Servlet is a **Java program** that runs on a server to handle **client requests** (usually HTTP) and generate dynamic content.

Architecture Components:

1. **Client** – Sends requests (browser or app).
2. **Web Server** – Receives requests and forwards to the **Servlet Container**.
3. **Servlet Container (Web Container)** – Manages lifecycle of servlets: loading, instantiation, initialization, request handling, and destruction.
4. **Servlet** – Processes requests, interacts with databases/business logic, generates response.
5. **Response** – Sent back to the client.

Flow:

Client → Web Server → Servlet Container → Servlet → Response → Client

Key Points:

- Implements **Servlet** interface or extends **HttpServlet**.
- Supports **multithreading** for handling multiple requests.
- Lifecycle managed by the container (**init()**, **service()**, **destroy()**).

(ii) JSP Life Cycle

JSP (JavaServer Pages) is converted into a **servlet** before execution. Its life cycle has **three main phases**:

1. Translation & Compilation

- JSP file → Servlet class.
- Only happens **once**, unless the JSP is modified.

2. Initialization

- **jspInit()** method is called to initialize resources.

3. Request Handling

- `jspService(HttpServletRequest req, HttpServletResponse res)` handles each client request.

4. Destruction

- `jspDestroy()` is called when the JSP is removed or server shuts down.

Key Points:

- Lifecycle is managed by the **JSP container**.
- JSP scripting elements (`<% %>`, `<%= %>`) are converted to servlet code.

(iii) Annotations and Its Types

Annotation is **metadata in Java** used to provide additional information about code elements (classes, methods, variables).

Common Types:

1. Marker Annotations

- No elements; just presence indicates behavior.
- Example: `@Override`, `@Deprecated`

2. Single-Value Annotations

- Contain a single element.
- Example: `@SuppressWarnings("unchecked")`

3. Multi-Value Annotations

- Contain multiple elements with name-value pairs.
- Example:

`@Entity`

`@Table(name="employee", schema="hr")`

Key Points:

- Used for **configuration, compile-time checking, or runtime processing**.
- Supported in **Spring, JPA, Hibernate**, etc.

(iv) Aspect-Oriented Programming (AOP)

AOP is a programming paradigm that **separates cross-cutting concerns** (like logging, security, transactions) from business logic.

Key Concepts:

1. **Aspect** – Module containing cross-cutting concerns.
2. **Join Point** – Point in the execution of a program (method call, exception, etc.).
3. **Advice** – Action taken at a join point (**@Before**, **@After**, **@Around**).
4. **Pointcut** – Expression defining **where advice applies**.
5. **Weaving** – Process of applying aspects to target objects (compile-time, load-time, or runtime).

Example (Spring AOP):

```
@Aspect
@Component
public class LoggingAspect {
    @Before("execution(* com.example.service.*(..))")
    public void logBefore() {
        System.out.println("Method execution started...");
    }
}
```

Advantages:

- Reduces **code duplication**.
- Improves **modularity** and **maintainability**.

December, 2023

Qus 1.

(a) What is J2EE ? Briefly discuss the components of J2EE.

Ans :

J2EE (Java 2 Platform, Enterprise Edition) is a Java platform for developing and deploying enterprise-level, multi-tiered, distributed applications.

Key Features:

- Supports **web-based applications, enterprise services, and distributed computing.**
- Provides **standardized APIs** for enterprise development (Servlets, JSP, EJB, JDBC, JMS, etc.).
- Enables **scalability, security, and portability** across platforms.
- Follows a **multi-tier architecture**: Client → Web → Business → Enterprise Information System (EIS).

Components of J2EE

Web Components

- **Servlets**: Handle client requests and generate dynamic responses.
- **JSP (JavaServer Pages)**: Simplifies dynamic web page creation using HTML with embedded Java.
- **Purpose**: Presentation layer for web applications.

Enterprise Beans (EJB Components)

- **Session Beans**: Implement business logic.
 - **Stateless** – No client-specific data stored between requests.
 - **Stateful** – Maintain conversational state for a client.
- **Message-Driven Beans**: Handle asynchronous messages via JMS.
- **Purpose**: Business layer for enterprise applications.

Client Components

- **Types:** Web clients, applets, Swing/AWT desktop applications, or thin clients.
- **Purpose:** Access enterprise services via web or remote calls.

Services

- **JDBC (Java Database Connectivity):** Connects to relational databases.
- **JMS (Java Messaging Service):** Messaging and asynchronous communication.
- **JNDI (Java Naming and Directory Interface):** Lookup and naming services.
- **Transactions (JTA):** Supports distributed transactions.
- **Security:** Authentication and authorization services.

Deployment Components

- **EAR (Enterprise Archive) Files:** Package EJBs, web modules, and libraries for deployment.
- **Deployment Descriptors:** XML files defining component configuration and security.

Summary Table

Layer	Component	Purpose
Client Layer	Web client, applet, Swing	Access application
Web Layer	Servlets, JSP	Presentation logic
Business Layer	Session Beans, Message-Driven Beans	Business logic
EIS Layer	JDBC, JMS, JNDI	Data and messaging services
Deployment	EAR, deployment descriptors	Packaging and configuration

Key Points

- J2EE provides a **standardized environment** for building **scalable and secure enterprise applications**.
- It separates **presentation, business, and data layers** for better modularity.

- Widely used for **large-scale web and enterprise applications**.

(b) What is a servlet ? Briefly discuss the lifecycle of servlet.

Ans :

A **Servlet** is a **Java program** that runs on a **server** to handle **client requests** and generate **dynamic responses**.

It is mainly used for **web applications** and runs inside a **Servlet Container** (like Tomcat).

Key Features:

- Handles **HTTP requests** (**GET**, **POST**).
- Can generate **dynamic content** (HTML, JSON, XML).
- Supports **multithreading** to serve multiple clients simultaneously.
- Part of **Java EE/Jakarta EE** technology.

Servlet Types:

- **GenericServlet** – Protocol-independent, abstract class.
- **HttpServlet** – Subclass of GenericServlet for HTTP-specific services.

Lifecycle of a Servlet

The **Servlet Lifecycle** is managed by the **Servlet Container** and has **three main phases**:

Initialization (init()** method)**

- Called **once** when the servlet is first loaded.
- Used to **initialize resources** like database connections.
- Method signature:

```
public void init(ServletConfig config) throws ServletException
```

Request Handling (service()** method)**

- Called **each time a client request** is made.
- Determines the request type (**GET**, **POST**, etc.) and calls the corresponding method:

- `doGet()` → Handles HTTP GET requests
- `doPost()` → Handles HTTP POST requests
- `doPut()`, `doDelete()`, etc. for other HTTP methods
- Example:

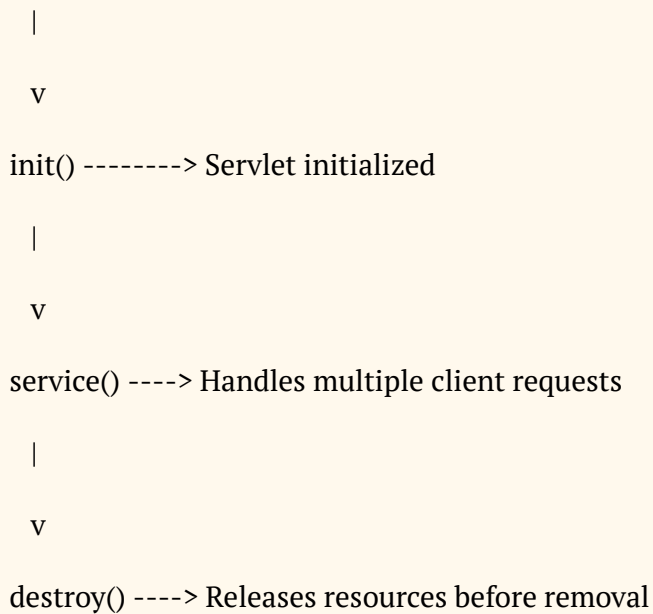
```
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    response.getWriter().println("Hello, Servlet!");
}
```

Destruction (`destroy()` method)

- Called **once** before the servlet is removed from memory.
- Used to **release resources** (close DB connections, file streams).
- After this, servlet is **garbage collected**.

Servlet Lifecycle Flow Diagram

Servlet Loaded



Key Points

- **Lifecycle is controlled by the Servlet Container.**
- **init()** → Initialization (once)
- **service()** → Request handling (multiple times)
- **destroy()** → Cleanup (once before removal)
- Servlets are **multithreaded**; a single instance can handle multiple requests concurrently.

(c) Differentiate between session and cookie. Give suitable example for each.

Ans :

Session vs Cookie

Feature	Session	Cookie
Definition	Server-side storage of user data during a user's interaction with a web application.	Client-side small text data stored in the browser to maintain user state.
Storage Location	Stored on the server .	Stored on the client (browser) .
Size Limit	Can store larger amounts of data (depending on server memory).	Limited to about 4KB per cookie .
Lifetime	Exists until user closes browser or session expires on server.	Can be persistent (stored with expiry date) or session-only (deleted when browser closes).
Security	More secure, data is on the server.	Less secure, can be accessed/modified by client.

Access	Accessed via server-side APIs (e.g., <code>HttpSession</code>).	Accessed via client-side (JavaScript) or server-side APIs (e.g., <code>request.getCookies()</code>).
Use Case	Storing user login information, shopping cart, server-side preferences.	Storing user preferences, theme, remember-me functionality.

Examples

Session Example (Java Servlet)

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.IOException;

public class SessionExample extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // Create or retrieve existing session
        HttpSession session = request.getSession();
        // Set attribute
        session.setAttribute("username", "Ritesh");
        response.getWriter().println("Session created with username: " + session.getAttribute("username"));
    }
}
```

- `request.getSession()` → retrieves existing session or creates a new one.
- Data (`username`) is stored **on the server**.

Cookie Example (Java Servlet)

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.IOException;
```

```

public class CookieExample extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse
response)
        throws ServletException, IOException {
        // Create a cookie
        Cookie userCookie = new Cookie("username", "Ritesh");
        userCookie.setMaxAge(24 * 60 * 60); // 1 day
        response.addCookie(userCookie);
        response.getWriter().println("Cookie created: " +
userCookie.getValue());
    }
}

```

- Cookie data is stored **on the client browser**.
- `setMaxAge()` defines how long the cookie persists.

Key Points

- **Sessions** are **server-side** → safer for sensitive data.
- **Cookies** are **client-side** → lightweight, but less secure.
- Often used **together**: session uses a **cookie (JSESSIONID)** to track user sessions.

(d) Write JSP program for Fibonacci series. Support your code with suitable comments.

Ans :

Here's a **complete JSP program** to generate a **Fibonacci series**, with comments explaining each part:

```

<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
    <title>Fibonacci Series JSP Example</title>
</head>
<body>

```

```

<h2>Fibonacci Series</h2>

<!-- JSP scriptlet to generate Fibonacci series -->
<%
    // Number of terms to generate
    int n = 10;

    // Variables to store previous two numbers
    int first = 0;
    int second = 1;

    // Print the first two numbers
    out.print("Fibonacci Series (first " + n + " terms): " + first + ",
" + second);

    int next;

    // Loop to generate remaining terms
    for(int i = 3; i <= n; i++) {
        next = first + second; // Calculate next term
        out.print(", " + next); // Print next term

        // Update previous two numbers
        first = second;
        second = next;
    }
%>
</body>
</html>

```

Explanation

1. **<%@ page %>** – JSP directive defining language and content type.
2. **Scriptlet <% ... %>** – Contains Java code executed on the server.
3. **Variables first and second** – Store the first two Fibonacci numbers.
4. **Loop** – Calculates the next term using **next = first + second** and prints it.
5. **out.print()** – Writes output to the HTML response.

Output:

Fibonacci Series (first 10 terms): 0, 1, 1, 2, 3, 5, 8, 13, 21, 34

(e) What are Interceptors in struts2 ? Briefly discuss the role of Interceptors in struts2.

Ans :

What are Interceptors in Struts2?

Interceptors are **objects in Struts2** that **intercept requests and responses** during the processing of an action.

They allow **pre-processing and post-processing logic** to be executed **before and after an action method** is invoked.

Key Points:

- Core feature of Struts2 framework.
- Used for **common functionality** like logging, authentication, validation, file upload, and exception handling.
- Every action in Struts2 can be intercepted by one or more interceptors.
- Interceptors are **reusable and configurable** through `struts.xml` or annotations.

Role of Interceptors

Pre-processing

- Executed **before the action method**.
- Example tasks:
 - Authentication check
 - Input validation
 - Logging request parameters

Post-processing

- Executed **after the action method** but **before sending the result to the view**.
- Example tasks:
 - Formatting the response
 - Logging execution time
 - Modifying result data

Advantages

- Promotes **code reusability** – same interceptor can be applied to multiple actions.
- **Separation of concerns** – business logic is separate from common tasks.
- Can be **stacked** – multiple interceptors can be applied in a sequence.

Example: Using an Interceptor

Custom Interceptor Class

```
import com.opensymphony.xwork2.ActionInvocation;
import com.opensymphony.xwork2.interceptor.Interceptor;

public class LoggingInterceptor implements Interceptor {

    @Override
    public void destroy() { }

    @Override
    public void init() { }

    @Override
    public String intercept(ActionInvocation invocation) throws Exception {
        System.out.println("Before action: Logging request");
        String result = invocation.invoke(); // Proceed to action
        System.out.println("After action: Logging response");
        return result;
    }
}
```

Configure in `struts.xml`

```
<struts>
  <package name="default" namespace="/" extends="struts-default">
    <interceptors>
      <interceptor name="logging"
class="com.example.LoggingInterceptor"/>
      <interceptor-stack name="loggingStack">
        <interceptor-ref name="logging"/>
        <interceptor-ref name="defaultStack"/>
      </interceptor-stack>
    </interceptors>
  </package>
</struts>
```

```

        </interceptor-stack>
    </interceptors>

    <action name="hello" class="com.example.HelloAction">
        <interceptor-ref name="loggingStack"/>
        <result>/hello.jsp</result>
    </action>
</package>
</struts>

```

Explanation:

- **intercept()** – Executes logic **before and after the action**.
- **invocation.invoke()** – Calls the next interceptor or action.
- **Interceptor Stack** – Defines the order of interceptors applied to an action.

Key Points

- Interceptors are **central to Struts2 architecture**.
- They **wrap action execution**, allowing common functionalities to be handled **consistently**.
- Can be **applied globally** or to **specific actions**.

(f) Differentiate between spring boot and spring MVC. 5 (g) Briefly discuss the term “Inversion of Control (IoC)”. Also, draw block diagram for IoC flow.

Ans :

Difference Between Spring Boot and Spring MVC

Feature	Spring Boot	Spring MVC
Definition	Framework to create standalone, production-ready Spring	Part of Spring Framework ; used for building web applications based on

	applications with minimal configuration.	the MVC (Model-View-Controller) pattern.
Configuration	Auto-configuration; minimal XML/annotation setup required.	Requires explicit configuration of DispatcherServlet, view resolvers, beans, etc.
Server	Can run with embedded servers like Tomcat/Jetty.	Needs an external servlet container (Tomcat/GlassFish) or WAR deployment.
Purpose	Simplifies Spring application setup, development, and deployment.	Handles web layer (HTTP requests and responses) in Spring applications.
Dependencies	Uses Spring Boot starters for quick setup.	Requires manual addition of Spring MVC, JSP/Thymeleaf, and other dependencies.
Use Case	Microservices, REST APIs, stand-alone applications.	Traditional web applications with MVC design.

Inversion of Control (IoC)

Definition:

Inversion of Control (IoC) is a **design principle** in which the control of object creation and dependency management is **shifted from the program to the framework**.

- In traditional programming: objects **create their dependencies**.
- In IoC: **the container/framework injects the dependencies** into objects.

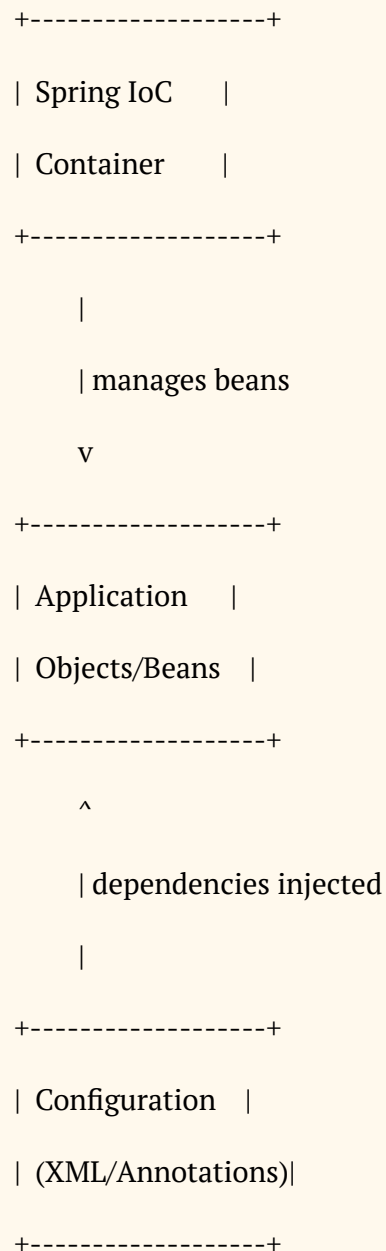
Purpose:

- Promotes **loose coupling**.
- Makes applications **more modular and testable**.

IoC in Spring

- Spring IoC container manages **beans** and their **dependencies**.
- Dependencies can be injected via:
 1. **Constructor Injection**
 2. **Setter Injection**

Block Diagram of IoC Flow



Flow Explanation:

1. **Configuration** defines beans and dependencies.
2. **Spring IoC Container** reads configuration and creates beans.
3. **Beans are injected** with dependencies automatically.
4. Application uses the beans without worrying about creation or lifecycle.

(h) What is Custom CSS ? How custom CSS can be included in to JSP ? Discuss.**Ans :**

CSS (Cascading Style Sheets) is used to **style HTML elements** – defining layout, colors, fonts, spacing, and overall appearance of a web page.

Custom CSS refers to **user-defined CSS rules** that are written specifically for an application, rather than using default browser styles or third-party frameworks (like Bootstrap).

Purpose of Custom CSS:

- Provides **unique branding** and visual identity.
- Allows **fine-grained control** over layout and styling.
- Enhances **user experience** and accessibility.

Including Custom CSS in JSP

There are **three main ways** to include custom CSS in a JSP page:

External CSS File

- **Best practice** – keeps CSS separate from HTML/JSP code.
- Place the CSS file in a **web-accessible folder**, e.g., `webapp/css/style.css`.

Example:

```
<link rel="stylesheet" type="text/css" href="css/style.css">
```

Usage in JSP:

```
<!DOCTYPE html>
<html>
<head>
    <title>Custom CSS Example</title>
    <link rel="stylesheet" type="text/css" href="css/style.css"> <!--
External CSS -->
</head>
<body>
    <h1 class="heading">Welcome to JSP with Custom CSS</h1>
</body>
</html>
```

Example CSS (**style.css**):

```
.heading {
    color: blue;
    font-family: Arial, sans-serif;
    text-align: center;
}
```

Internal CSS

- CSS is written within the **<style>** tag in the JSP page **<head>** section.

Example:

```
<head>
    <title>Custom CSS Example</title>
    <style>
        .heading {
            color: green;
            font-size: 24px;
            text-align: center;
        }
    </style>
</head>
```

```
</style>
</head>
```

Inline CSS

- CSS is **applied directly to HTML elements** using the **style** attribute.
- Not recommended for large applications due to poor maintainability.

Example:

```
<h1 style="color:red; font-size:28px; text-align:center;">Welcome to JSP</h1>
```

Best Practices

- Prefer **external CSS** for maintainability and reusability.
- Use **class and ID selectors** instead of inline styles.
- Organize CSS files by functionality (layout, typography, theme).
- Combine with frameworks (like Bootstrap) but override styles with custom CSS when needed.

Flow of Including CSS in JSP

Custom CSS file (style.css)

|

v

JSP page <link rel="stylesheet" href="css/style.css">

|

v

Browser renders HTML elements styled as per custom CSS

Qus 2.

(a) What is the need of application properties in spring boot ? Explain how application properties of spring boot can be set though properties files, command-line arguments and environment variables.

Ans :

What is the Need of Application Properties in Spring Boot?

Application properties in Spring Boot are used to **externalize configuration** from the code, so that the same application can run in different environments (development, testing, production) with different configurations.

Key Purposes:

1. **External Configuration:** Allows changing settings (like database URL, server port) without modifying code.
2. **Environment Specific:** Supports different profiles (`dev`, `test`, `prod`).
3. **Centralized Management:** All configuration in a single place (`application.properties` or `application.yml`).
4. **Flexibility:** Can override properties via **command-line arguments** or **environment variables**.

Common Uses:

- Server configuration (`server.port=8081`)
- Database configuration (`spring.datasource.url`, `spring.datasource.username`)
- Logging (`logging.level.org.springframework=DEBUG`)

How Application Properties Can Be Set

Using `application.properties` File

- Default location: `src/main/resources/application.properties`
- Spring Boot automatically loads this file at startup.

Example (application.properties):

```
server.port=8081

spring.datasource.url=jdbc:mysql://localhost:3306/testdb

spring.datasource.username=root

spring.datasource.password=root123

logging.level.org.springframework=DEBUG
```

Accessing Properties in Code:

```
@Value("${server.port}")

private int serverPort;
```

Using Command-Line Arguments

- Properties can be **overridden at runtime** using `--property=value` syntax.
- Useful for **temporary overrides** without changing files.

Example Command:

```
java -jar myapp.jar --server.port=9090 --spring.datasource.username=admin
```

- Spring Boot gives **highest priority** to command-line arguments over properties files.

Using Environment Variables

- Properties can also be set via **OS environment variables**.
- Spring Boot automatically maps **uppercase environment variables with underscores** to properties.

Example:

```
export SERVER_PORT=7070

export SPRING_DATASOURCE_PASSWORD=admin123
```

- Spring Boot maps `SERVER_PORT` → `server.port`
- Environment variables override **default properties in `application.properties`**.

Property Resolution Priority in Spring Boot

Priority	Source
1	Command-line arguments
2	Environment variables
3	<code>application.properties</code> / <code>application.yml</code> in config folder
4	Default values in code (<code>@Value(defaultValue="...")</code>)

Summary

- **Application properties** allow external configuration and flexibility.
- Can be set via:
 1. **Properties file** (`application.properties` or `application.yml`)
 2. **Command-line arguments** (`--property=value`)
 3. **Environment variables** (`SERVER_PORT=8080`)
- Properties **override in order**: command-line → environment → file → defaults.

(b) What is Hibernate ? Briefly discuss the various states of Hibernate Entity Life Cycle. Also, draw block diagram for the same.

Ans :

What is Hibernate?

Hibernate is a **Java-based ORM (Object-Relational Mapping) framework** that simplifies **database interactions** by mapping Java objects to database tables.

Key Features:

- Provides **automatic mapping** between Java objects and relational tables.
- Supports **HQL (Hibernate Query Language)** for database operations.
- Handles **transactions, caching, and lazy loading** efficiently.
- Reduces boilerplate JDBC code (no manual SQL for CRUD operations).

Use Case: Replace manual JDBC code with **object-oriented data persistence**.

Hibernate Entity Life Cycle

A **Hibernate entity** goes through various **states** during its life cycle in relation to a **Hibernate Session**.

Transient State

- The object is **created using new operator** but **not associated with Hibernate session**.
- No **identifier value** assigned.
- Will not be saved to the database unless **save()** or **persist()** is called.

Example:

```
Employee emp = new Employee();  
  
emp.setName("Ritesh");
```

Persistent State

- The object is **associated with a Hibernate session**.
- Hibernate **tracks changes automatically**.
- Any change is **synchronized with the database** when the session is flushed or committed.

Methods that make object persistent: `session.save()`, `session.persist()`, `session.update()`

Example:

```
Session session = sessionFactory.openSession();
```

```
Transaction tx = session.beginTransaction();

session.save(emp); // Now emp is in persistent state

tx.commit();
```

Detached State

- The object is **no longer associated with a Hibernate session**.
- Changes to the object are **not automatically synchronized** with the database.
- Can be reattached using `update()` or `merge()`.

Example:

```
session.close(); // emp becomes detached

emp.setName("Ritesh Tiwari");
```

Removed State

- The object is **marked for deletion** in the session.
- Once the transaction is committed, the corresponding **row in the database is deleted**.

Example:

```
session.delete(emp); // emp moves to removed state
```

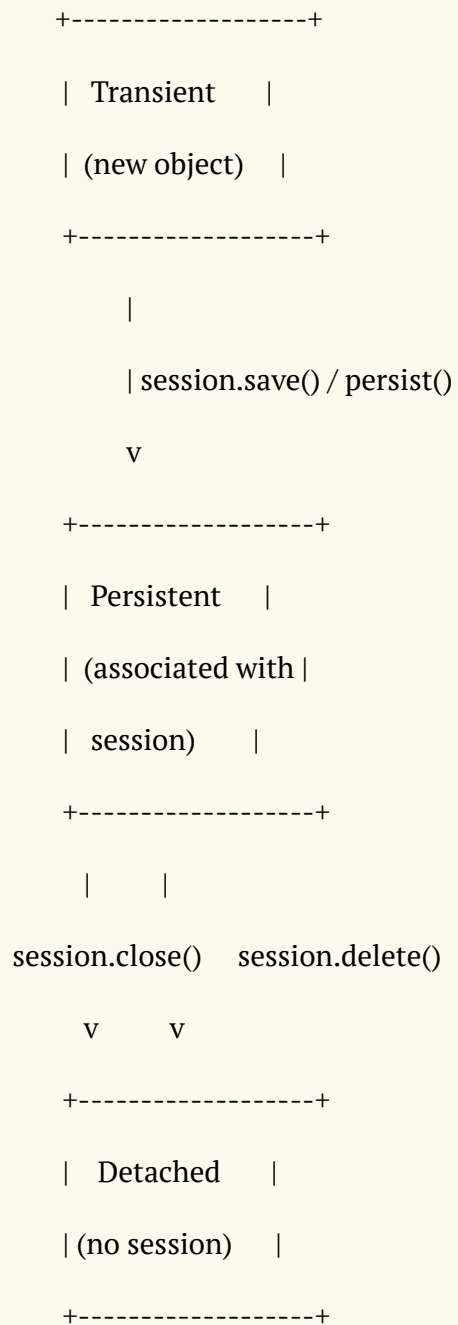
Summary Table of Hibernate States

State	Description	Example Methods
Transient	Not associated with session; not saved to DB	<code>new Employee()</code>
Persistent	Associated with session; tracked by Hibernate	<code>save()</code> , <code>persist()</code> , <code>update()</code>
Detached	No longer associated with session; not tracked	After <code>session.close()</code>

Removed Marked for deletion from DB

`delete()`

Block Diagram of Hibernate Entity Life Cycle



```

      |
session.update() / merge()

      v
+-----+
| Removed State |
+-----+

```

Key Points:

- Hibernate automatically manages **object states** and **synchronization with the database**.
- Understanding entity states is **crucial for effective session and transaction management**.

Qus 3.

(a) What is the relationship between JPA, spring data JPA and Hibernate ? Explain Eager and Lazy loading in Hibernate. How does the Eager and Lazy loading improves the performance of Hibernate applications ?

Ans :

Relationship between JPA, Spring Data JPA, and Hibernate

Layer	Description	Role
JPA (Java Persistence API)	A Java specification for ORM (Object-Relational Mapping).	Defines standard APIs for mapping Java objects to database tables and performing CRUD operations.

Hibernate	A JPA implementation (also a full ORM framework).	Provides actual implementation of JPA interfaces and additional features like caching, lazy loading, and HQL.
Spring Data JPA	A Spring abstraction layer on top of JPA.	Simplifies data access by providing Repository interfaces (like <code>CrudRepository</code> , <code>JpaRepository</code>) to reduce boilerplate code.

Relationship Flow:

Application -> Spring Data JPA -> JPA -> Hibernate (or any JPA provider)

- JPA: Standard API
- Hibernate: Implementation
- Spring Data JPA: Convenience layer on top of JPA

Eager and Lazy Loading in Hibernate

(i) Eager Loading

- **Definition:** Loads the **related entities immediately** when the parent entity is loaded.
- **Annotation:** `@OneToMany(fetch = FetchType.EAGER)` or `@ManyToOne(fetch = FetchType.EAGER)`
- **Use Case:** When you always need associated entities along with the parent entity.

Example:

@Entity

```
public class Department {
```

```
    @OneToMany(mappedBy="department", fetch=FetchType.EAGER)
```

```
    private Set<Employee> employees;
```

```
}
```

- Fetching a `Department` automatically fetches all `Employee` objects.

(ii) Lazy Loading

- **Definition:** Loads the **related entities on-demand**, only when accessed.
- **Annotation:** `@OneToMany(fetch = FetchType.LAZY)` or `@ManyToOne(fetch = FetchType.LAZY)`
- **Use Case:** When associated entities may **not be required immediately**, to save memory and processing.

Example:

`@Entity`

```
public class Department {
```

```
    @OneToMany(mappedBy="department", fetch=FetchType.LAZY)
```

```
    private Set<Employee> employees;
```

```
}
```

- Fetching a `Department` **does not load employees** until `getEmployees()` is called.

How Eager and Lazy Loading Improve Performance

Loading Type	Performance Benefit
Eager	Avoids multiple additional queries if the data is always required; reduces lazy initialization issues .
Lazy	Reduces initial memory consumption and database load ; prevents unnecessary joins; improves performance for large collections.

Summary:

- **Eager Loading:** Good for **small, frequently accessed relationships**.

- **Lazy Loading:** Good for **large collections or optional relationships**, avoids fetching unnecessary data.

Key Points

1. **JPA:** Standard ORM API
2. **Hibernate:** JPA implementation with additional features
3. **Spring Data JPA:** Simplifies JPA usage with repository abstractions
4. **Eager vs Lazy:** Controls when related entities are loaded; careful choice **optimizes performance**.

(b) What is Message Digest ? How does message digest differ from message authentication code ? Give characteristics of both.

Ans :

What is a Message Digest?

A **Message Digest** is a **fixed-size, unique representation of a message** produced by a **hash function**.

It is used to **verify data integrity**, ensuring that the message has **not been altered** during transmission.

Key Points:

- Input: Arbitrary-length message
- Output: Fixed-length hash value (digest)
- **One-way function:** Cannot reverse hash to obtain original message
- Common algorithms: **MD5, SHA-1, SHA-256**

Example (SHA-256):

Message: "Hello World"

Message Digest: a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e

Message Authentication Code (MAC)

A **Message Authentication Code (MAC)** is a **short piece of information** used to **verify both the integrity and authenticity of a message**.

Unlike a simple message digest, **MAC requires a secret key** shared between sender and receiver.

Key Points:

- Ensures **integrity + authenticity**
- Input: Message + secret key
- Output: Fixed-length code
- Common algorithms: **HMAC-MD5, HMAC-SHA1**

Example:

Message: "Hello World"

Secret Key: "MySecretKey"

MAC: b0344c61d8db38535ca8afceaf0bf12b881dc200

Differences Between Message Digest and MAC

Feature	Message Digest	Message Authentication Code (MAC)
Purpose	Ensure integrity of data	Ensure integrity + authenticity
Key	No key required	Requires secret key
Security	Detects accidental or malicious changes	Detects tampering and confirms sender authenticity
Use Case	File verification, checksums	Secure communication, digital signatures
Algorithm	MD5, SHA-1, SHA-256	HMAC-MD5, HMAC-SHA1

Characteristics**Message Digest:**

1. **One-way** – cannot reconstruct original message

2. **Fixed-length output** regardless of input size
3. **Sensitive to changes** – small change in input drastically changes digest
4. **No secret key** – cannot confirm sender identity

Message Authentication Code (MAC):

1. **Key-dependent** – requires shared secret key
2. **Detects tampering** – ensures data integrity
3. **Authenticates sender** – only sender with key can generate correct MAC
4. **Fixed-length output** – similar to hash, but secure with key

Summary

- **Message Digest:** Integrity only, one-way hash
- **MAC:** Integrity + authenticity, key-based hash

Qus 4.

(a) Briefly discuss the following security vulnerabilities of any web app :

- (i) **Cross Site Scripting (XSS)**
- (ii) **Cross-Site Request Forgery (CSRF)**
- (iii) **SQL Injection (SQLi)**
- (iv) **Denial-of-Service (DoS)**
- (v) **Insecure Direct Object References (IDOR)**

Ans :

1 Cross-Site Scripting (XSS)

Definition:

XSS occurs when an attacker injects **malicious scripts** into a trusted web application, which are then executed in the **victim's browser**.

Types:

- **Stored XSS:** Script is permanently stored on the server (e.g., in database).
- **Reflected XSS:** Script is reflected via URL or input field.
- **DOM-based XSS:** Script executed by manipulating client-side DOM.

Impact:

- Steal user cookies or session tokens
- Deface websites
- Perform actions on behalf of the user

Prevention:

- Input validation and encoding
- Use frameworks that auto-escape content (e.g., JSP `<c:out>`)

② Cross-Site Request Forgery (CSRF)

Definition:

CSRF tricks a logged-in user into **performing unwanted actions** on a web application without their knowledge.

Example:

- User logged in to bank site
- Attacker tricks user to click a malicious link that **transfers money**

Impact:

- Unauthorized transactions
- Changes in user settings

Prevention:

- Use **CSRF tokens** in forms
- Check **referrer/origin headers**
- Use **same-site cookies**

③ SQL Injection (SQLi)

Definition:

SQLi occurs when an attacker **injects malicious SQL queries** into input fields to manipulate the database.

Example:

```
' OR 1=1; --
```

Impact:

- Bypass authentication
- Retrieve, modify, or delete sensitive data
- Execute administrative database commands

Prevention:

- Use **prepared statements** / **parameterized queries**
- Input validation
- Limit database privileges

Denial-of-Service (DoS)**Definition:**

DoS attacks aim to **make a web application unavailable** by overwhelming the server with requests.

Types:

- **Volume-based attacks:** Flood server with traffic
- **Protocol attacks:** Exploit protocol weaknesses (e.g., SYN flood)
- **Application-layer attacks:** Overload application resources

Impact:

- Service downtime
- Loss of revenue and reputation

Prevention:

- Rate limiting and throttling
- Use firewalls and load balancers
- Monitor traffic anomalies

Insecure Direct Object References (IDOR)**Definition:**

IDOR occurs when an application exposes **internal object identifiers** (like user IDs, file names) without proper access checks.

Example:

- URL: `/profile?userId=123`
- Attacker changes `userId` to access another user's data

Impact:

- Unauthorized access to sensitive data
- Data leakage

Prevention:

- Implement **authorization checks** for every request
- Avoid exposing raw IDs; use **mapping or random identifiers**

✓ Summary Table

Vulnerability	Impact	Prevention
XSS	Steal session, deface site	Input validation, encoding
CSRF	Unauthorized actions	CSRF tokens, same-site cookies
SQLi	Database manipulation	Prepared statements, input validation

DoS	Service downtime	Rate limiting, monitoring, firewalls
IDOR	Unauthorized data access	Proper access checks, avoid raw IDs

(b) Describe role-based login. Write the sample code to configure the role-based login.

Ans :

Role-Based Login (RBL) is a security mechanism in web applications that **restricts access to resources based on the user's role**.

Key Points:

- Users are assigned **roles** (e.g., ADMIN, USER, MANAGER).
- Each role is granted **specific access permissions**.
- Helps enforce **authorization rules** on the server side.
- Commonly used in **Spring Security** and other frameworks.

Example Use Case:

- **ADMIN** can access `/admin/**` URLs.
- **USER** can access `/user/**` URLs.
- Unauthorized users get **403 Forbidden** or redirected to a login page.

Sample Spring Boot Security Configuration for Role-Based Login

```
import org.springframework.context.annotation.Configuration;
import
org.springframework.security.config.annotation.authentication.builders.Auth
enticationManagerBuilder;
import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebS
ecurity;
import
org.springframework.security.config.annotation.web.configuration.WebSecurit
yConfigurerAdapter;
```

```

@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(AuthenticationManagerBuilder auth) throws
Exception {
        // In-memory authentication with roles
        auth.inMemoryAuthentication()
            .withUser("admin").password("{noop}admin123").roles("ADMIN")
            .and()
            .withUser("user").password("{noop}user123").roles("USER");
    }

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .authorizeRequests()
                // Role-based access control
                .antMatchers("/admin/**").hasRole("ADMIN")
                .antMatchers("/user/**").hasRole("USER")
                .anyRequest().authenticated()
            .and()
            .formLogin()
                .loginPage("/login") // custom login page
                .loginProcessingUrl("/perform_login") // URL to submit
username/password
                .defaultSuccessUrl("/home", true)
                .permitAll()
            .and()
            .logout()
                .logoutUrl("/logout")
                .logoutSuccessUrl("/login?logout")
                .permitAll()
            .and()
            .exceptionHandling()
                .accessDeniedPage("/403"); // page for unauthorized access
    }
}

```

Explanation

1. AuthenticationManagerBuilder

- Configures **users and their roles**.
- `{noop}` means **plain text password** (for demo purposes).

2. HttpSecurity

- `.antMatchers()` – defines **URL patterns and role access**.
- `.formLogin()` – customizes login page and processing URL.
- `.logout()` – defines logout behavior.
- `.exceptionHandling()` – handles unauthorized access.

3. Role Enforcement

- Only users with the required role can access certain endpoints.

Key Points

- **Role-Based Login = Authentication + Authorization**
- Roles can be **stored in database** (instead of in-memory).
- Spring Security automatically enforces **access rules** at URL and method levels.

(c) What is role based access control ? Write the simple configuration code to restrict the URL access based on role.

Ans :

Role-Based Access Control (RBAC) is a **security mechanism** that **restricts user access to resources based on assigned roles**.

Key Points:

- Users are assigned **roles** such as `ADMIN`, `USER`, `MANAGER`.
- Each role has **permissions** to access certain URLs or methods.
- Helps **enforce authorization rules** and protect sensitive parts of the application.
- Commonly used in **Spring Security** to control access at **URL or method level**.

Example Use Case:

- `/admin/**` → Accessible only by **ADMIN** role.
- `/user/**` → Accessible by **USER** role.
- Other URLs → Accessible to any authenticated user.

Sample Spring Boot Security Configuration for RBAC

```
import org.springframework.context.annotation.Configuration;
import
org.springframework.security.config.annotation.authentication.builders.Auth
enticationManagerBuilder;
import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebS
ecurity;
import
org.springframework.security.config.annotation.web.configuration.WebSecurit
yConfigurerAdapter;

@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(AuthenticationManagerBuilder auth) throws
Exception {
        // In-memory authentication with roles
        auth.inMemoryAuthentication()
            .withUser("admin").password("{noop}admin123").roles("ADMIN")
            .and()
            .withUser("user").password("{noop}user123").roles("USER");
    }

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .authorizeRequests()
                // URL access restriction based on roles
                .antMatchers("/admin/**").hasRole("ADMIN")
                .antMatchers("/user/**").hasRole("USER")
                .anyRequest().authenticated() // all other requests require
```

```

authentication
    .and()
    .formLogin()
        .loginPage("/login")           // custom login page
        .loginProcessingUrl("/perform_login") // login form submit
URL
        .defaultSuccessUrl("/home", true)
        .permitAll()
    .and()
    .logout()
        .logoutUrl("/logout")
        .logoutSuccessUrl("/login?logout")
        .permitAll()
    .and()
    .exceptionHandling()
        .accessDeniedPage("/403"); // page for unauthorized access
}
}

```

Explanation of RBAC Configuration

1. Authentication Setup

- In-memory users with roles: **ADMIN** and **USER**.
- **{noop}** indicates plain-text passwords (for demo purposes).

2. Authorization Rules

- **.antMatchers("/admin/**").hasRole("ADMIN")** → only ADMIN can access **/admin** URLs.
- **.antMatchers("/user/**").hasRole("USER")** → only USER can access **/user** URLs.
- **.anyRequest().authenticated()** → all other URLs require login.

3. Login & Logout

- Custom login page with URL **/login**.
- Logout URL is **/logout**, redirects to login page after logout.

4. Access Denied Handling

- Redirects users to **/403** page if they try to access a URL they are not authorized for.

Key Points

- **RBAC = Authentication + Authorization.**
- Can be implemented at **URL level** (as shown) or **method level** using `@PreAuthorize` annotations.
- Enhances **security** by restricting sensitive resources to authorized roles.

Qus 5. Write short notes on the following :

(i) Design Patterns

(ii) Servlet Collaboration

(iii) Components of JSP

(iv) Spring Framework

Ans :

(i) Design Patterns

Definition:

Design Patterns are **reusable solutions** to **common software design problems**. They provide a **template for solving issues** in object-oriented design.

Types of Design Patterns:

1. **Creational:** Object creation mechanisms (e.g., Singleton, Factory, Builder)
2. **Structural:** Object composition (e.g., Adapter, Decorator)
3. **Behavioral:** Object interaction (e.g., Observer, Strategy)

Advantages:

- Promotes **code reuse**
- Improves **maintainability**
- Provides **standard solutions** for common problems

(ii) Servlet Collaboration

Definition:

Servlet Collaboration refers to the **mechanism where multiple servlets work together** to handle a client request.

Techniques for Collaboration:

1. **RequestDispatcher**
 - `forward()` → forwards request to another servlet
 - `include()` → includes output of another servlet
2. **Servlet Chaining**
 - One servlet calls another servlet to complete a request
3. **Shared Session or Context**
 - Use **HttpSession** or **ServletContext** to share data between servlets

Use Case:

- Separating request processing logic across multiple servlets
- Modular web application design

(iii) Components of JSP**JavaServer Pages (JSP) Components:**

1. **Directives**
 - Control page settings and compilation
 - Example: `<%@ page language="java" %>`
2. **Scriptlets**
 - Java code embedded in JSP
 - Example: `<% int x = 5; %>`
3. **Expressions**
 - Output Java values to the response
 - Example: `<%= x %>`
4. **Declarations**
 - Declare methods or variables
 - Example: `<%! int count; %>`
5. **Actions**
 - JSP tags that control behavior (e.g., `<jsp:include>`, `<jsp:useBean>`)
6. **Implicit Objects**

- Predefined objects like `request`, `response`, `session`, `out`

(iv) Spring Framework

Definition:

Spring is a **lightweight Java framework** for **enterprise application development**.

Core Features:

1. **Inversion of Control (IoC)** – container manages object creation and dependency injection
2. **Aspect-Oriented Programming (AOP)** – separates cross-cutting concerns like logging, security
3. **Transaction Management** – declarative or programmatic transaction support
4. **Data Access Support** – JDBC, ORM, and JPA integration
5. **Spring MVC** – web framework for building MVC applications

Advantages:

- Promotes **loose coupling**
- Simplifies **enterprise application development**
- Supports **modular and testable code**

June, 2024

Qus 1.

(a) Differentiate between web server and web container.

Ans :

Here's a **clear and exam-ready answer** explaining the **difference between a Web Server and a Web Container**:

Web Server vs Web Container

Feature	Web Server	Web Container (Servlet Container)
Definition	A software system that handles HTTP requests and responses from clients (usually browsers).	A part of a web server (or application server) that manages the execution of servlets and JSPs .
Primary Role	Delivers static content like HTML, CSS, JavaScript, and images.	Handles dynamic content generated by servlets and JSPs.
Examples	Apache HTTP Server, Nginx, Microsoft IIS	Apache Tomcat, Jetty, GlassFish, JBoss (servlet container inside application server)
Technology Support	Supports only static web pages (HTML, CSS, JS).	Supports Java EE technologies like Servlets, JSP, and Filters.
Processing	Processes static requests directly from the file system.	Invokes Java classes (servlets) to generate dynamic responses .
Lifecycle Management	Does not manage Java objects or servlet lifecycle.	Manages servlet lifecycle — <code>init()</code> , <code>service()</code> , <code>destroy()</code> .

Interaction	Acts as a front-end that forwards dynamic requests to web container (if integrated).	Executes servlets and JSPs, often embedded inside the web server.
Use Case	Best for hosting static websites or acting as a reverse proxy .	Best for running Java web applications that need backend logic.

Example Scenario:

1. A user requests `index.html` → handled directly by the **web server** (static content).
2. A user requests `/login` (a servlet URL) → forwarded by the web server to the **web container**, which executes the servlet and returns a dynamic response.

Summary

- **Web Server** → Handles **static content** (HTML, images, etc.)
- **Web Container** → Handles **dynamic content** (Servlets, JSPs)

Example:

- **Apache Tomcat** can act as **both a web server and a web container** — it serves static files and executes servlets/JSPs.
- **Apache HTTP Server** is a **pure web server** — to run servlets, it integrates with Tomcat (the web container).

(b) What are “Creational Design Patterns” ? Write their utility.

Ans :

Here’s a **clear and concise, exam-ready explanation** of **Creational Design Patterns** and their **utility** 📌

Definition:

Creational Design Patterns are a category of **design patterns** that deal with the **process of object creation**.

They **abstract the instantiation process** and help make the system **independent of how its objects are created, composed, and represented**.

In simple terms — these patterns **help create objects in a flexible, reusable, and controlled way** instead of using direct constructors (**new** keyword).

Purpose / Utility of Creational Design Patterns

Utility / Advantage	Description
1. Flexibility in Object Creation	Provides a flexible way to create objects without binding the application to specific classes.
2. Code Reusability	Object creation logic can be reused in multiple places.
3. Encapsulation	Hides the complex creation logic from client code.
4. Reduces Code Duplication	Centralizes object creation in one place (factory or builder).
5. Promotes Loose Coupling	Client code depends on interfaces rather than concrete classes.

Types of Creational Design Patterns

Pattern	Description / Utility
1. Singleton Pattern	Ensures only one instance of a class exists and provides a global point of access. <i>Use Case:</i> Database connection, Logger, Configuration manager.
2. Factory Method Pattern	Defines an interface for creating objects , but lets subclasses decide which class to instantiate. <i>Use Case:</i> When the exact type of object to create is determined at runtime.

- | | |
|------------------------------------|--|
| 3. Abstract Factory Pattern | Provides an interface for creating families of related objects without specifying their concrete classes. <i>Use Case:</i> Cross-platform GUI components (e.g., Windows/Mac buttons). |
| 4. Builder Pattern | Separates object construction from its representation so that the same construction process can create different representations. <i>Use Case:</i> Creating complex objects like reports or documents step-by-step. |
| 5. Prototype Pattern | Creates new objects by cloning existing ones (prototypes) . <i>Use Case:</i> When object creation is expensive (e.g., deep cloning configurations or game characters). |

Example: Singleton Pattern

```
public class DatabaseConnection {
    private static DatabaseConnection instance;

    private DatabaseConnection() {
        System.out.println("Database connected.");
    }

    public static DatabaseConnection getInstance() {
        if (instance == null)
            instance = new DatabaseConnection();
        return instance;
    }
}
```

Utility:

Ensures only one instance of `DatabaseConnection` exists throughout the application.

Summary

Aspect

Description

Category	Creational Design Pattern
Main Purpose	Object creation management
Key Benefit	Reduces tight coupling and complexity
Examples	Singleton, Factory, Builder, Prototype, Abstract Factory

(c) How struts 2 differs from struts 1 ? List the various features of struts 1.

Ans :

Difference Between Struts 1 and Struts 2

Basis	Struts 1	Struts 2
Architecture	Based on Servlet API (Action classes extend Action).	Based on MVC and interceptor-based architecture (uses POJO Actions).
Action Class	Must extend org.apache.struts.action.Action .	Plain Old Java Object (POJO) — no need to extend any framework class.
Thread Safety	Single Action instance serves multiple requests → not thread-safe .	New Action object is created for each request → thread-safe .
Request & Response Handling	Action classes depend on HttpServletRequest and HttpServletResponse .	Uses ValueStack and OGNL (Object Graph Navigation Language) to access data.

Configuration	Configured using struts-config.xml .	Uses annotation-based or XML-based (struts.xml) configuration.
Form Beans	Uses separate ActionForm beans.	No need for ActionForm — form data directly mapped to Action class properties.
Validation	Uses separate validate() method in ActionForm .	Uses XML or annotation-based validation (validation.xml) .
Tag Libraries	Limited and less flexible.	Enhanced UI tags with built-in AJAX support.
Integration	Difficult to integrate with modern frameworks.	Easily integrates with Spring, Hibernate , and other frameworks.
Result Types	Only forwards or redirects to JSPs.	Supports multiple result types (JSP, Velocity, Freemarker, etc.).

Features of Struts 1

1. MVC Architecture

- Implements the **Model-View-Controller** design pattern separating business logic, presentation, and control flow.

2. Action Classes and Form Beans

- Uses **Action** and **ActionForm** classes to handle requests and manage form data.

3. XML-Based Configuration

- Centralized configuration using **struts-config.xml** for defining mappings between requests and actions.

4. Tag Libraries

- Provides **HTML and logic tags** to simplify JSP development.

5. Validation Framework

- Built-in **form validation** using `validate()` method or separate XML-based validator rules.

6. Error Handling

- Built-in **exception handling** and error messages management through resource bundles.

7. Internationalization (i18n)

- Supports **multiple languages and locales** using resource bundles.

8. Extensibility

- Can be extended with custom plug-ins, interceptors, and actions.

9. Centralized Controller

- Uses `ActionServlet` as the **Front Controller**, managing all requests in a single entry point.

Summary

Aspect	Struts 1	Struts 2
Programming Model	Servlet-based	POJO-based
Thread Safety	Not thread-safe	Thread-safe
Form Handling	ActionForm required	Direct binding to Action class
Configuration	<code>struts-config.xml</code>	<code>struts.xml</code> / annotations
Ease of Use	Complex	Simplified and flexible

In short:

- **Struts 1**: Older, servlet-dependent, less flexible.

- **Struts 2:** Modern, POJO-based, annotation-driven, easier to integrate with other frameworks.

(d) What is Hibernate framework in Java ? What is the utility of Hibernate ? Give advantages of Hibenate framework.

Ans :

Hibernate is an **open-source Object-Relational Mapping (ORM) framework** for Java that simplifies interaction between **Java objects and relational databases**.

It allows developers to **map Java classes to database tables** and Java object properties to table columns, eliminating the need to write complex **JDBC code** (like **ResultSet**, **PreparedStatement**, etc.).

Hibernate is part of the **Java Persistence API (JPA)** ecosystem and supports automatic SQL generation and transaction management.

Utility of Hibernate

The **main utility** (or purpose) of Hibernate is to:

1. **Simplify database operations** — by automatically converting Java objects into database records (and vice versa).
2. **Reduce boilerplate JDBC code** — developers don't need to manually handle connections, statements, or result sets.
3. **Provide database independence** — the same code can work with different databases (MySQL, Oracle, PostgreSQL, etc.) with minimal changes.
4. **Manage relationships** — easily handle relationships such as **One-to-One**, **One-to-Many**, and **Many-to-Many**.
5. **Support caching** — improves performance by minimizing database access using first-level and second-level caches.
6. **Handle transactions automatically** — integrates with **JTA** and **Spring** for declarative transaction management.

Advantages of Hibernate Framework

Advantage

Description

- | | |
|--|--|
| 1. Simplified Data Access | No need to write SQL or JDBC code; Hibernate generates SQL automatically. |
| 2. Portability | Works with multiple databases without code changes — just update the configuration file. |
| 3. Caching Mechanism | Provides first-level and second-level caching to improve performance. |
| 4. Lazy and Eager Loading | Loads related data only when needed, reducing memory overhead. |
| 5. Transaction Management | Integrates seamlessly with JTA and Spring for declarative transaction handling. |
| 6. HQL (Hibernate Query Language) | Allows database-independent queries using object-oriented syntax instead of SQL. |
| 7. Automatic Table Creation | Can automatically create and update database schema using configuration properties. |
| 8. Easy Integration | Integrates well with other Java frameworks like Spring , Struts , and JSF . |
| 9. Maintains Object State | Automatically tracks object states — Transient, Persistent, and Detached. |
| 10. Open Source | Free and actively maintained under the GNU LGPL license. |

Example (Simple Hibernate Mapping)

```
@Entity
@Table(name = "employee")
public class Employee {
```

```

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private int id;

@Column(name = "emp_name")
private String name;

private double salary;

// Getters and setters
}

```

➡ This class maps to a database table named **employee** automatically. Hibernate handles the **SQL queries** behind the scenes.

Summary

- **Hibernate** = ORM framework for Java → bridges the gap between Java objects and relational databases.
- **Utility** → simplifies CRUD operations, ensures portability, and manages transactions efficiently.
- **Advantages** → automatic SQL handling, caching, portability, transaction management, and easy integration with frameworks like Spring.

(e) Differentiate between load() and get() methods.

Ans :

Difference between **load()** and **get()** methods in Hibernate

Basis	get() Method	load() Method
-------	---------------------	----------------------

Definition	Retrieves the object immediately from the database.	Returns a proxy object and fetches data only when needed (lazy loading).
Fetching Type	Eager loading — hits the database as soon as it is called.	Lazy loading — hits the database only when the object is actually used.
If Record Not Found	Returns null if the object does not exist.	Throws ObjectNotFoundException if the object does not exist.
Performance	Slower , as it always queries the database.	Faster , as it delays the query until data is required.
Return Type	Returns the real object .	Returns a proxy (dummy) object of the entity.
Use Case	When you are not sure whether the record exists or not.	When you are sure the record exists and want better performance.

Example

```
// Using get()
Employee emp1 = session.get(Employee.class, 1);
System.out.println(emp1.getName()); // Data fetched immediately

// Using load()
Employee emp2 = session.load(Employee.class, 2);
System.out.println(emp2.getName()); // Data fetched only here (lazy)
```

In short:

- `get()` → Immediate fetch → returns **null** if not found.

- `load()` → Lazy fetch → throws exception if not

(f) Explain the term Aspect Oriented Programming with suitable example.

Ans :

Aspect-Oriented Programming (AOP) is a programming paradigm that aims to **separate cross-cutting concerns** from the main business logic of an application.

A **cross-cutting concern** is a functionality that is used across multiple modules — such as **logging, security, transaction management, or exception handling** — but does not belong to the core business logic.

AOP allows developers to **add or remove such concerns declaratively** (without modifying the core code).

Key Concepts of AOP

Term	Description
Aspect	A module that encapsulates a cross-cutting concern (e.g., logging, security).
Join Point	A specific point in the program, such as method execution, where an aspect can be applied.
Advice	The action taken by an aspect at a particular join point (e.g., before, after, or around a method).
Pointcut	An expression that defines <i>where</i> an advice should be applied (which methods/classes).
Weaving	The process of applying aspects to the target objects at runtime, compile-time, or load-time.

Example: Logging with AOP (Spring Framework)

Without AOP (Traditional Approach)

You have to write logging code in every method:

```
public class PaymentService {  
  
    public void processPayment() {  
        System.out.println("Logging: Payment started.");  
        // Business logic  
        System.out.println("Processing payment...");  
        System.out.println("Logging: Payment completed.");  
    }  
}
```

✗ Problem: Logging code is repeated in every method → violates separation of concerns.

With AOP (Using Spring AOP)

You define a separate **Aspect** for logging:

Aspect Class

```
@Aspect  
@Component  
public class LoggingAspect {  
  
    @Before("execution(* com.example.service.PaymentService.*(..))")  
    public void logBefore() {  
        System.out.println("Logging: Method execution started...");  
    }  
  
    @After("execution(* com.example.service.PaymentService.*(..))")  
    public void logAfter() {  
        System.out.println("Logging: Method execution finished...");  
    }  
}
```

```
}
```

Business Logic

```
@Component
public class PaymentService {
    public void processPayment() {
        System.out.println("Processing payment...");
    }
}
```

Output:

Logging: Method execution started...

Processing payment...

Logging: Method execution finished...

Advantages of AOP

1. **Separation of Concerns** – Keeps cross-cutting code separate from business logic.
2. **Code Reusability** – Common functionality written once and applied across modules.
3. **Cleaner Code** – Reduces boilerplate and repetitive code.
4. **Easier Maintenance** – Changing an aspect (e.g., logging policy) affects all modules uniformly.
5. **Better Modularity** – Increases clarity and maintainability of large applications.

Summary

Aspect-Oriented Programming (AOP) allows developers to modularize cross-cutting concerns such as logging, security, and transactions into reusable **aspects**, improving **code modularity, readability, and maintenance**.

(g) Compare spring boot and spring MVC.

Ans :

Comparison between Spring Boot and Spring MVC

Basis	Spring MVC	Spring Boot
Definition	A web framework used to build web applications following the Model-View-Controller (MVC) architecture.	A framework built on top of Spring and Spring MVC to simplify configuration and development.
Configuration	Requires manual configuration of DispatcherServlet, ViewResolver, DataSource, etc.	Provides auto-configuration , which automatically sets up required components.
Setup Complexity	More complex — needs <code>web.xml</code> , context files, and dependency management.	Very simple — minimal setup using annotations and starter dependencies.
Dependency Management	Requires developer to add and configure dependencies manually in <code>pom.xml</code> or <code>build.gradle</code> .	Provides Spring Boot Starters (predefined dependency bundles) for easy setup.
Embedded Server	Does not provide an embedded server; needs deployment on external servers like Tomcat or Jetty.	Comes with an embedded server (Tomcat, Jetty, Undertow) — runs as a standalone JAR.
Deployment	Deployed as a WAR file to an external server.	Can be deployed as a JAR or WAR file — JAR runs directly via <code>java -jar</code> .

Configuration Files	Uses XML or Java-based configuration for beans and mappings.	Uses application.properties or application.yml for configuration.
Focus	Provides framework for building web apps with MVC pattern.	Provides rapid application development by simplifying Spring configuration.
Learning Curve	Slightly steep — requires understanding of Spring configuration.	Easier to start — less boilerplate and fewer configurations.
Actuator Support	Not available by default.	Provides Spring Boot Actuator for monitoring and health checks.
Goal	Build structured, modular web applications.	Build production-ready Spring applications quickly.

Example Use Case

Use Case	Preferred Framework
Building a traditional, manually controlled MVC web app	Spring MVC
Building a standalone REST API or microservice quickly	Spring Boot

Summary

- **Spring MVC** → Framework for developing web apps using MVC design pattern.
- **Spring Boot** → Simplifies Spring MVC development with **auto-configuration**, **embedded servers**, and **starter dependencies**.

In short:

Spring Boot **uses** Spring MVC internally, but removes the need for complex setup and configuration — making it ideal for **rapid development and microservices**.

(h) What is message digest ? Write the steps to send encrypted data with message digest and for the validation of encrypted data.

Ans :

A **Message Digest** is a **fixed-size hash value** generated from an arbitrary amount of input data (message) using a **hash function**.

It is a key concept in **data integrity** and **cryptographic security**, ensuring that the message has **not been altered** during transmission.

Definition:

A **Message Digest** is the **digital fingerprint** of a message — a short, fixed-length value computed using a **one-way hash algorithm** like **SHA-256** or **MD5**.

Properties of Message Digest:

1. **Fixed length:** Output size is always the same, regardless of input size.
 - e.g., SHA-256 → 256 bits, MD5 → 128 bits
2. **Irreversible:** Cannot regenerate the original message from its digest.
3. **Unique:** Small changes in input → completely different digest (Avalanche effect).
4. **Deterministic:** Same input always produces the same digest.
5. **Integrity Check:** Detects if even a single bit of data changes.

Common Hash Algorithms

- **MD5** (Message Digest Algorithm 5) – produces a 128-bit hash.
- **SHA-1** (Secure Hash Algorithm 1) – produces a 160-bit hash.
- **SHA-256 / SHA-512** – stronger and widely used in modern systems.

 **Steps to Send Encrypted Data with Message Digest**

When sending confidential data over a network, **encryption** and **message digest** are often combined to ensure **confidentiality** and **integrity**.

Step-by-Step Process

1 Sender Side (Encryption and Digest Creation)

1. **Original Message:**

The sender starts with a message M .

2. **Generate Message Digest:**

Apply a hash function (like SHA-256) to the message.

```
[
MD = Hash(M)
]
```

3. **Encrypt the Message (optional):**

Encrypt M using a secret or public key encryption algorithm (e.g., AES, RSA).

```
[
E(M) = Encrypt(M)
]
```

4. **Attach Digest:**

The sender sends both $E(M)$ and the digest MD to the receiver.

2 Receiver Side (Decryption and Validation)

1. **Decrypt the Message:**

Receiver decrypts the message:

```
[
M' = Decrypt(E(M))
]
```

2. **Recalculate Digest:**

Apply the same hash function on M' to generate MD' :

```
[
MD' = Hash(M')
]
```

3. Compare Digests:

Compare received digest **MD** with calculated digest **MD'** :

- If **MD = MD'**, → message is **authentic and unaltered**.
- If **MD ≠ MD'**, → message has been **tampered** or corrupted.

Illustration

Sender Side:

Message (M) → [Hash Function] → Message Digest (MD)

Message (M) → [Encryption] → Encrypted Message (E(M))

Send → E(M) + MD

Receiver Side:

E(M) → [Decryption] → Message (M')

M' → [Hash Function] → MD'

Compare: MD == MD' ? → Valid / Tampered

Example (Conceptual)

Let's assume:

Message: "Hello World"

Hash (SHA-256): a591a6d40bf420404a0117... (Digest)

Encrypted Message: Yh28sld88slf==

Sender sends:

Encrypted Message: Yh28sld88slf==

Digest: a591a6d40bf420404a0117...

Receiver checks:

- Decrypts message → "Hello World"
- Hashes again → a591a6d40bf420404a0117...
 ✓ Matches → Data is valid and untampered.

Summary

Aspect	Explanation
Message Digest	Fixed-length hash of a message ensuring data integrity.
Purpose	Detects any unauthorized modification of the message.
Used With	Encryption and digital signatures for secure communication.
Validation	Receiver compares recalculated digest with received digest.

Qus 2.

(a) Explain the various components of J2EE architecture with suitable block diagram.

Ans :

J2EE (Java 2 Platform, Enterprise Edition) is a **platform-independent, Java-based framework** used for developing, building, and deploying **enterprise-level web applications**.

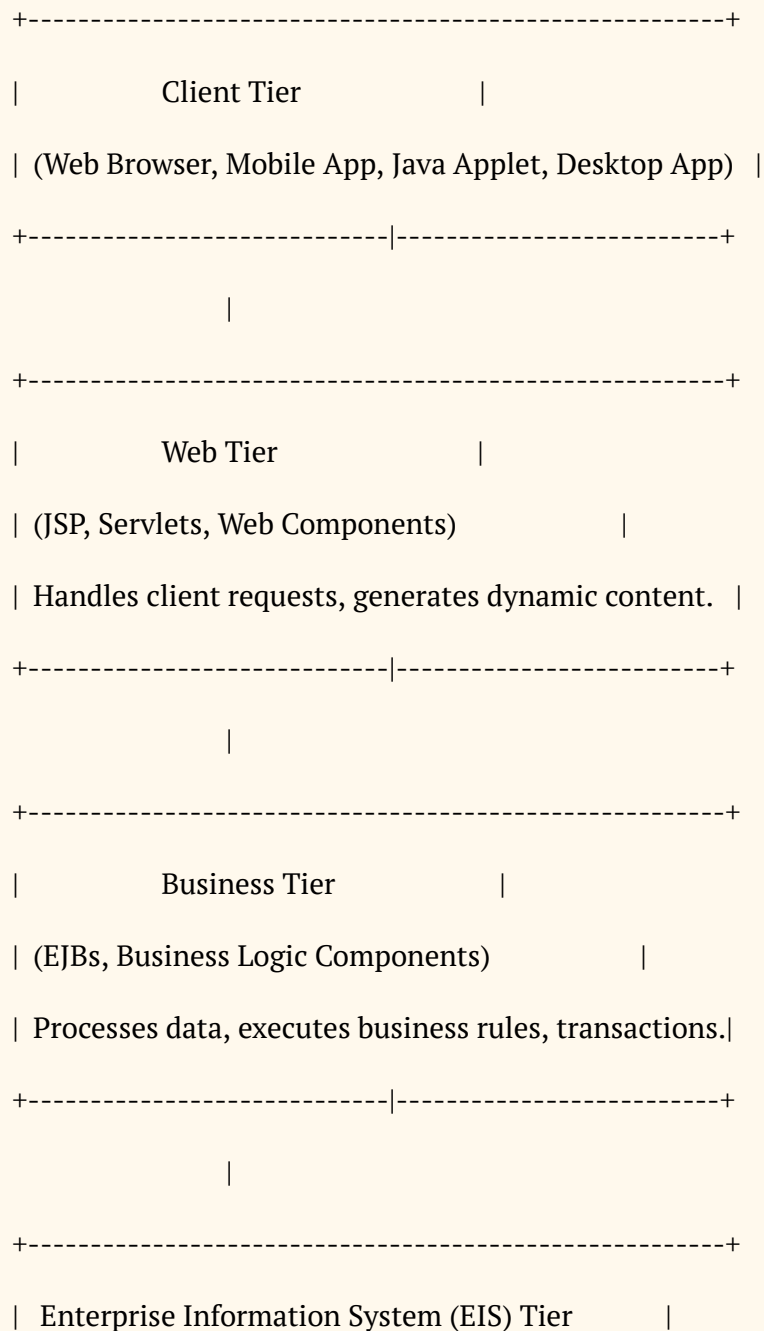
It extends the **Java Standard Edition (Java SE)** by providing APIs and services for **distributed computing, web services, transaction management, and multi-tiered applications**.

J2EE Multi-Tier Architecture

The **J2EE architecture** is **multi-tiered**, typically divided into **four logical layers**:

1. Client Tier
2. Web Tier
3. Business Tier
4. Enterprise Information System (EIS) Tier

Block Diagram of J2EE Architecture



| (Databases, ERP Systems, Legacy Systems) |

| Stores and retrieves persistent data. |

+-----+

Components of J2EE Architecture

Client Tier (Presentation Layer)

- Represents **end-user applications** that interact with the server.
- Can be a **web browser**, **mobile app**, or **desktop application**.
- Sends requests to the web server and displays responses.

Technologies Used:

- HTML, CSS, JavaScript
- Java Applets or JavaFX
- Web Clients (through JSP/Servlets)

Web Tier (Web Layer)

- Acts as the **interface between client and business logic**.
- Receives client requests, processes them, and sends responses.
- Handles **presentation logic** and **request routing**.

Technologies Used:

- **Servlets:** For handling HTTP requests and responses.
- **JSP (JavaServer Pages):** For generating dynamic HTML content.
- **Web Container:** Manages lifecycle of servlets and JSPs (e.g., Apache Tomcat).

Business Tier (Application Layer)

- Contains **business logic** — the rules and operations that define how data is processed.
- Handles **transactions**, **security**, and **object persistence**.
- Separates business logic from presentation logic.

Technologies Used:

- **EJB (Enterprise JavaBeans):** For distributed, transactional, and secure business components.
- **JTA (Java Transaction API):** For managing transactions.
- **JMS (Java Message Service):** For asynchronous communication.

Enterprise Information System (EIS) Tier

- The **data storage layer**.
- Interacts with **databases, ERP, or legacy systems**.
- Responsible for **data persistence and retrieval**.

Technologies Used:

- **JDBC (Java Database Connectivity)**
- **JCA (Java Connector Architecture)**
- **Entity Beans / ORM tools (like Hibernate)**

J2EE Containers

J2EE uses **containers** to manage various components and services:

Container Type	Purpose
Web Container	Manages Servlets, JSPs, and handles HTTP communication.
EJB Container	Manages business logic components (EJBs) and transactions.
Application Client Container	Runs standalone Java client applications.
Applet Container	Runs Java applets in web browsers.

Advantages of J2EE Architecture

1. **Platform Independent** – runs on any system with JVM.
2. **Scalable and Distributed** – supports multi-tier and load-balanced systems.

3. **Reusable Components** – modular development using EJBs, JSPs, and Servlets.
4. **Transaction and Security Support** – built-in APIs for authentication and transaction management.
5. **Integration** – connects easily with databases and legacy systems.

Summary

Layer	Main Function	Technologies Used
Client Tier	User Interface	HTML, JavaFX, JSP
Web Tier	Request Handling, UI Logic	Servlets, JSP, JSF
Business Tier	Business Logic	EJB, JTA, JMS
EIS Tier	Data Storage & Integration	JDBC, JCA, Hibernate

In short:

J2EE Architecture is a **multi-tier enterprise framework** that separates presentation, business logic, and data layers — making web applications **modular, scalable, and maintainable**.

(b) Explain the servlet life cycle. Also compare the generic servlet with HTTP servlet.

Ans :

A **Servlet** is a **Java program** that runs on a **web server** and is used to create **dynamic web content**.

Servlets handle **client requests** (usually HTTP) and generate **responses** (such as HTML or JSON).

Servlets are managed by a **Servlet Container** (e.g., Apache Tomcat), which controls their **life cycle**.

Servlet Life Cycle

The **Servlet Life Cycle** defines the **stages** through which a servlet passes — from creation to destruction.

The Servlet container manages the **life cycle** through the following **five main phases**:

1. Loading and Instantiation

- The servlet class is **loaded** into memory by the container when it is first requested (or at startup if configured with `<load-on-startup>`).
- A **single instance** of the servlet is created.

```
public class MyServlet extends HttpServlet {

    // Servlet class loaded and instance created

}
```

2. Initialization (`init()` method)

- Called **once** when the servlet is first created.
- Used for **initial setup** tasks like establishing database connections or reading configuration parameters.

```
public void init() throws ServletException {

    // Initialization code

    System.out.println("Servlet is initializing...");

}
```

3. Request Handling (`service()` method)

- Called **for each client request**.
- It determines the type of request (GET, POST, etc.) and dispatches it to the appropriate method (`doGet()`, `doPost()`, etc.).

```
public void service(ServletRequest req, ServletResponse res)

    throws ServletException, IOException {
```

```
// Code to process request and generate response

}
```

4. Destruction (destroy() method)

- Called **once** when the servlet is taken out of service (e.g., during server shutdown).
- Used to release resources like closing database connections or files.

```
public void destroy() {

    // Cleanup code

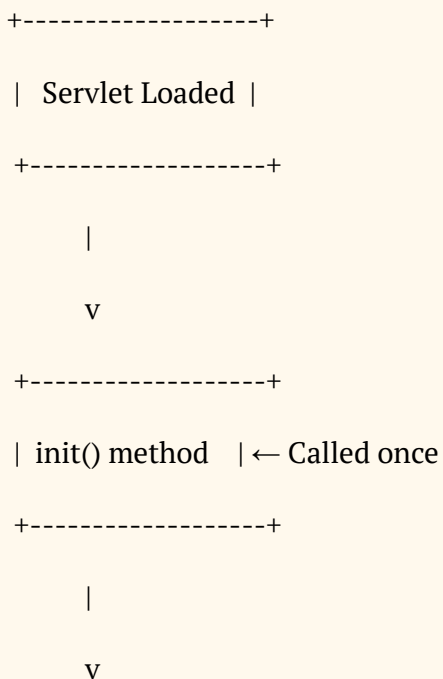
    System.out.println("Servlet is being destroyed...");

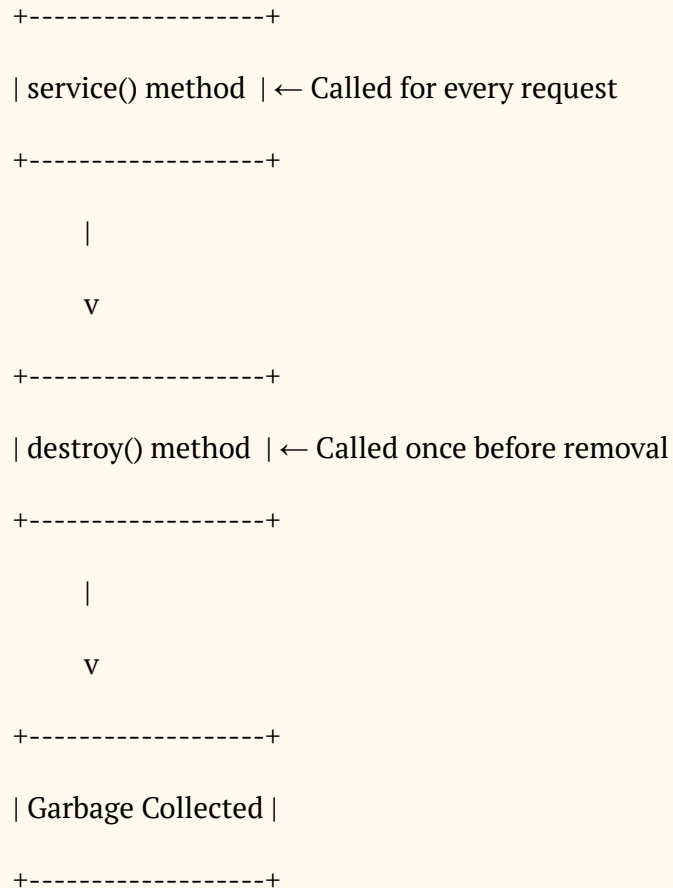
}
```

5. Garbage Collection

- After `destroy()` is called and no references remain, the servlet object is **garbage collected** by the JVM.

Servlet Life Cycle Flow Diagram





Comparison: GenericServlet vs HttpServlet

Feature	GenericServlet	HttpServlet
Package	<code>javax.servlet</code>	<code>javax.servlet.http</code>
Protocol Support	Protocol-independent (can support FTP, SMTP, etc.)	Specifically designed for HTTP protocol
Methods	Defines <code>service(ServletRequest, ServletResponse)</code> method	Defines <code>doGet()</code> , <code>doPost()</code> , <code>doPut()</code> , <code>doDelete()</code> etc.

Request Handling	Must override <code>service()</code> method	Overrides <code>doGet()</code> or <code>doPost()</code> for HTTP requests
Session Management	No built-in session management	Built-in session management using cookies and session tracking
Usage	Used for developing generic protocol-independent servlets	Used for developing web applications using HTTP
Example	<code>public class MyServlet extends GenericServlet</code>	<code>public class MyHttpServlet extends HttpServlet</code>

Example: Simple HttpServlet

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class HelloServlet extends HttpServlet {
    public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        out.println("<h2>Welcome to Servlet Life Cycle Example!</h2>");
    }

    public void init() throws ServletException {
        System.out.println("Servlet Initialized");
    }

    public void destroy() {
        System.out.println("Servlet Destroyed");
    }
}
```

Summary

Stage	Method	Purpose
Initialization	<code>init()</code>	Initializes resources
Request Handling	<code>service() / doGet() / doPost()</code>	Processes client requests
Destruction	<code>destroy()</code>	Frees resources

In short:

The **Servlet life cycle** ensures efficient management of servlet instances — loading once, serving multiple requests, and cleaning up when no longer needed.

HttpServlet extends **GenericServlet** to provide HTTP-specific functionality, simplifying web development.

(c) Discuss the role of cookies in session management. How does persistent cookies differ from non-persistent cookies ?

Ans :

Role of Cookies in Session Management

What are Cookies?

A **cookie** is a small piece of information (a key–value pair) stored by a **web browser** on the **client's computer**.

Cookies are sent by the **server** to the **client** via HTTP headers and then sent back by the browser with each subsequent request to the same server.

In Java, cookies are represented by the class `javax.servlet.http.Cookie`.

Why Cookies are Needed in Session Management

HTTP is a **stateless protocol**, meaning that every client request is independent — the server does not remember previous interactions.

To maintain **user session information** (like login state, shopping cart contents, preferences, etc.), cookies are used to **associate multiple requests with a single user session**.

How Cookies Work in Session Management

1. **Client sends first request** → No cookie is present.
2. **Server creates a new session** and generates a unique **session ID**.
3. **Server sends the cookie** (with session ID) to the client in the HTTP response header.
4. **Browser stores the cookie** locally.
5. **For subsequent requests**, the browser automatically sends the cookie back to the server.
6. **Server identifies the session** using the session ID stored in the cookie.

Example: Creating and Reading Cookies in Servlet

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class CookieExample extends HttpServlet {
    public void doGet(HttpServletRequest request, HttpServletResponse
response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Create a new cookie
        Cookie cookie = new Cookie("username", "Ritesh");
        cookie.setMaxAge(60 * 60 * 24); // 1 day in seconds
        response.addCookie(cookie); // Add cookie to response

        out.println("<h3>Cookie 'username' is set successfully!</h3>");

        // Retrieve cookies from request
```



```

        Cookie[] cookies = request.getCookies();
        if (cookies != null) {
            for (Cookie c : cookies) {
                out.println("<p>" + c.getName() + " : " + c.getValue() +
"</p>");
            }
        }
    }
}

```

Types of Cookies

Cookies can be classified into two main types based on their **lifetime**:

Type	Description	Storage Duration	Example Use
Persistent Cookie	Stored on client's hard drive and remains available even after the browser is closed.	Until the expiry time set using <code>setMaxAge()</code> is reached.	Remembering login credentials or user preferences.
Non-Persistent Cookie (Session Cookie)	Stored only in browser's memory (RAM) and is deleted when the browser is closed.	Only for the current session.	Temporary session tracking, like items in a shopping cart.

Difference between Persistent and Non-Persistent Cookies

Basis	Persistent Cookie	Non-Persistent (Session) Cookie
Storage	Stored on the user's local disk	Stored in browser memory (RAM)

Duration	Exists beyond the browser session (until expiry date/time)	Exists only for the duration of the current session
Set using	<code>cookie.setMaxAge(time_in_seconds)</code>	No expiry time set (<code>setMaxAge()</code> not used)
Deleted when	Expiry time is reached or user manually clears cookies	Browser is closed
Use Case	“Remember Me” features, saved preferences	Temporary session identification, shopping cart

In Summary

- **Cookies** enable **session tracking** by storing small pieces of user-specific information on the client side.
- They help maintain **state** across multiple HTTP requests.
- **Persistent**
- **cookies** survive browser restarts, while **non-persistent cookies** are deleted when the session ends.

Key Servlet Cookie Methods

Method	Description
<code>new Cookie(String name, String value)</code>	Creates a cookie
<code>setMaxAge(int expiry)</code>	Sets expiry time in seconds
<code>getName()</code>	Returns cookie name
<code>getValue()</code>	Returns cookie value

```
response.addCookie(Cookie c)
```

Adds cookie to response

```
request.getCookies()
```

Retrieves all cookies sent by client

Qus 3.

(a) Explain the cooking and flow of struts 2 with suitable block diagram.

Ans :

Struts 2 is an **MVC (Model–View–Controller)** based **Java web application framework** used to develop dynamic enterprise web applications.

It is the upgraded version of **Apache Struts 1**, jointly developed by **Apache Software Foundation** and **OpenSymphony**.

Struts 2 integrates the best features of **Struts 1** and **WebWork framework**.

Working / Flow of Struts 2 Framework

The Struts 2 request processing flow can be described in **10 key steps**, as shown below 📌

Step-by-Step Flow of Struts 2

❶ User Request

- The user sends a request (for example, by clicking a button or submitting a form).
- This request is first sent to the **Filter Dispatcher** (in older versions) or **StrutsPrepareAndExecuteFilter** (in newer versions).

❷ Filter Dispatcher / Front Controller

- Acts as the **Front Controller** of the framework.
- Intercepts all incoming requests.
- Determines the **action** to be executed based on the **struts.xml** configuration.

❸ Action Mapping

- The **Filter Dispatcher** reads the configuration file **struts.xml** to find the corresponding **Action class** for the request URL.

4 Interceptors Stack

- Before the Action class executes, a series of **Interceptors** are applied.
- Interceptors perform common tasks like input validation, logging, file upload, authentication, etc.

5 Action Execution

- The mapped **Action class** (Java class) is executed.
- The Action interacts with the **Model** or **business logic** (like database operations through Hibernate, JDBC, etc.).

6 Return Result

- The Action class returns a **result name** (like “success”, “error”, etc.) to the framework.

7 Result Mapping

- The framework again checks **struts.xml** for the corresponding **result page (JSP or another view)** mapped with that result name.

8 View Rendering

- The selected **View (usually a JSP page)** is rendered to display the response to the user.

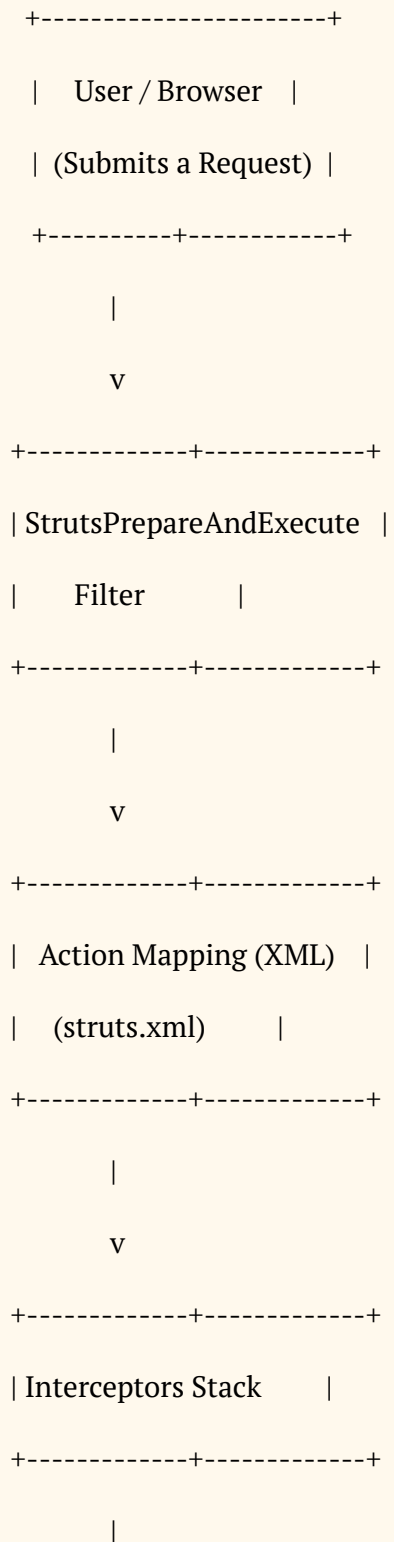
9 Response to Client

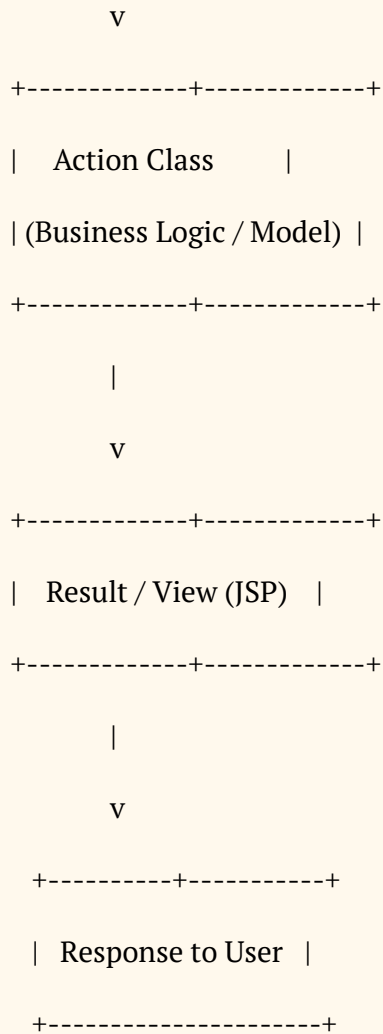
- The response is sent back to the **user’s browser**.

10 End of Request

- The Interceptors execute **post-processing** logic (like cleanup or resource release) and the response cycle completes.

Struts 2 Request Flow – Block Diagram





Example of **struts.xml** Configuration

```

<struts>
  <package name="default" extends="struts-default">
    <action name="login" class="com.example.LoginAction">
      <result name="success">/welcome.jsp</result>
      <result name="error">/error.jsp</result>
    </action>
  </package>
</struts>
  
```

Example Action Class

```
package com.example;

import com.opensymphony.xwork2.ActionSupport;

public class LoginAction extends ActionSupport {
    private String username;
    private String password;

    public String execute() {
        if ("admin".equals(username) && "1234".equals(password)) {
            return SUCCESS;
        } else {
            return ERROR;
        }
    }

    // Getters and Setters
}
```

Summary of Key Components

Component	Role
Filter Dispatcher	Front Controller, routes all requests.
Action Class	Contains business logic.
Interceptors	Performs cross-cutting tasks (validation, logging, etc.).
Result (JSP/View)	Displays response to user.
struts.xml	Defines mappings between actions and results.

Conclusion

- Struts 2 uses a **MVC architecture** and a **Front Controller pattern**.
- The **Filter Dispatcher** controls the entire request-response cycle.
- **Interceptors** and **Actions** provide flexibility and separation of concerns, making Struts 2 powerful for building large-scale Java web applications.

(b) How does Built-in Java Annotation differ from Java Custom Annotation ? Briefly discuss the various annotations classified under each.

Ans :

Annotations in Java are **metadata** (data about data) used to provide **information to the compiler or runtime environment**.

They do **not affect program logic** but help tools, libraries, and the JVM perform certain operations like validation, compilation checks, dependency injection, etc.

Difference between Built-in and Custom Annotations

Basis	Built-in Annotations	Custom Annotations
Definition	Predefined annotations provided by the Java language.	User-defined annotations created using <code>@interface</code> .
Purpose	Used to give instructions to compiler or runtime (e.g., suppress warnings, override methods).	Used to define custom metadata for special processing (e.g., frameworks like Spring, Hibernate).
Declared by	Java (in <code>java.lang</code> or <code>java.lang.annotation</code> packages).	Programmer-defined using <code>@interface</code> .
Processing	Automatically handled by Java compiler.	Requires manual processing (via reflection or frameworks).

Example `@Override, @Deprecated,` `@MyAnnotation, @Entity,`
 `@SuppressWarnings` `@Autowired`

Built-in Java Annotations

These are **standard annotations** provided by Java to help the compiler and runtime.

A. Compiler Annotations

Annotation	Description
<code>@Override</code>	Ensures that a method overrides a method from its superclass.
<code>@Deprecated</code>	Marks a method, class, or field as deprecated (not recommended for use).
<code>@SuppressWarnings</code>	Instructs the compiler to ignore specific warnings .

Example:

```
class Example {
    @Deprecated
    void oldMethod() {
        System.out.println("Deprecated method");
    }

    @Override
    public String toString() {
        return "Example class";
    }

    @SuppressWarnings("unchecked")
    void demo() {
        java.util.List list = new java.util.ArrayList();
        list.add("Test");
    }
}
```

B. Meta-Annotations

These are annotations that apply **to other annotations** — they define how annotations behave.

Annotation	Purpose
<code>@Target</code>	Specifies where the annotation can be applied (class, method, field, etc.).
<code>@Retention</code>	Defines how long the annotation is retained (source, class, or runtime).
<code>@Documented</code>	Indicates the annotation should appear in Javadoc.
<code>@Inherited</code>	Allows subclasses to inherit an annotation from their parent class.
<code>@Repeatable</code>	Allows the same annotation to be applied multiple times on a single element.

Example:

```
import java.lang.annotation.*;

@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface MyMeta {
    String value();
}
```

Custom (User-Defined) Annotations

Custom annotations are defined by programmers to **add metadata** that can later be **read and processed** at runtime using **Reflection API** or frameworks.

Creating a Custom Annotation

Syntax:

```
@interface AnnotationName {  
    // Elements or methods (optional)  
}
```

Example:

```
import java.lang.annotation.*;  
  
@Target(ElementType.METHOD)  
@Retention(RetentionPolicy.RUNTIME)  
@interface MyAnnotation {  
    String author() default "Ritesh";  
    int version() default 1;  
}
```

Using the Custom Annotation:

```
public class Demo {  
    @MyAnnotation(author = "Ritesh Tiwari", version = 2)  
    public void display() {  
        System.out.println("Custom annotation example");  
    }  
}
```

```
}
```

Processing Custom Annotation (via Reflection):

```
import java.lang.reflect.Method;

public class TestAnnotation {

    public static void main(String[] args) throws Exception {

        Method m = Demo.class.getMethod("display");

        MyAnnotation anno = m.getAnnotation(MyAnnotation.class);

        System.out.println("Author: " + anno.author());

        System.out.println("Version: " + anno.version());

    }

}
```

Classification of Annotations

Type	Examples	Purpose
Built-in (Predefined)	<code>@Override</code> , <code>@Deprecated</code> , <code>@SuppressWarnings</code>	Provided by Java for compiler and runtime checks
Meta-Annotations	<code>@Target</code> , <code>@Retention</code> , <code>@Documented</code> , <code>@Inherited</code> , <code>@Repeatable</code>	Define behavior of other annotations

Custom (User-Defined)	<code>@Entity, @Autowired, @MyAnnotation</code>	Developer-defined metadata for frameworks or custom logic
------------------------------	---	---

Summary

Aspect	Built-in Annotation	Custom Annotation
Defined by	Java Language	Programmer
Processing	By compiler/runtime	By reflection or frameworks
Usage	Code checks, compiler instructions	Custom metadata and automation
Example	<code>@Override, @Deprecated</code>	<code>@Entity, @Autowired, @MyAnnotation</code>

Qus 4.

(a) What is spring boot ? Explain the need of spring boot. Describe the working of spring boot with suitable block diagram.

Ans :

Spring Boot is a framework built on top of the **Spring Framework** that simplifies the process of developing **standalone, production-grade, and Spring-based applications** with **minimal configuration**.

It eliminates the need for complex XML configurations by providing:

- Auto-configuration
- Embedded servers (Tomcat, Jetty)
- Starter dependencies
- Production-ready tools

Definition

Spring Boot is an extension of the Spring framework that provides a faster and easier way to set up, configure, and run both simple and web-based applications.

Need for Spring Boot

Traditional **Spring applications** required a lot of manual configuration — XML files, dependency management, and web server setup.

Spring Boot was introduced to overcome these **complexities** by providing a **ready-to-use setup**.

Why Spring Boot is Needed (Advantages)

Need / Feature	Explanation
Simplified Configuration	Eliminates the need for complex XML files through auto-configuration .
Embedded Web Servers	Built-in servers like Tomcat , Jetty , or Undertow let you run apps directly (no WAR deployment needed).
Starter Dependencies	Provides Spring Boot Starters to automatically include commonly used dependencies (e.g., <code>spring-boot-starter-web</code>).
Rapid Development	Ideal for microservices and RESTful APIs .
Production Ready	Offers built-in actuators for monitoring, metrics, and health checks.
Reduced Boilerplate Code	Focus only on business logic, not on infrastructure setup.

Working of Spring Boot (Flow Explanation)

Spring Boot applications work by combining **auto-configuration**, **component scanning**, and **starter dependencies** to automatically create and run Spring applications.

Let's understand step by step 📌

Step-by-Step Working Flow

1 Application Entry Point

Every Spring Boot application starts with a **main class** annotated with:

`@SpringBootApplication`

- This annotation is a combination of:
 - `@Configuration` → Marks the class as a configuration source.
 - `@EnableAutoConfiguration` → Enables automatic configuration based on dependencies.
 - `@ComponentScan` → Scans the base package for components, services, and controllers.

2 Auto-Configuration

- Spring Boot automatically configures the application based on the **JAR dependencies** present in the classpath.
- For example, if `spring-boot-starter-web` is present → it auto-configures a **DispatcherServlet** and **Tomcat server**.

3 Component Scanning

- Spring Boot scans the package for classes annotated with:
 - `@Component`
 - `@Service`
 - `@Repository`
 - `@Controller` or `@RestController`
- These components are registered in the **Spring Application Context** automatically.

4 Embedded Server Start

- An **embedded web server** (like Tomcat) is started automatically.

You can run the application directly using:

```
java -jar appname.jar
```

- or from an IDE.

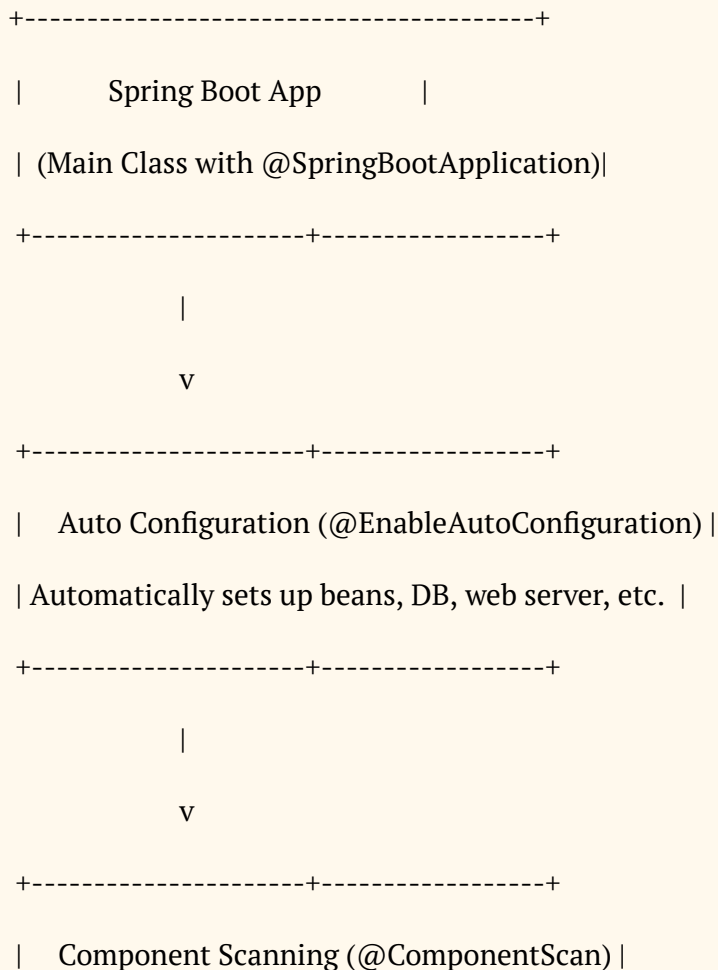
5 Handling Web Requests

- Requests are handled by **Controllers**, and responses are generated (e.g., as HTML or JSON).
- The **DispatcherServlet** acts as the **Front Controller**, managing request flow.

6 Spring Boot Actuator (Optional)

- Monitors and manages the application through endpoints like `/actuator/health`, `/actuator/info`.

Block Diagram: Spring Boot Architecture



| Detects @Controller, @Service, @Repository beans |

+-----+-----+

|

v

+-----+-----+

| Embedded Server (Tomcat/Jetty/Undertow) |

| Starts automatically |

+-----+-----+

|

v

+-----+-----+

| DispatcherServlet (Front Controller) |

| Handles requests and sends responses |

+-----+-----+

|

v

+-----+-----+

| REST Controllers / Business Logic |

| Process data and return output |

+-----+-----+

Example: Simple Spring Boot Application

```
import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;

@SpringBootApplication
@RestController
public class MyApp {

    public static void main(String[] args) {
        SpringApplication.run(MyApp.class, args);
    }

    @GetMapping("/")
    public String home() {
        return "Welcome to Spring Boot Application!";
    }
}
```

Output:

Welcome to Spring Boot Application!

Summary

Feature	Description
Framework Type	Simplified Spring Framework
Key Annotation	<code>@SpringBootApplication</code>
Core Features	Auto-configuration, Embedded Servers, Starters
Purpose	Rapid development and deployment of Java apps
Architecture	Based on Spring MVC + Auto Configuration + Embedded Server

(b) Explain any four of the following web application security vulnerabilities :

- (i) Cross site scripting**
- (ii) Cross site request forgery**
- (iii) SQL injection**
- (iv) Denial of Service**
- (v) Insecure direct object reference**

Ans :

Web Application Security Vulnerabilities

Web applications often handle sensitive user data such as login credentials, payment details, or personal information.

If not properly secured, attackers can exploit **vulnerabilities** to steal data, impersonate users, or disrupt services.

Below are **four common web security vulnerabilities** 📌

(i) Cross-Site Scripting (XSS)

♦ **Definition:**

Cross-Site Scripting (XSS) occurs when an attacker injects **malicious JavaScript code** into a web page viewed by other users.

The browser executes this script, thinking it's from a trusted site.

♦ **How it Works:**

- The attacker finds an input field (like a comment box or search bar) that doesn't properly validate or escape user input.
- Malicious code is inserted (e.g., `<script>alert('Hacked!')</script>`).
- When another user loads the page, the script runs in their browser.

♦ **Example:**

```
<input type="text" value="<script>alert('You are hacked!')</script>">
```

When displayed on the page, this triggers a JavaScript alert — or worse, steals cookies/sessions.

♦ **Prevention:**

- Escape all user input before rendering it in HTML.
- Use frameworks that automatically escape data (e.g., React, Angular).
- Implement a **Content Security Policy (CSP)**.

✚ (ii) **Cross-Site Request Forgery (CSRF)**

♦ **Definition:**

CSRF tricks an authenticated user's browser into making **unintended requests** to a web application, **without the user's consent**.

♦ **How it Works:**

- The user logs into a website (e.g., bank.com).
- The session cookie is stored in the browser.
- The attacker sends the user a malicious link or embedded form (e.g., in an email).
- When the user clicks it, their browser unknowingly sends a request to bank.com (like transferring money).

♦ **Example:**

```

```

If the user is logged in, this could transfer money automatically.

♦ **Prevention:**

- Use **CSRF tokens** in forms.
- Validate the HTTP **Referer** header.
- Require **re-authentication** for sensitive actions.

✖ (iii) SQL Injection (SQLi)

◆ Definition:

SQL Injection occurs when an attacker inserts **malicious SQL code** into a database query, allowing them to **access, modify, or delete** database data.

◆ How it Works:

- A web application takes user input (like a login form) and constructs an SQL query **without validation**.
- The attacker manipulates the query to bypass authentication or expose data.

◆ Example:

String query = "SELECT * FROM users WHERE username='" + user + "' AND password='" + pass + "'";

If the attacker enters:

user = admin

pass = ' OR '1'='1

The query becomes:

SELECT * FROM users WHERE username='admin' AND password=' OR '1'='1';

This always returns **true**, logging in the attacker.

◆ Prevention:

- Use **Prepared Statements** or **Parameterized Queries**.
- Validate and sanitize all input.
- Use ORM frameworks like Hibernate or JPA.

✖ (iv) Denial of Service (DoS)

◆ Definition:

A **Denial of Service (DoS)** attack aims to make a website or web server **unavailable to legitimate users** by overwhelming it with **excessive traffic or requests**.

◆ How it Works:

- Attackers flood the server with requests, exhausting CPU, memory, or bandwidth resources.
- The server slows down or crashes, causing **service disruption**.

◆ Example:

A botnet repeatedly sends thousands of requests per second to a web application:

```
GET /index.html HTTP/1.1
```

```
Host: targetsite.com
```

◆ Prevention:

- Use **rate limiting** and **firewalls**.
- Implement **CAPTCHA** for critical operations.
- Use **CDNs (Content Delivery Networks)** to absorb high traffic.
- Monitor unusual traffic patterns.

✖ (v) Insecure Direct Object Reference (IDOR)

◆ Definition:

IDOR occurs when an application provides direct access to an internal object (like a file, record, or database entry) using user-supplied input **without proper authorization checks**.

◆ How it Works:

- URLs or APIs expose internal identifiers (IDs) directly.

- The attacker changes the ID in the URL to access other users' data.

♦ **Example:**

`https://bank.com/account?accNo=1001`

If the attacker changes it to:

`https://bank.com/account?accNo=1002`

They may see another user's account details.

♦ **Prevention:**

- Enforce **authorization checks** for every request.
- Use **indirect references** (e.g., random tokens instead of IDs).
- Never trust user input to control access directly.

✓ **Summary Table**

Vulnerability	Full Form	Attack Type	Goal	Prevention
XSS	Cross-Site Scripting	Code Injection	Run malicious scripts	Escape user input
CSRF	Cross-Site Request Forgery	Request Forgery	Perform actions as user	Use CSRF tokens
SQLi	SQL Injection	Code Injection	Steal or modify data	Use Prepared Statements
DoS	Denial of Service	Resource Exhaustion	Crash or slow service	Rate limit, firewalls

IDOR	Insecure Direct Object Reference	Access Control Flaw	Access unauthorized data	Validate authorization
-------------	-------------------------------------	------------------------	--------------------------------	---------------------------

Qus 5. Write short notes on the following :

(a) Java Secure Socket Extension (JSSE)

(b) Spring security modules

(c) Role Based Access Control (RBAC)

(d) Cross Site Request Foregery (CSRF)

(e) Hibernate Query Language (HQL) and Insert, Delete, Update, Select queries in HQL

Ans :

(a) Java Secure Socket Extension (JSSE)

♦ **Definition:**

The **Java Secure Socket Extension (JSSE)** is a set of packages in Java that provide **APIs for secure communication** over the network using **SSL (Secure Socket Layer)** and **TLS (Transport Layer Security)** protocols.

♦ **Purpose:**

It ensures **data confidentiality, integrity, and authentication** between client and server during data transmission.

♦ **Key Features:**

- Supports **SSL and TLS** for secure socket communication.
- Provides **encryption and decryption** for data protection.
- Enables **certificate-based authentication**.

- Implements **HTTPS** connections.

♦ **Important Classes:**

- **SSLSocket** and **SSLServerSocket** – for secure communication.
- **SSLContext** – defines environment for SSL/TLS operations.
- **KeyManager** and **TrustManager** – manage certificates and keys.

♦ **Example:**

```
SSLContext context = SSLContext.getInstance("TLS");
```

```
context.init(null, null, null);
```

```
SSLSocketFactory factory = context.getSocketFactory();
```

```
SSLSocket socket = (SSLSocket) factory.createSocket("example.com", 443);
```

(b) Spring Security Modules

♦ **Definition:**

Spring Security is a powerful framework that provides **authentication**, **authorization**, and **protection** against common web attacks.

♦ **Main Modules:**

1. **Core Module**

- Provides basic authentication and authorization APIs.
- Handles security metadata and access control decisions.

2. **Web Module**

- Integrates security with web applications using filters.
- Handles login, logout, and session management.

3. **Config Module**

- Enables Java-based and XML-based configuration of security settings.
- Example: **@EnableWebSecurity**.

4. **OAuth2 / OpenID Module**

- Provides social login and third-party authentication (e.g., Google, Facebook).

5. LDAP Module

- Allows integration with directory-based authentication (Lightweight Directory Access Protocol).

(c) Role-Based Access Control (RBAC)

◆ Definition:

RBAC is a method of restricting system access based on **user roles**. Each role is assigned specific **permissions** or **privileges**, and users are assigned one or more roles.

◆ How It Works:

- **User → Role → Permissions**
- Access is granted based on the **role**, not individual users.

◆ Example (Spring Security Configuration):

```
@Override
protected void configure(HttpSecurity http) throws Exception {
    http.authorizeRequests()
        .antMatchers("/admin/**").hasRole("ADMIN")
        .antMatchers("/user/**").hasAnyRole("USER", "ADMIN")
        .anyRequest().authenticated()
        .and()
        .formLogin();
}
```

◆ Advantages:

- Simplifies security management.
- Easier to maintain and scale.
- Reduces risk of unauthorized access.

(d) Cross-Site Request Forgery (CSRF)

◆ Definition:

CSRF is a web security vulnerability that allows an attacker to trick a logged-in user's browser into making **unwanted requests** to a trusted site.

♦ **Example:**

```

```

If the victim is logged into **bank.com**, this request executes automatically.

♦ **Protection:**

- Use **CSRF Tokens** – unique, secret tokens added to each form.
- Spring Security enables CSRF protection by default.
- Example:

```
http.csrf().enable();
```

♦ **How CSRF Token Works:**

1. Server generates a random token and sends it with a form.
2. The client submits the form with the token.
3. Server verifies the token – if invalid or missing, request is rejected.

(e) Hibernate Query Language (HQL)

♦ **Definition:**

Hibernate Query Language (HQL) is an **object-oriented query language** similar to SQL but works with **Hibernate entities** instead of database tables.

♦ **Features:**

- Independent of the database.
- Uses class names and property names instead of table and column names.
- Supports **CRUD** operations (Create, Read, Update, Delete).

♦ **1. SELECT Query**

Fetches records from an entity.

```
String hql = "FROM Employee WHERE salary > 50000";
```

```
List<Employee> list = session.createQuery(hql).list();
```

♦ 2. INSERT Query

HQL doesn't support direct insert into entity — usually done via saving an object.

```
Employee e = new Employee("Ritesh", 40000);
```

```
session.save(e);
```

♦ 3. UPDATE Query

```
String hql = "UPDATE Employee SET salary = 60000 WHERE id = 1";
```

```
int result = session.createQuery(hql).executeUpdate();
```

♦ 4. DELETE Query

```
String hql = "DELETE FROM Employee WHERE id = 2";
```

```
int result = session.createQuery(hql).executeUpdate();
```

♦ 5. Example – Complete HQL CRUD Flow

```
// Read
List<Employee> employees = session.createQuery("from Employee").list();

// Update
session.createQuery("update Employee set salary=50000 where
id=1").executeUpdate();

// Delete
session.createQuery("delete from Employee where id=3").executeUpdate();
```

✓ Summary Table

Topic	Purpose	Key Point / Example
JSSE	Provides SSL/TLS secure communication	<code>SSLSocket, SSLContext</code>
Spring Security Modules	Handles authentication & authorization	Core, Web, Config, LDAP, OAuth2
RBAC	Grants access based on roles	<code>.hasRole("ADMIN")</code>
CSRF	Prevents forged requests	Uses CSRF token verification
HQL	Object-oriented SQL for Hibernate	<code>FROM Employee WHERE id=1</code>